2015

# Themida en .NET

Snat
[CrackSLatinoS]

# Themida en .NET

CrackSLatinoS

Octubre 2015

# 1. Contenido

1. Contenido ....1
2. Introducción ....2
3. Material necesario ....3
4. Principios Básicos ....4
5. Instalando el Software ....6
6. Protegiendo Olly ....8
7. Primeros pasos ....9
8. Utilizando la Tabla ....13
9. Dumpeando lo indumpeable ....29
10. Batiendo el anti-dump ....38
11. Fijando la variable ....52
12. RadAsm + MASM32 ....56
13. WinLicense al contraataque ....68
14. Último asalto ....78
15. Notas Finales ....86

# 2. Introducción

En el texto de hoy trataremos de desempacar una aplicación desarrollada en .NET, el objetivo será el desempacado en sí sin entrar en detalle acerca de las posibles limitaciones propias que pueda traer la aplicación en la medida de lo posible.

Antes de comenzar quiero decir que es la primera aplicación .NET que trato de desempacar, lo que significará un claro hándicap que intentaremos paliar de alguna manera. Ya veremos qué hacer según las diferentes situaciones en que nos encontremos durante el desarrollo.

También me gustaría decir que considero este texto como la tercera parte de los tutoriales *Themida en Visual Basic* y *Themida en Visual C++* (Teorías número 1567 y 1569 de CrackSLatinoS), ya que se tratará de seguir los métodos allí utilizados. Utilizaremos incluso el mismo script que en la parte 2 (el ampliado), aunque es posible el desempacado sin utilizarlo, este es un caso muy especial. Seguiremos con la misma metodología utilizada en ambos tutoriales.

En este caso veremos que el packer protege lo que puede, si hacemos una comparativa vemos esto:

| | Themida en Visual Basic | Themida en Visual C++ | Themida en .NET |
|---|---|---|---|
| Stolen OEP | x | x | x |
| Virtualización | x | ✓ | x |
| Tratamiento JMP/CALL | ✓ | x | x |
| Más de 1 DLL | x | ✓ | x |

Entonces, no hace nada ¿???

No es que no haga nada sino que teniendo en cuenta las particularidades de los .NET lo protege de otra manera a la que nos tiene acostumbrados. Lo mejor que podemos hacer es pasar a estudiar la protección.

Vamos a ello..

# 3. Material necesario

El software con el que trabajaremos será Direct Access versión 2.8.0.0

www.nagarsoft.com/directaccess/

Durante el tutorial utilizaremos las siguientes herramientas:

- ✓ OllyDbg 1.10 de Oleh Yuschuk
  http://www.ollydbg.de

  Plugins:

  - StrongOD v0.4..8.892 by °£·çôÂó°
  - PhantOm 1.20 by Hellsp@wn & Archer
  - ODbgScript 1.82.6 by Epsylon
  - OllyDump v.3.00.110 de Gigapede

- ✓ RDG Packer Detector v0.7.5 by RDGMax.
- ✓ MegaDumper 1.0 by CodeCracker / SND
- ✓ Universal Fixer 1.0 by CodeCracker / SND
- ✓ RadASM (C) Ketil0 2001-2004
- ✓ Compilador de MASM (Utilizaremos MASM32 v8.2)
- ✓ Explorer Suite IV (para CFF Explorer VIII) de Daniel Pistelli
  http://ntcore.com/exsuite.php
- ✓ ILDASM 2.0.50727.42 de Microsoft
- ✓ .NET Reflector 8.2.0.42 de RedGate

Todas estas tools son descargables del web de Ricardo Narvaja:

http://www.ricardonarvaja.info/WEB/

# 4. Principios Básicos

Antes de continuar daremos un breve repaso a lo que es el .NET Framework, hay muchos textos (y muy buenos) al respecto en la red, y debido a mi poca experiencia en este lenguaje no podemos más que esperar un vistazo superficial del entorno.

Según la teoría la Plataforma .NET consta de tres partes principales:

- ✓ .NET Framework: Entorno de desarrollo.
- ✓ Productos .NET: Aplicaciones basadas en .NET.
- ✓ Servicios .NET: Facilita la creación de servicios en la plataforma .NET

El .NET Framework provee de un entorno de ejecución llamado Common Language Runtime (CLR), el cual se encarga de la ejecución del código del programa, escrito en código con naturaleza de Programación Orientada a Objetos (POO).

Este CLR tiene como función principal dar soporte y manejar la ejecución del código orientado a esta plataforma, asimismo soporta gran variedad de tipos de aplicación, desde aplicaciones Web hasta aplicaciones con interfaz gráfica para Windows.

El código fuente compilado genera unos binarios (a partir del compilador) en código IL (Lenguaje Intermedio, *Intermediate Language*) que posteriormente será traducido a Lenguaje Maquina (mediante el Compilador JIT) en tiempo de ejecución, mediante el CLR.

Este CLR dispone de tres clases de JITters (compiladores JIT):

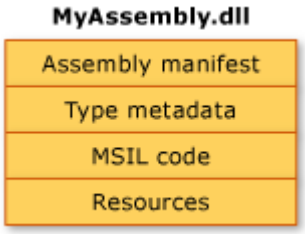
1. JIT Principal (Main JIT): Convierte el código MSIL (*Microsoft Intermediate Language*) a código nativo sin ningún tipo de optimización.
2. Compilador JIT: Este Optimiza el código MSIL.
3. PreJIT: Basado en el Main JIT y funciona como los compiladores tradicionales (compila el código MSIL a código nativo durante la fase de compilación, en lugar de hacerlo en tiempo de ejecución).

El compilador JIT es el encargado, pues, de leer el código MSIL y generar las instrucciones nativas equivalentes. Asimismo, durante la fase de compilación se generan los metadatos los cuales describen los tipos de código y la definición de cada uno de éstos. Los metadatos se almacenan en un archivo PE (en una extensión del formato PE). en él se acomoda además del código los metadatos, lo que permite al Sistema Operativo reconocer la imagen del archivo. Estos metadatos, además, auto describen el código, y serán consultados durante la ejecución (*runtime*) del mismo.

Por otro lado están los *Assemblies*, que son conjuntos de archivos compilados que contienen mínimamente el contenido ejecutable y un archivo MANIFEST en el que se almacenan los datos relacionados con los archivos contenidos en el Assembly, así como la relación de éstos elementos con otros. Estos MANIFEST se pueden guardar o bien en un archivo PE (.exe o .dll) con código MSIL o bien en un archivo PE con esta sola información. Básicamente estos MANIFEST tienen como finalidad el enumerar los archivos que conforman la unidad Assembly, manejar referencias de tipos y recursos asociadas a los archivos que contengan las declaraciones o implementaciones. Enumerar las dependencias que pudieran haber respecto a otros Assemblies, así como dar una autodefinición del propio Assembly.

Una imagen vale más que mil palabras:

- ✓ Archivo único:




- ✓ Multi-archivo:

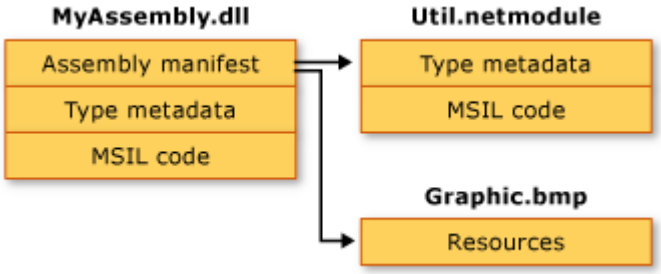



Imagen extraída de https://msdn.microsoft.com/en-us/library/zst29sk2%28v=vs.90%29.aspx

Nos quedamos con que estos Assemblies contienen módulos, éstos contienen clases, que a su vez contienen Métodos, campos, constructores, etc..

Para ensamblar se utiliza Ilasm y para desensamblar Ildasm (que ya conocemos).

Y de teoría poco más, seguimos en la siguiente sección..

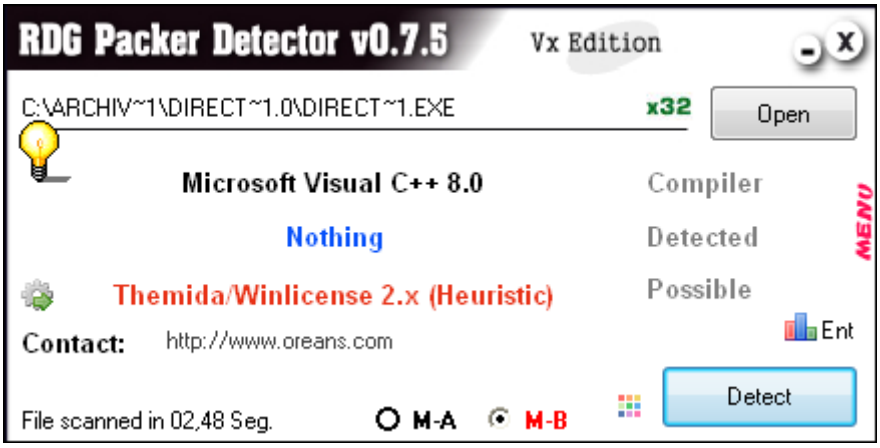
# 5. Instalando el Software

Hecho un primer repaso superficial de la estructura .NET pasamos a instalar el programa en sí.

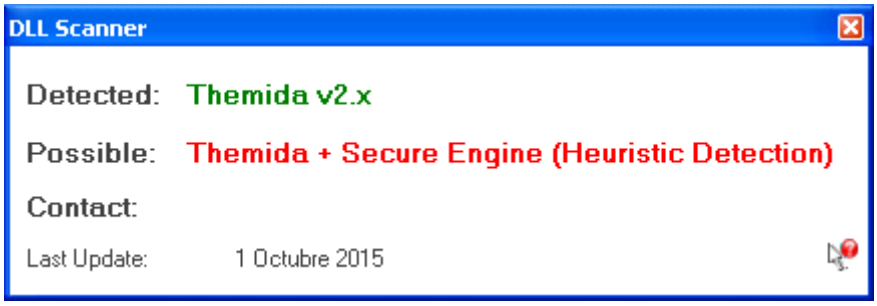
Las siguientes ventanas las pasamos con 'Next' todas hasta completar la instalación.

Cuando termine estamos listos para pasarle RDG Packer Detector al ejecutable DirectAccess.exe a ver qué nos dice:

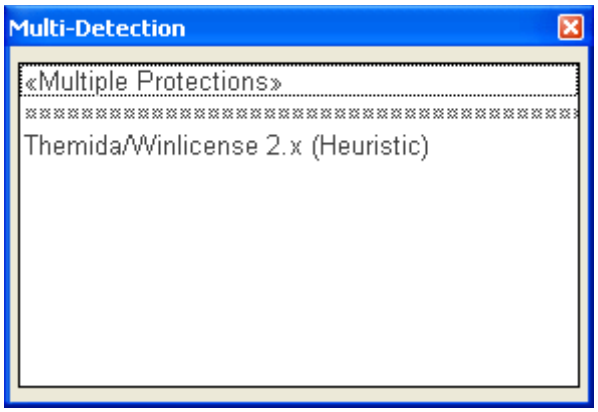
- ✓ Modo M-A: Nada
- ✓ Modo M-B:



Después aparece esta otra ventana:




Por último aparece esta otra ventanita:




Todas esta información apunta a que el ejecutable está empacado con Themida 2.x más WinLicense y/o Secure Engine.

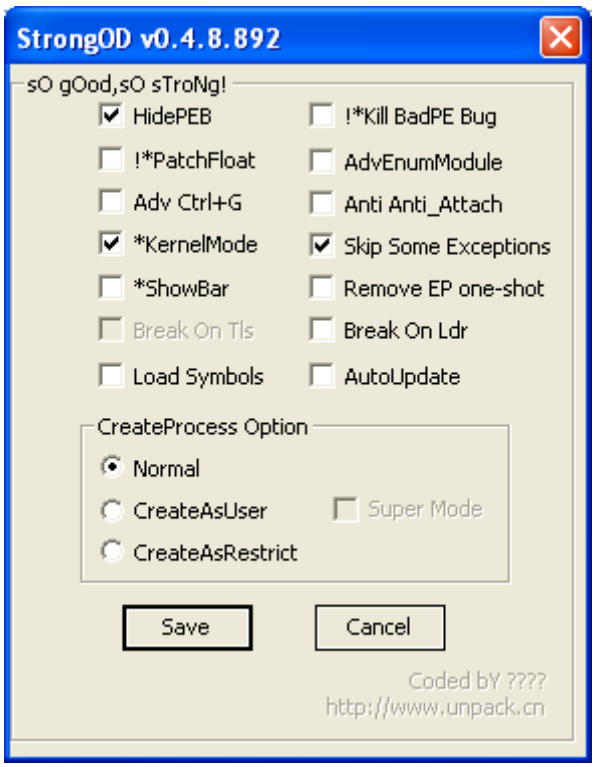
También nos muestra información del compilador: Microsoft Visual C++ 8.0. Pero el tutorial no va de .NET? se preguntarán algunos (no sin razón). Pues sí, así es, aunque el compilador no sea detectado. A favor de RDG he de decir que no he encontrado ningún analizador de ejecutables que lo saque bien (y habré probado como 10 distintos).

De momento tenemos suficiente información.

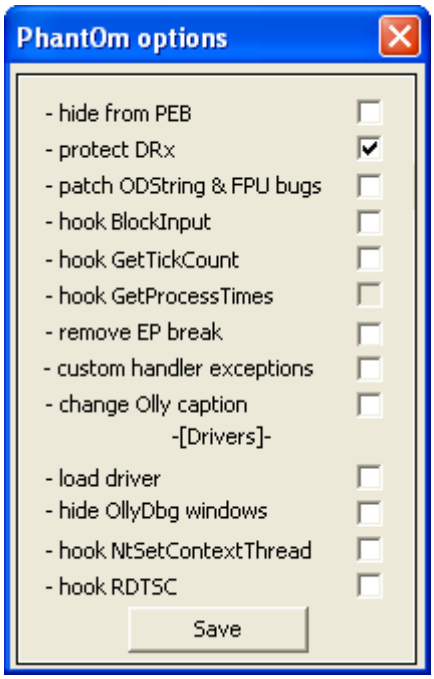
# 6. Protegiendo Olly

Como siempre con este packer tenemos que hacer unos ajustes previos antes de empezar si queremos ejecutarlo bajo OllyDbg.

Configuramos el plugin StrongOD de esta manera:




Y también el PhantOm de esta otra:




Con ello preservaremos los HBP que pongamos, y nos vamos a la siguiente sección..

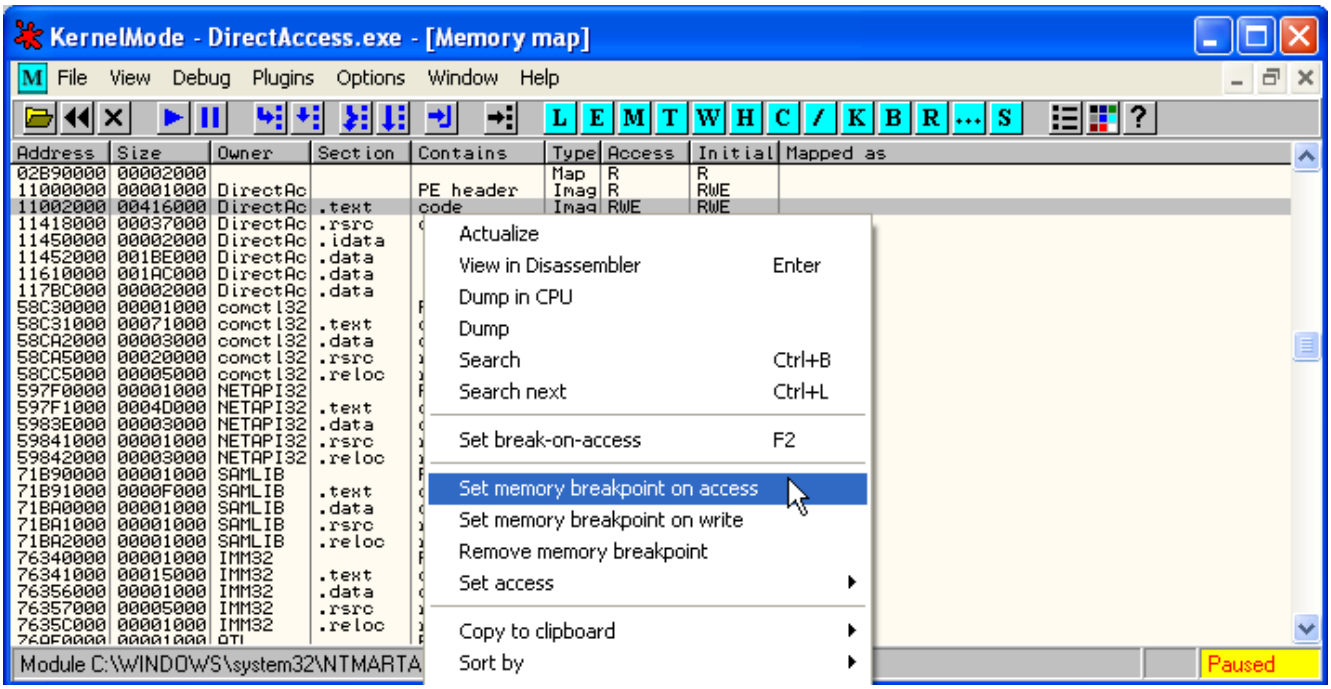
# 7. Primeros pasos

Nos vamos al directorio de instalación del programa y cargamos DirectAccess.exe en Olly:

```
Address   Hex dump        Disassembly                          Comment
<ModuleE  52              push   edx                           ntdll.KiFastSystemCallRet
111B1DD4  BA 64000000     mov    edx, 0x64
111B1DD9  EB 1B           jmp    short 111B1DF6
111B1DDB  B9 00100000     mov    ecx, 0x1000
111B1DE0  EB 05           jmp    short 111B1DE7
111B1DE2  03C1            add    eax, ecx
111B1DE4  03C3            add    eax, ebx
111B1DE6  49              dec    ecx
111B1DE7  0BC9            or     ecx, ecx
111B1DE9  75 F7           jnz    short 111B1DE2
111B1DEB  52              push   edx                           ntdll.KiFastSystemCallRet
111B1DEC  54              push   esp
111B1DED  54              push   esp
111B1DEE  FF15 33004511   call   dword ptr [<&kernel32.lstr    kernel32.lstrcpyA
111B1DF4  5A              pop    edx                           kernel32.7C816037
111B1DF5  4A              dec    edx                           ntdll.KiFastSystemCallRet
111B1DF6  0BD2            or     edx, edx                      ntdll.KiFastSystemCallRet
111B1DF8  75 E1           jnz    short 111B1DDB
111B1DFA  5A              pop    edx                           kernel32.7C816037
111B1DFB  E9 00A26000     jmp    117BC000
111B1E00  0000            add    byte ptr [eax], al
```

Ese es el EntryPoint, tratemos de descubrir el OEP.

Como hicimos en el tute anterior *Themida en Visual C++* colocaremos un BP en el RET de GetProcessHeap, cuando pare quitamos el BP y colocamos un BPM on Access en la sección de código:




Damos RUN y tras varias paradas se detiene en el OEP:

```
Address   Hex dump        Disassembly                          Comment
11416F2E  FF25 00200011   jmp    dword ptr [0x11002000]        mscoree._CorExeMain
11416F34  FF25 00200011   jmp    dword ptr [0x11002000]        mscoree._CorExeMain
11416F3A  0000            add    byte ptr [eax], al
11416F3C  0000            add    byte ptr [eax], al
11416F3E  0000            add    byte ptr [eax], al
11416F40  0000            add    byte ptr [eax], al
11416F42  0000            add    byte ptr [eax], al
11416F44  0000            add    byte ptr [eax], al
11416F46  0000            add    byte ptr [eax], al
11416F48  0000            add    byte ptr [eax], al
```

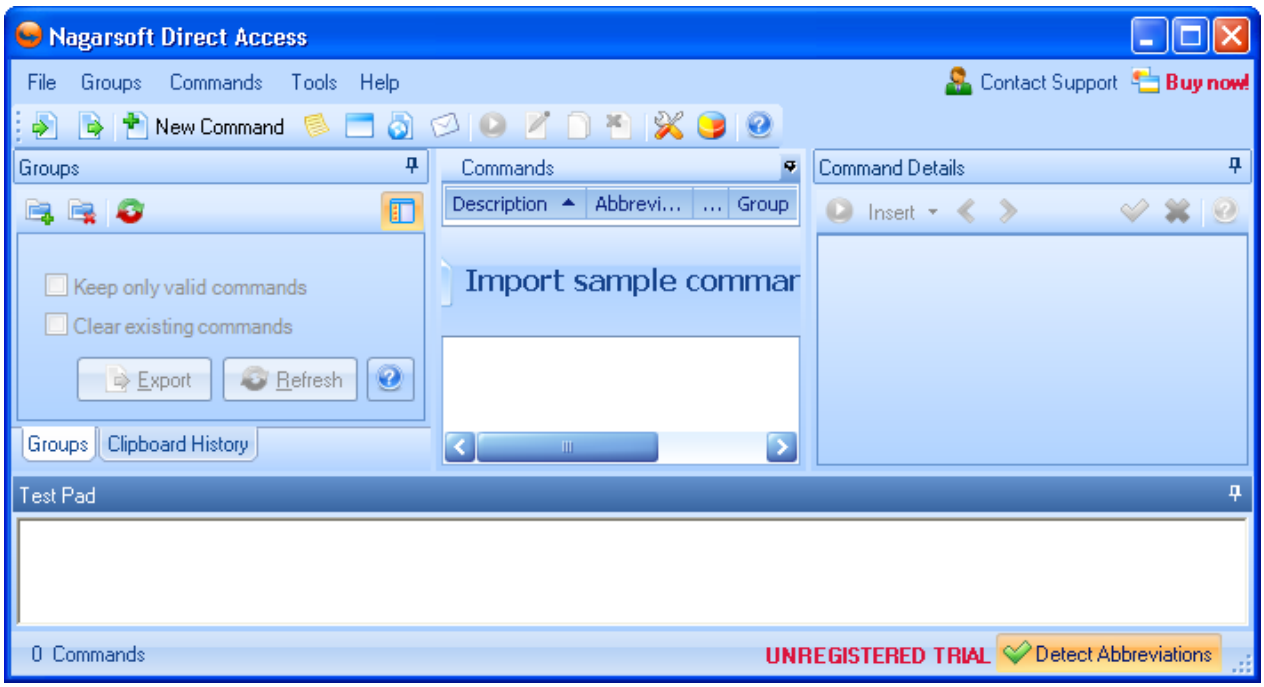
Ya se ven cosas atípicas:

1. El imageBase del archivo es 0x11000000h
2. El OEP está en un JMP a la IAT
3. El salto apunta a la mscoree.dll [identifica a un .NET]

De esto se desprende que no se trata de un ejecutable compilado en Visual C++ sino más bien en .NET

Veamos si corre, demos RUN..

Es un trial de 21 días, pulsemos *Try* a ver qué se esconde tras la puerta:




La ventana principal del programa.

Ya hemos visto suficiente. Cargamos DirectAccess.exe en Olly y colocamos un *BPM on Write* en la sección de código, damos RUN y tras parar en un *rep movsb* se detiene aquí:

```
Address   Hex dump        Disassembly                          Comment
115F872C  890A            mov     dword ptr [edx], ecx         mscoree._CorExeMain
115F872E  5A              pop     edx                          DirectAc.11002000
115F872F  FC              cld
115F8730  AD              lodsd                                // ** lee campo_JMP
115F8731  60              pushad
115F8732  50              push    eax                          DirectAc.11002000
115F8733  53              push    ebx                          DirectAc.115F8FC6
115F8734  58              pop     eax                          DirectAc.11002000
115F8735  5B              pop     ebx                          DirectAc.115F8FC6
115F8736  66:81C2 C4F4    add     dx, 0xF4C4
115F873B  61              popad
```

Está escribiendo en la IAT:

```
Address   Hex dump                                         ASCII
11002000  DB 4D 00 79 00 00 00 00 48 00 00 00 02 00 05 00  .M.y....H.......
11002010  EC 65 39 00 F4 08 08 00 03 00 00 00 61 10 00 06  .e9.........a...
11002020  F8 28 06 00 F3 3C 33 00 00 00 00 00 00 00 00 00  .(...<3.........
11002030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
11002040  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
11002050  13 30 03 00 29 00 00 00 01 00 00 11 02 7B 12 00  .0..)........{..
11002060  00 04 0A 06 0B 07 03 28 2A 00 00 0A 74 2A 00 00  .......(*...t*..
11002070  01 0C 02 7C 12 00 00 04 08 07 28 01 00 00 2B 0A  ...|......(...+.
11002080  06 07 33 DF 2A 00 00 00 13 30 03 00 29 00 00 00  ..3.*....0..)...
11002090  01 00 00 11 02 7B 12 00 00 04 0A 06 0B 07 03 28  .....{..........(
```

Esta es la dirección de la API *mscoree._CorExeMain* que tenía en ECX, nuevamente <F9>y caemos aquí:

```
Address   Hex dump        Disassembly                          Comment
1146B47D  66:8F02         pop     word ptr [edx]
1146B480  ^ E9 4DE0FEFF   jmp     114594D2
1146B485  01F7            add     edi, esi                     DirectAc.11602C30
1146B487  5E              pop     esi                          DirectAc.11602C30
1146B488  FF3424          push    dword ptr [esp]
1146B48B  5B              pop     ebx
1146B48C  56              push    esi                          DirectAc.11602C30
1146B48D  89E6            mov     esi, esp
Stack [0012FF62]=000025FF
ds:[11416F34]=0000
```

Está construyendo un salto. Pulsamos <F9> y para aquí:

```
Address   Hex dump        Disassembly                          Comment
11461BF1  8F02            pop     dword ptr [edx]              DirectAc.11002000
11461BF3  ^ E9 DA78FFFF   jmp     114594D2
11461BF8  5B              pop     ebx                          DirectAc.11002000
11461BF9  v E9 F1990000   jmp     1146B5EF
11461BFE  F7DF            neg     edi                          DirectAc.114585E9
11461C00  51              push    ecx
11461C01  57              push    edi                          DirectAc.114585E9
11461C02  68 0044586C     push    0x6C584400
Stack [0012FF60]=11002000 (DirectAc.11002000)
ds:[11416F36]=00000000
```

Pasamos la instrucción con <F7> y miramos cómo quedó:

```
Address   Hex dump        Disassembly                          Comment
11416F2E  - FF25 00200011 jmp     dword ptr [0x11002000]       mscoree._CorExeMain
11416F34  - FF25 00200011 jmp     dword ptr [0x11002000]       mscoree._CorExeMain
11416F3A  0000            add     byte ptr [eax], al
11416F3C  0000            add     byte ptr [eax], al
11416F3E  0000            add     byte ptr [eax], al
11416F40  0000            add     byte ptr [eax], al
11416F42  0000            add     byte ptr [eax], al
11416F44  0000            add     byte ptr [eax], al
11416F46  0000            add     byte ptr [eax], al
11416F48  0000            add     byte ptr [eax], al
```

Escribió el segundo salto (el de abajo) que apunta a la misma api.

Colocamos un BP común con <F2> en ambos JMP y dejamos correr el proceso con <F9>, tras varias paradas por escritura (sin aparente interés) se detiene en el primero de ellos:

```
Address   Hex dump          Disassembly                          Comment
11416F2E  - FF25 00200011   jmp     dword ptr [0x11002000]       mscoree._CorExeMain
11416F34  - FF25 00200011   jmp     dword ptr [0x11002000]       mscoree._CorExeMain
11416F3A    0000            add     byte ptr [eax], al
11416F3C    0000            add     byte ptr [eax], al
11416F3E    0000            add     byte ptr [eax], al
11416F40    0000            add     byte ptr [eax], al
11416F42    0000            add     byte ptr [eax], al
11416F44    0000            add     byte ptr [eax], al
ds:[11002000]=79004DDB (mscoree._CorExeMain)
```

Éste será nuestro OEP. Lo anotamos:

- ✓ OEP: 0x11416F2Eh

Y nos vamos a estudiar la rutina de la IAT donde hace uso de la tabla en el siguiente punto.

# 8. Utilizando la Tabla

Volvamos al principio, reiniciamos Olly y colocamos un *BPM on Write* en la sección de código, vamos pulsando <F9> hasta que pare por escritura en la IAT (2ª parada):

```
Address   Hex dump   Disassembly                          Comment
115F872C  890A       mov     dword ptr [edx], ecx         mscoree._CorExeMain
115F872E  5A         pop     edx
115F872F  FC         cld
115F8730  AD         lodsd                                // ** lee campo_JMP
115F8731  60         pushad
115F8732  50         push    eax                          DirectAc.11002000
115F8733  53         push    ebx                          DirectAc.115F8FC6
115F8734  58         pop     eax                          DirectAc.11002000
ds:[esi]=[115EF969]=C0082DE5
eax=11002000

Address   Hex dump                                        ASCII
115EF959  0B 89 03 E9 28 00 00 00 00 00 00 00 00 00 00 00  ....(...........
115EF969  E5 2D 08 C0 AA AA AA AA FF FF FF FF DD DD DD DD  .-..............
115EF979  EE EE EE EE DD DD DD DD 00 00 00 00 00 00 00 00  ................
115EF989  E9 3D 00 00 00 60 8B BD 60 4B 09 10 83 EF 04 31  .=...`..`K.....1
115EF999  C9 89 C8 BB 20 83 B8 ED 83 C7 04 BE 08 00 00 00  .... ...........
115EF9A9  D1 E8 19 D2 21 DA 31 D0 4E 0F 85 F1 FF FF FF 83  ....!.1.N.......
115EF9B9  C1 01 89 07 81 F9 00 01 00 00 0F 82 D1 FF FF FF  ................
115EF9C9  61 C3 89 C1 8D 85 7A D9 1D 10 89 85 0C 51 0C 10  a.....z......Q..
115EF9D9  E8 0C 00 00 00 E7 70 C0 9E D2 B0 03 BA B5 D8 A1  ......p.........
115EF9E9  B0 8B 8D E2 1B 04 10 5E 0F 8F 1C 00 00 00 E8 09  .......^........
```

Hacemos un *Follow in Dump -> Memory Address* sobre el *lodsd* y encontramos la Tabla de la IAT, el primer campo (el campo_API) ocupa la posición 0x115EF961h que el packer borró. Colocamos un *HBP On Access -> Byte* en esta dirección, reiniciamos Olly, damos RUN, y descubrimos donde lee el campo_API:

```
Address   Hex dump   Disassembly                          Comment
115F5779  F9         stc
115F577A  AD         lodsd                                // ** lee campo_API
115F577B  F9         stc                                  // ** pueden ser EEEEEEEE
115F577C  89B5 ADFA1410  mov     dword ptr [ebp+0x1014FAAD]  DirectAc.115EF965
115F5782  60         pushad
115F5783  0F8C 02000000  jl      115F578B
115F5789  60         pushad
115F578A  61         popad
ds:[esi]=[115EF965]=35D60500
eax=BC2C8D46
```

Ahí está, ESI apunta al siguiente campo (se modificó en el *lodsd*), esta tabla la conocemos de los anteriores textos ,no profundizaremos, recorreremos la rutina donde escribe la IAT anotando únicamente los datos que necesitamos para el script (que también tenemos escrito de las anteriores partes). Llegamos aquí:

```
Address   Hex dump   Disassembly                          Comment
115F58A8  80E3 5B    and     bl, 0x5B
115F58AB  8BF8       mov     edi, eax
115F58AD  61         popad
115F58AE  3385 8D500910  xor     eax, dword ptr [ebp+0x1009508D]  // ** XOR con llave
115F58B4  E9 0C000000    jmp     115F58C5                         // ** encriptacion
115F58B9  90         nop
115F58BA  BB 5AC34A35    mov     ebx, 0x354AC35A
115F58BF  F8         clc
ss:[114A70A1]=00000000
eax=BC2C8D46
```

Donde encripta el campo_API con la actual llave de encriptación, en este caso es 0x00h ya que se trata del primer Registro (y único) de la tabla.

Seguimos con <F7> y llegamos a la primera operación con el campo_API:

```
Address   Hex dump   Disassembly                          Comment
115F58C5  C1C8 03    ror     eax, 0x3                     // ** 1a operacion campo_API
115F58C8  E9 0E000000    jmp     115F58DB
115F58CD  2E:C3      ret                                  Superfluous prefix
115F58CF  4A         dec     edx
115F58D0  E3 42      jecxz   short 115F5914
115F58D2  C0BA B3C45D54  sar     byte ptr [edx+0x545DC4B  Shift constant out of range 1
115F58D9  D4 CF      aam     0xCF
115F58DB  51         push    ecx
```

La anotamos:

ROR BC2C8D46, 3 == D78591A8

Seguimos traceando y llega a la segunda operación:

```
Address   Hex dump     Disassembly                  Comment
115F58F1   81F1 79C2E0A2 xor     ecx, 0xA2E0C279
115F58F7   29C8          sub     eax, ecx
115F58F9   59            pop     ecx
115F58FA   29D0          sub     eax, edx            // ** 2a operacion campo_API
115F58FC   05 F13CA35C   add     eax, 0x5CA33CF1     // ** EDX==CONSTANTE1 >> 15E8D954
115F5901 v E9 13000000   jmp     115F5919
115F5906   FB            sti
115F5907   27            daa
edx=15E8D954  <------ CONSTANTE1
eax=7AE254B7
```

Es importante también porque en este punto obtenemos el valor de la CONSTANTE1 que utiliza para desencriptar en esta función. La operación queda de esta manera:

SUB 7AE254B7, 15E8D954 == 64F97B63

Valor que reajusta en la siguiente línea quedando EAX==0xC19CB854h. Ha utilizado un par de operaciones superfluas para enmascarar el resultado:

```
115F58F7    29C8            sub     eax,  ecx          ; // ** ECX==0x5CA33CF1h
115F58F9    59              pop     ecx
115F58FA    29D0            sub     eax, edx           ; // ** 2a operación campo_API
115F58FC    05 F13CA35C     add     eax, 0x5CA33CF1    ; // ** EDX==CONSTANTE1 >> 15E8D954
115F5901    E9 13000000     jmp     115F5919
```

Luego llegan dos operaciones casi consecutivas:

```
Address   Hex dump     Disassembly                  Comment
115F5919   C1C0 10       rol     eax, 0x10           // ** 4a operacion campo_API
115F591C v 0F88 01000000 js      115F5923
115F5922   F5            cmc
115F5923   31C8          xor     eax, ecx            // ** ECX==CONSTANTE2 >> 453F7FFA
115F5925   F5            cmc
115F5926   899D 8D500910 mov     dword ptr [ebp+0x1009508D], ebx  // ** instala llave encriptacion
115F592C v 0F8E 01000000 jle     115F5933
115F5932   F8            clc
```

La primera es la 4ª operación aplicada al campo_API.

Hizo lo siguiente:

ROL C19CB854, 10 == B854C19C

La que viene después en 0x115F5923h es un XOR con ECX.

Quedando:

XOR B854C19C, 453F7FFA == FD6BBE66 >> semilla_API

Además en ECX encontramos el valor de la CONSTANTE2==0x453F7FFAh.

También establece como nueva llave de encriptación el valor de EBX:

```
Address   Hex dump     Disassembly                                  Comment
115F5919    C1C0 10    rol     eax, 0x10                            // ** 4a operacion campo_API
115F591C  v 0F88 01000000 js    115F5923
115F5922    F5         cmc
115F5923    31C8       xor     eax, ecx                             // ** ECX==CONSTANTE2 >> 453F7FFA
115F5925    F5         cmc
115F5926    899D 8D500910 mov   dword ptr [ebp+0x1009508D], ebx      // ** instala llave encriptacion
115F592C  v 0F8E 01000000 jle   115F5933
115F5932    F8         clc
ebx=BC2C8D46  <------ Nueva llave encriptación
ss:[114A70A1]=00000000
```

El contenedor está en la dirección 0x114A70A1h.

Este resultado (0xFD6BBE66h) es el valor_Semilla de la API que valorará después:

```
Address   Hex dump     Disassembly                                  Comment                    Registers (FPU)
115F592C  v 0F8E 01000000 jle   115F5933                                                        EAX FD6BBE66
115F5932    F8         clc                                                                      ECX 453F7FFA
115F5933    3D 00000100 cmp     eax, 0x10000                         // ** valor semilla bajo ? EDX 15E8D954
115F5938  v 0F83 38020000 jnb   115F5B76                                                        EBX BC2C8D46
115F593E    60         pushad                                                                   ESP 0012FF6C
115F593F    66:BE 3691 mov      si, 0x9136                                                      EBP 01412014
115F5943  v E9 07000000 jmp     115F594F                                                        ESI 115EF965 DirectAc.
115F5948    51         push     ecx                                                            EDI 115F2004 DirectAc.
Jump is taken                                                                                   EIP 115F5938 DirectAc.
115F5B76=115F5B76
```

Lo compara con 0x10000h y si NO es inferior (como en este caso) salta hacia 0x115F5B76h donde la procesará para el caso General (Busca la API por nombre, como ya vimos en anteriores tutoriales).

Si seguimos traceando llegamos al bucle donde busca la semilla en la tabla de semillas:

```
Address   Hex dump     Disassembly                                  Comment
115F5BA5    8B95 0E1B0 mov      edx, dword ptr [ebp+0x10041B0E]     // ** puntero a tabla semillas
115F5BAB    FC         cld
115F5BAC    3B02       cmp      eax, dword ptr [edx]                // ** semillas iguales ?
115F5BAE  v 0F84 FF000 je       <Semilla_encontrada>
115F5BB4    60         pushad
115F5BB5    60         pushad
115F5BB6    60         pushad
115F5BB7    61         popad
ss:[11453B22]=02550000  <------ Puntero a Tabla Semillas
edx=15E8D954
```

Es muy importante la dirección del puntero de esta tabla. Veámosla:

```
Address   Hex dump                                                ASCII
02550000  8C F5 FD 6E DA 6F 95 53 74 C3 4B 08 5C AC D5 17         ...n.o.St.K.\...
02550010  AB 1F 62 D3 2B 96 89 CE DB 64 F5 32 A0 2F 6C 61         ..b.+....d.2./la
02550020  72 A3 57 DD F6 63 07 94 D8 4C C4 5E 46 C3 EC 6C         r.W..c...L.^F..l
02550030  8B 82 CC 45 CA 50 FE A1 E4 61 10 8A 4F B9 20 74         ...E.P...a..O. t
02550040  51 FF 3F AC FF 71 EE 0A 1D 57 94 37 96 13 7B E9         Q.?..q...W.7..{.
02550050  2B 62 44 BE 28 C0 66 16 F0 B6 A1 8D EC 40 C0 56         +bD.(.f......@.V
02550060  4C 3B A6 5D A0 DE 80 1E 85 BC 07 C0 4F E7 49 6C         L;.]........O.Il
02550070  C6 47 0E F2 89 F0 A6 77 47 DA 13 53 9D 02 30 F7         .G.....wG..S..0.
02550080  37 C4 5F 28 28 D0 D8 C3 E5 C4 2F A8 5A D5 97 D3         7._((...../.Z...
02550090  25 26 B6 27 96 17 C4 1A 68 60 64 F1 27 B2 DF 5F         %&.'....h`d.'..
```

Nos quedamos con la dirección del puntero:

- ✓ Puntero a Tabla Semillas: 0x11453B22h

Y lo anotamos. Lo siguiente es ver qué valor obtuvo tras el loop: 0x77h que después se guarda:

```
Address   Hex dump     Disassembly                                  Comment
<Semilla    898D 761B0 mov      dword ptr [ebp+0x10041B76], ecx     // ** salva posicion semilla
115F5CB9  v E9 0F00000 jmp      115F5CCD
115F5CBE    15 F298015 adc      eax, 0x590198F2
115F5CC3    8E07       mov      es, word ptr [edi]                  Modification of segment registe
115F5CC5    CB         retf                                         Far return
115F5CC6    22E0       and      ah, al
115F5CC8    3F         aas
115F5CC9    FD         std
ecx=00000077  <------ Posición Semilla en la Tabla
ss:[11453B8A]=0000007B
```

Significa que la semilla ocupa la posición 0x77h (119d) en la tabla de Semillas y que utilizará posteriormente para obtener la posición de ésta en el Array AddressOfNames.

Pero antes hace una triple comprobación con la base de la DLL (mscoree.dll):

```
Address   Hex dump   Disassembly                          Comment
115F5CF6    3B8D 5A1A0 cmp     ecx, dword ptr [ebp+0x10041A5A] // ** user32.dll ?
115F5CFC  v 74 10      je      short 115F5D0E
115F5CFE    3B8D 05170 cmp     ecx, dword ptr [ebp+0x10041705] ADVAPI32.77DA0000
115F5D04  v 74 08      je      short 115F5D0E                  // ** advapi32.dll ?
115F5D06    3B8D 25140 cmp     ecx, dword ptr [ebp+0x10041425] kernel32.7C800000
115F5D0C  v 75 31      jnz     short 115F5D3F                  // ** kernel32.dll ?
115F5D0E    C685 FD001 mov     byte ptr [ebp+0x101E00FD], 0x1
115F5D15    8D95 0A011 lea     edx, dword ptr [ebp+0x101E010A]
ss:[11453A6E]=7E390000 (USER32.7E390000)
ecx=79000000 (mscoree.#141)
```

Revisa si se trata del kernel, user o advapi (para virtualizarlas).

Después opera con el valor semilla en la tabla de semillas (índice_Semilla):

```
Address   Hex dump   Disassembly                          Comment
115F5DEF    D1E0       shl     eax, 1                      // ** 1a operacion posicion_Semilla
115F5DF1  v 0F89 0D000 jns     115F5E04
115F5DF7  v E9 0800000 jmp     115F5E04
115F5DFC    22E0       and     ah, al
115F5DFE    4F         dec     edi                         DirectAc.115F2004
115F5DFF    B1 99      mov     cl, 0x99
115F5E01    22E1       and     ah, cl
115F5E03    F2:        prefix repne:                       Superfluous prefix
eax=00000077  <-- Posición Semilla
```

Recordemos: el dword 0xFD6BBE66h lo encontró en la posición 0x77h dentro de la tabla:

```
Address   Hex dump                                         ASCII
025501DC  66 BE 6B FD 2F 14 5E 9B 15 C8 12 E3 D5 05 77 E5  f.k./.^.......w.
025501EC  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
025501FC  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0255020C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0255021C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0255022C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0255023C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0255024C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0255025C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0255026C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Llegamos a esta dirección 0x25501DCh mediante la siguiente operación:

Base_Tabla + (posición_Semilla * 4) >>
0x02550000h + (0x77h * 4) >>
0x02550000h + 0x1DCh >>
0x02550000h + 0x1DCh >> <u>0x25501DCh</u>

Por esto el packer va multiplicando constantemente mediante SHL.

Anotamos el resultado de la 1ª operación con la posicion_Semilla de la dirección 0x115F5DEFh

SHL 00000077, 1 == 000000EE

Este valor 0x000000EEh es el desplazamiento dentro del array AddressOfNameOrdinals de la DLL, por esto le agrega la dirección base del array 0x79001674h en la siguiente parada:

```
Address   Hex dump     Disassembly                                  Comment
115F5E28    61         popad
115F5E29    0385 61E80 add     eax, dword ptr [ebp+0x100CE861]      // ** AddressOfNameordinal
115F5E2F  v E9 0E00000 jmp     115F5E42
115F5E34    DA74E0 53  fidiv   dword ptr [eax+0x53]
115F5E38    DBFA       fstp    edx                                  Illegal use of register
115F5E3A    6220       bound   esp, qword ptr [eax]
115F5E3C    3940 55    cmp     dword ptr [eax+0x55], eax
115F5E3F    AD         lodsd
ss:[114E0875]=79001674 (mscoree.79001674)   <- Base array AddressOfNameOrdinals
eax=000000EE   <- Desplazamiento
```

Multiplicó x2 el valor de la posicion_Semilla porque este array AddressOfNameOrdinals es un array de Words (2 bytes).

Anotamos la operación:

ADD 000000EE, 79001674 == 79001762

Vayamos allí con Olly:

```
Address   Hex dump                                          ASCII
79001762  79 00 78 00 7A 00 7B 00 6D 73 63 6F 72 65 65 2E   y.x.z.{.mscoree.
79001772  64 6C 6C 00 43 4C 52 43 72 65 61 74 65 49 6E 73   dll.CLRCreateIns
79001782  74 61 6E 63 65 00 43 61 6C 6C 46 75 6E 63 74 69   tance.CallFuncti
79001792  6F 6E 53 68 69 6D 00 43 6C 6F 73 65 43 74 72 73   onShim.CloseCtrs
790017A2  00 43 6C 72 43 72 65 61 74 65 4D 61 6E 61 67 65   .ClrCreateManage
790017B2  64 49 6E 73 74 61 6E 63 65 00 43 6F 45 45 53 68   dInstance.CoEESh
790017C2  75 74 44 6F 77 6E 43 4F 4D 00 43 6F 49 6E 69 74   utDownCOM.CoInit
790017D2  69 61 6C 69 7A 65 43 6F 72 00 43 6F 49 6E 69 74   ializeCor.CoInit
790017E2  69 61 6C 69 7A 65 45 45 00 43 6F 55 6E 69 6E 69   ializeEE.CoUnini
790017F2  74 69 61 6C 69 7A 65 43 6F 72 00 43 6F 55 6E 69   tializeCor.CoUni
```

Obtiene el word 0x0079h, si a este valor le agregáramos el valor Base de la Tabla de Exportación:

```
[ Structure Lister ]
->Export Table
   Characteristics:        0x00000000
   TimeDateStamp:          0x4AF3AF84   (GMT: Fri Nov 06 05:09:24 2009)
   MajorVersion:           0x0000
   MinorVersion:           0x0000   -> 0.00
   Name:                   0x0000176A   ("mscoree.dll")
   Base:                   0x00000011   <-
   NumberOfFunctions:      0x0000007E
```

Hacemos la suma:

ADD 0079, 0011 == 008A

Y buscamos este ordinal:

```
0x0084   0x000169A9 "StrongNameTokenFromAssembly"
0x0085   0x00016A84 "StrongNameTokenFromAssemblyEx"
0x0086   0x00016B76 "StrongNameTokenFromPublicKey"
0x0087   0x00019C7F "TranslateSecurityAttributes"
0x0088   0x00015030 "_CorDllMain"
0x0089   0x0001A981 "_CorExeMain2"
0x008A   0x00004DDB "_CorExeMain"   <-
0x008B   0x0002033B "_CorImageUnloading"
```

Lo encontramos, se le asocia la función *_CorExeMain* con offset 0x00004DDBh dentro de la DLL, pero este no es el procedimiento seguido por el packer, está buscando por Nombre (no por ordinal).

```
Address   Hex dump     Disassembly                    Comment
115F5E62    FC         cld
115F5E63    61         popad
115F5E64    61         popad
115F5E65    66:AD      lodsw                          // ** lee de tabla AddressOfNameOrdinals
115F5E67  v 0F89 08000 jns      115F5E75
115F5E6D  v 0F8C 02000 jl       115F5E75
115F5E73    60         pushad
115F5E74    61         popad
ds:[esi]=[79001762]=0079
ax=0000
```

Traceamos la rutina con <F7> y vemos qué otras operaciones hace:

```
Address   Hex dump     Disassembly                    Comment
115F5E67  v 0F89 08000 jns      115F5E75
115F5E6D  v 0F8C 02000 jl       115F5E75
115F5E73    60         pushad
115F5E74    61         popad
115F5E75    FC         cld
115F5E76    C1E0 02    shl      eax, 0x2              // ** 2a operacion con posicion_Semilla
115F5E79  v E9 0500000 jmp      115F5E83              // ** reajusta indice
115F5E7E    4D         dec      ebp
eax=00000079
```

Opera con la posicion_Semilla, haciendo:

SHL 00000079, 2 == 000001E4

Acaba de obtener el desplazamiento (0x000001E4h) dentro del array AddressOfFunctions, sumándole después la dirección de inicio de este array:

```
Address   Hex dump     Disassembly                    Comment
115F5E83    0385 9F441 add      eax, dword ptr [ebp+0x1017449F]  // ** add AddressOfFunctions
115F5E89    FC         cld
115F5E8A    31F0       xor      eax, esi              mscoree.79001764
115F5E8C    31C6       xor      esi, eax
115F5E8E    31F0       xor      eax, esi              mscoree.79001764
115F5E90    F5         cmc
115F5E91  v E9 0600000 jmp      115F5E9C
ss:[115864B3]=79001290 (mscoree.79001290)   Base array AddressOfFunctions
eax=000001E4   Desplazamiento
```

Quedará:

ADD 000001E4, 79001290 == 79001474

Obtiene así el offset de la API en la Librería:

```
Address   Hex dump                                        ASCII
79001474  DB 4D 00 00 3B 03 02 00 D0 42 00 00 00 00 00 00 .M..;....B......
79001484  4D 4C 01 00 76 17 00 00 88 17 00 00 99 17 00 00 ML..v...........
79001494  A3 17 00 00 BC 17 00 00 CC 17 00 00 DC 17 00 00 ................
790014A4  EB 17 00 00 FD 17 00 00 0E 18 00 00 1A 18 00 00 ................
790014B4  32 18 00 00 43 18 00 00 59 18 00 00 70 18 00 00 2...C...Y...p...
790014C4  89 18 00 00 9C 18 00 00 B1 18 00 00 C2 18 00 00 ................
790014D4  D1 18 00 00 DB 18 00 00 EA 18 00 00 04 19 00 00 ................
790014E4  11 19 00 00 24 19 00 00 48 19 00 00 58 19 00 00 ....$...H...X...
790014F4  68 19 00 00 7A 19 00 00 8C 19 00 00 A0 19 00 00 h...z...........
```

Donde está el valor 0x00004DDBh, es un dword, de ahí la multiplicación *2. Otra vez <F9> y llegamos aquí:

```
Address   Hex dump     Disassembly                    Comment
115F5E9C    AD         lodsd                          // ** obtiene offset API
115F5E9D    60         pushad
115F5E9E    60         pushad
115F5E9F    60         pushad
115F5EA0    61         popad
115F5EA1    66:81CA E8 or       dx, 0x1FE8
ds:[esi]=[79001474]=00004DDB   Offset API
eax=79001764
```

Dejando el resultado en EAX==0x00004DDBh al que le agrega el ImageBase de la DLL justo después:

```
Address   Hex dump   Disassembly                    Comment
115F5EB1    61       popad
115F5EB2    2D A477D65 sub     eax, 0x55D677A4
115F5EB7    05 425AED6 add     eax, 0x6DED5A42
115F5EBC    01C8     add       eax, ecx              // ** le agrega imageBase
115F5EBE    2D 425AED6 sub     eax, 0x6DED5A42
115F5EC3    05 A477D65 add     eax, 0x55D677A4
ecx=79000000 (mscoree.#141)
eax=18173079
```

Quedando:

ADD 00004DDB, 79000000 == 79004DDB

Si miramos el panel auxiliar vemos que es la dirección de *mscoree._CorExeMain*:

```
ecx=79000000 (mscoree.#141)
eax=79004DDB (mscoree._CorExeMain)
```

API en EAX

Que dejó en EAX.

Una vez que ha resuelto la dirección de la api podemos hacer un resumen de lo que ha hecho:

| OPERACIÓN | | RESULTADO |
|---|---|---|
| **XOR campo_API, llave_encriptacion** | == | valor1 |
| **ROR valor1, 3** | == | valor2 |
| **SUB valor2, 15E8D954** | == | valor3 >> Utiliza CONSTANTE1 |
| **ROL valor3, 10** | == | valor4 |
| **XOR valor4, 453F7FFA** | == | semilla_API >> Utiliza CONSTANTE2 |
| **posición semilla_API en tabla_Semillas** | == | indice_Semilla |
| **SHL indice_Semilla, 1** | == | valor5 |
| **ADD valor5, AddressOfNameOrdinals** | == | valor6 |
| **MOV EAX, dword ptr [valor6]** | == | valor7 |
| **SHL valor7, 2** | == | valor8 |
| **ADD valor8, AddressOfFunctions** | == | offset API |
| **ADD offset API, Base DLL** | == | dirección API |

Son EXACTAMENTE las MISMAS operaciones que utilizó Themida en la desencriptación de la IAT para EventMeister, el programa que estudiamos en la anterior parte.

Después de esto le toca el turno al campo_IAT:

```
Address   Hex dump   Disassembly                    Comment
115F85DE    60       pushad
115F85DF    61       popad
115F85E0    AD       lodsd                         // ** lee campo_IAT
115F85E1    60       pushad
115F85E2    8BCB     mov       ecx, ebx            DirectAc.115F8FC6
115F85E4    B8 AB5F216 mov     eax, 0x64215FAB
ds:[esi]=[115EF965]=35D60500
eax=79004DDB
```

Valor campo IAT

Quedó en EAX==0x35D60500h, el siguiente de la Tabla:

```
Address   Hex dump                                          ASCII
115EF961  00 00 00 00 00 05 D6 35 E5 2D 08 C0 AA AA AA AA  ........5.-......
115EF971  FF FF FF FF DD DD DD DD EE EE EE EE DD DD DD DD  ................
115EF981  00 00 00 00 00 00 00 00 E9 3D 00 00 00 60 8B BD  .........=...`..
115EF991  60 4B 09 10 83 EF 04 31 C9 89 C8 BB 20 83 B8 ED  `K.....1.... ...
115EF9A1  83 C7 04 BE 08 00 00 00 D1 E8 19 D2 21 DA 31 D0  ............!.1.
115EF9B1  4E 0F 85 F1 FF FF FF 83 C1 01 89 07 81 F9 00 01  N...............
115EF9C1  00 00 0F 82 D1 FF FF FF 61 C3 89 C1 8D 85 7A D9  ........a.....z.
115EF9D1  1D 10 89 85 0C 51 0C 10 E8 0C 00 00 00 E7 70 C0  .....Q........p.
```

Lo vigilamos a ver qué operaciones le aplica, antes pasa por esta zona:

```
Address   Hex dump    Disassembly                          Comment
115F8664    80BD FD001 cmp     byte ptr [ebp+0x101E00FD], 0x0 // ** flag virtualizacion activo ?
115F866B  v 0F84 6A000 je      115F86DB
115F8671    F5         cmc
115F8672    83BD 06011 cmp     dword ptr [ebp+0x101E0106], 0x0 // ** flag excepcion activo ?
115F8679  v 0F85 5C000 jnz     115F86DB
115F867F  v E9 1300000 jmp     115F8697
115F8684    6A 61      push    0x61
115F8686    7A 43      jpe     short 115F86CB
115F8688    AA         stosb
115F8689    2F         das
115F868A    F3:        prefix rep:                          Superfluous prefix
115F868B    B9 68A9787 mov     ecx, 0x7178A968
115F8690    5D         pop     ebp
115F8691    CE         into
115F8692    5F         pop     edi                          mscoree._CorExeMain
115F8693    C0DA 40    rcr     dl, 0x40                     Shift constant out of range 1..31
115F8696    4E         dec     esi                          DirectAc.115EF969
115F8697    FC         cld
115F8698    FF95 044B0 call    dword ptr [ebp+0x10094B04]   // ** Funcion Virtual
115F869E    FC         cld
115F869F  v E9 0B00000 jmp     115F86AF
115F86A4    47         inc     edi                          mscoree._CorExeMain
115F86A5  v 72 51      jb      short 115F86F8
115F86A7    131D 19FAF adc     ebx, dword ptr [0xB1F8FA19]
ss:[115F2111]=00
```

Es la entrada a la Función Virtual que vimos en el texto anterior donde sí entraba, en este caso NO entra, ejecutándose el JE de 0x115F8664h que nos envía aquí:

```
Address   Hex dump    Disassembly                          Comment
115F86DB    C1C0 05    rol     eax, 0x5                     // ** 1a operacion campo_IAT
115F86DE    60         pushad
115F86DF  v 0F84 00000 je      115F86E5
115F86E5    61         popad
115F86E6    56         push    esi                          DirectAc.115EF969
115F86E7    53         push    ebx                          DirectAc.115F8FC6
115F86E8    BB FA7F3F4 mov     ebx, 0x453F7FFA
115F86ED    89DE       mov     esi, ebx                     DirectAc.115F8FC6
115F86EF    5B         pop     ebx                          DirectAc.115F8FC6
115F86F0    01F0       add     eax, esi                     // ** 2a operacion campo_IAT
115F86F2    5E         pop     esi                          // ** ESI==CONSTANTE2 >> 453F7FFA
115F86F3  v 0F8C 07000 jl      115F8700
115F86F9  v 0F86 01000 jbe     115F8700
115F86FF    FC         cld
115F8700    0385 E1130 add     eax, dword ptr [ebp+0x100413E1] // ** 3a operacion campo_IAT
115F8706  v E9 0A00000 jmp     115F8715                     // ** agrega imageBase
```

Donde le aplica tres operaciones al campo_IAT, quedando:

ROL 35D60500, 5 == BAC0A006
ADD BAC0A006, 453F7FFA == 00002000
ADD 00002000, 11000000 == 11002000 >> <u>Dirección de la IAT</u>

Después recuperará la API y la escribe en la dirección de la IAT recién desencriptada:

```
Address   Hex dump    Disassembly                          Comment
115F8715    8B8D AA161 mov     ecx, dword ptr [ebp+0x101B16AA] mscoree._CorExeMain
115F871B    FC         cld
115F871C    81EC 04000 sub     esp, 0x4
115F8722    891424     mov     dword ptr [esp], edx
115F8725    BA 0000000 mov     edx, 0x0
115F872A    01C2       add     edx, eax                     DirectAc.11002000
115F872C    890A       mov     dword ptr [edx], ecx
ss:[115C36BE]=79004DDB (mscoree._CorExeMain)
ecx=00000002
```

Vamos a la dirección 0x11002000h y allí la encontramos:

```
Address   Hex dump                                          ASCII
11002000  DB 4D 00 79 00 00 00 00 48 00 00 00 02 00 05 00  .M.y....H.......
11002010  EC 65 39 00 F4 08 08 00 03 00 00 00 61 10 00 06  .e9.........a...
11002020  F8 28 06 00 F3 3C 33 00 00 00 00 00 00 00 00 00  .(...<3.........
11002030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
11002040  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
11002050  13 30 03 00 29 00 00 00 01 00 00 11 02 7B 12 00  .0..).........{..
11002060  00 04 0A 06 0B 07 03 28 2A 00 00 0A 74 2A 00 00  .......(*...t*..
11002070  01 0C 02 7C 12 00 00 04 08 07 28 01 00 00 2B 0A  ...|......(...+.
```

Una vez que ha escrito la API obtendrá el valor del campo_JMP:

```
Address   Hex dump   Disassembly                  Comment
115F872F  FC         cld
115F8730  AD         lodsd                        // ** lee campo_JMP
115F8731  60         pushad
115F8732  50         push    eax                  DirectAc.11002000
115F8733  53         push    ebx                  DirectAc.115F8FC6
115F8734  58         pop     eax                  DirectAc.11002000
ds:[esi]=[115EF969]=C0082DE5
eax=11002000
```

Si hacemos un *Follow in Dump -> Memory address* sobre el *lodsd* vemos que efectivamente lee de la Tabla:

```
Address   Hex dump                                          ASCII
115EF969  E5 2D 08 C0 AA AA AA AA FF FF FF FF DD DD DD DD  .-..............
115EF979  EE EE EE EE DD DD DD DD 00 00 00 00 00 00 00 00  ................
115EF989  E9 3D 00 00 00 60 8B BD 60 4B 09 10 83 EF 04 31  .=...'..'K.....1
115EF999  C9 89 C8 BB 20 83 B8 ED 83 C7 04 BE 08 00 00 00  .... ...........
115EF9A9  D1 E8 19 D2 21 DA 31 D0 4E 0F 85 F1 FF FF FF 83  ....!.1.N.......
115EF9B9  C1 01 89 07 81 F9 00 01 00 00 0F 82 D1 FF FF FF  ................
115EF9C9  61 C3 89 C1 8D 85 7A D9 1D 10 89 85 0C 51 0C 10  a.....z......Q..
115EF9D9  E8 0C 00 00 00 E7 70 C0 9E D2 B0 03 BA B5 D8 A1  ......p.........
```

Le toca el turno al valor 0xC0082DE5h que quedará en EAX y revisa si debe procesarlo después:

```
Address   Hex dump   Disassembly                  Comment
115F8781  83F8 FF    cmp     eax, -0x1
115F8784  0F85 98000 jnz     115F8822             // ** tratamos el salto ?
115F878A  0F82 18000 jb      115F87A8
115F8790  E9 1300000 jmp     115F87A8
115F8795  63E2       arpl    dx, sp
115F8797  A1 B51E291 mov     eax, dword ptr [0x1F291E
115F879C  E5 59      in      eax, 0x59            I/O command
115F879E  9E         sahf
```

En este caso EAX != -0x1h y por tanto se ejecuta el SALTO y nos deja directamente aquí:

```
Address   Hex dump   Disassembly                  Comment
115F8822  C1C0 03    rol     eax, 0x3             // ** 1a operacion campo_JMP
115F8825  F5         cmc
115F8826  0385 E1130 add     eax, dword ptr [ebp+0x100413E1]  // ** 2a operacion campo_JMP
115F882C  F9         stc
115F882D  83BD 99070 cmp     dword ptr [ebp+0x10040799], 0x1
115F8834  0F84 5E050 je      115F8D98
115F883A  F5         cmc
```

Donde aplica dos operaciones para desencriptar el campo_JMP:

ROL C0082DE5, 3 == BAC0A006
ADD 00416F2E, 11000000 == 11416F2E >> <u>Dirección del SALTO</u>

Por último revisa si debe reconstruir el salto:

```
Address   Hex dump   Disassembly                  Comment
115F8A49  FF95 4C680 call    dword ptr [ebp+0x1009684C]  DirectAc.11547AA9
115F8A4F  F5         cmc
115F8A50  803F 90    cmp     byte ptr [edi], 0x90 // ** romper el salto ?
115F8A53  0F84 4A000 je      115F8AA3
115F8A59  0F88 1F000 js      115F8A7E
115F8A5F  60         pushad
115F8A60  E8 1400000 call    115F8A79
115F8A65  1C 4F      sbb     al, 0x4F
ds:[11416F2E]=FF
```

Mira si el primer byte es 0xFFh, vayamos allí y veámoslo:

```
Address   Hex dump    Disassembly                          Comment
11416F2E  - FF25 00200011 jmp     dword ptr [0x11002000]   mscoree._CorExeMain
11416F34    0000      add     byte ptr [eax], al
11416F36    0000      add     byte ptr [eax], al
11416F38    0000      add     byte ptr [eax], al
11416F3A    0000      add     byte ptr [eax], al
11416F3C    0000      add     byte ptr [eax], al
11416F3E    0000      add     byte ptr [eax], al
11416F40    0000      add     byte ptr [eax], al
```

No va a hacer nada puesto que ya hay un salto allí.

De aquí pasa a leer el siguiente campo de la Tabla [0x115EF971h]==0xFFFFFFFFh y después el siguiente [0x115EF975h]==0xDDDDDDDDh como se ve en la siguiente captura:

```
Address   Hex dump    Disassembly                          Comment
115F87A3  - E9 7ABFFC3 jmp      465C4722
115F87A8    813E DDDDD cmp      dword ptr [esi], 0xDDDDDDDD  // ** campo con D's ?
115F87AE  v 0F85 6E000 jnz      115F8822
115F87B4  v E9 1000000 jmp      115F87C9
115F87B9  - E9 DCCD00F jmp      0560559A
115F87BE    50         push     eax
115F87BF    D092 8AB6E rcl      byte ptr [edx+0x6BEEB68A], 1
115F87C5    25 2648515 and      eax, 0x50514826
```

El JNZ no se ejecuta y el ciclo actual termina.

Después para en la lectura del campo_API, que en este caso son EEEEEEEE:

```
Address   Hex dump    Disassembly                          Comment
115F57B9    60         pushad
115F57BA    81C8 CF014 or       eax, 0x2F4201CF
115F57C0    66:BE 2434 mov      si, 0x3424
115F57C4    61         popad
115F57C5    3D EEEEEEE cmp      eax, 0xEEEEEEEE              // ** siguiente campo son E's ?
115F57CA  v 0F85 C0000 jnz      115F5890
115F57D0  v E9 1400000 jmp      115F57E9
115F57D5    96         xchg     eax, esi                     DirectAc.115EF97D
eax=EEEEEEEE
```

El salto NO se ejecuta, por último revisa las DDDDDDDD que quedan en la Tabla:

```
Address   Hex dump    Disassembly                          Comment
115F57E9    813E DDDDD cmp      dword ptr [esi], 0xDDDDDDDD  // ** campo con D's ?
115F57EF  v 0F85 9B000 jnz      115F5890
115F57F5    60         pushad
115F57F6  v E9 0A00000 jmp      115F5805
115F57FB    E5 7D      in       eax, 0x7D                    I/O command
115F57FD    60         pushad
115F57FE    D10E       ror      dword ptr [esi], 1
115F5800    F74479 A4  test     dword ptr [ecx+edi*2-0x5C], 0x
ds:[115EF97D]=DDDDDDDD
```

Tampoco se ejecuta el JNZ y salimos por el JMP. Pasa a ejecutar otra zona de la rutina desde la que llama a la api ntdll.ZwFreeVirtualMemory para liberar el bloque de memoria 0x02550000h donde estaba la Tabla de Semillas. Finaliza los trabajos en este salto:

```
Address   Hex dump    Disassembly                          Comment
115F4F07    83F9 00    cmp      ecx, 0x0                     // ** ultima DLL ?
115F4F0A  v 0F84 A3400 je       115F8FB3
115F4F10    60         pushad
115F4F11    B3 46      mov      bl, 0x46
115F4F13    E8 1400000 call     115F4F2C
115F4F18    58         pop      eax                          mscoree.79004DE9
115F4F19  ^ E1 F1      loopde   short 115F4F0C
115F4F1B    831A 51    sbb      dword ptr [edx], 0x51
```

Esa es la SALIDA, ha terminado.

Sólo hay una API, es una micro-IAT (la más pequeña que he visto a día de hoy).

A partir de aquí comienza a auto-encriptarse:

```
Address   Hex dump     Disassembly                          Comment
115FC009    60         pushad
115FC00A    C4A1 9704A les     esp, fword ptr [ecx+0x7AAF0497]   Modification of segment
115FC010    1C E8      sbb     al, 0xE8
115FC012    47         inc     edi                              DirectAc.115FC02B
115FC013    67:9D      popfd                                    Superfluous prefix
115FC015    0286 5601F add     al, byte ptr [esi+0x17F80156]
115FC01B    C459 E0    les     ebx, fword ptr [ecx-0x20]        Modification of segment
115FC01E    CF         iretd
115FC01F    0B8F 10C1E or      ecx, dword ptr [edi+0xEFE0C110]
115FC025    CE         into
115FC026    BF 9F99142 mov     edi, 0x2C14999F
115FC02B    108D 8D18A adc     byte ptr [ebp+0x1EA0188D], cl
115FC031    10B8 44661 adc     byte ptr [eax+0x32156644], bh
115FC037    29F9       sub     ecx, edi                         DirectAc.115FC02B
115FC039    C1E9 02    shr     ecx, 0x2
115FC03C    3107       xor     dword ptr [edi], eax
115FC03E    D0C8       ror     al, 1
115FC040    83C7 04    add     edi, 0x4
115FC043    49         dec     ecx
115FC044  ^ 75 F6      jnz     short 115FC03C
115FC046    60         pushad
115FC047    81C9 39959 or      ecx, 0x3D999539
115FC04D    61         popad
115FC04E  v E9 0E00000 jmp     115FC061
115FC053    C7         ???                                      Unknown command

Address   Hex dump                                          ASCII
115FBF8F  B8 61 C5 FA 03 2F 60 CA FD A8 5B E0 7D CE 5F CF  .a.../'...[.}._.
115FBF9F  02 23 B7 63 B6 B3 D5 FC 36 3E A7 A4 92 8F 10 C1  .#.c....6>......
115FBFAF  2C EF CC B9 F9 20 02 36 32 ED 98 D7 BF 6F 05 8A  ,.... .62....o..
115FBFBF  32 30 27 83 6D 9F D4 DB 20 57 12 E2 D9 E5 D2 36  20'.m... W.....6
115FBFCF  C1 13 E3 D6 9E D9 33 83 7F 30 54 8D 28 D5 14 A1  ......3..0T.(...
115FBFDF  3E C6 18 32 99 9C 72 B1 CB 63 E6 96 9C DB 30 F5  >..2..r..c....0.
115FBFEF  95 76 98 BF D2 AB 08 22 9A DA 6F CF 64 4F EC F3  .v.....".."..o.dO..
```

Si seguimos traceando veremos que tras llamar a kernel32.SetEvent se queda en un loop:

```
Address   Hex dump     Disassembly                          Comment
115FC1CA    3A72 F5    cmp     dh, byte ptr [edx-0xB]
115FC1CD    6A 00      push    0x0
115FC1CF    FF95 A9180 call    dword ptr [ebp+0x100418A9]
115FC1D5    89C0       mov     eax, eax
115FC1D7  ^ EB F4      jmp     short 115FC1CD
115FC1D9    66:48      dec     ax
115FC1DB    1C BA      sbb     al, 0xBA
115FC1DD    4E         dec     esi                              DirectAc.115EE8E1
```

De aquí no parece salir nunca, si entramos en el call de la imagen veremos que hace una llamada a la función *ntdll.RtlActivateActivationContextUnsafeFast* y a *ntdll.ZwDelayExecution*, si tenemos paciencia, mucha paciencia, al final llegaremos al salto al OEP:

```
Address   Hex dump     Disassembly                          Comment
1146084F    61         popad
11460850    9D         popfd
11460851    C3         ret
11460852  ^ E9 41E4FFF jmp     1145EC98
11460857    01E9       add     ecx, ebp
11460859    81E9 56167 sub     ecx, 0x307F1656
1146085F    5D         pop     ebp                              DirectAc.11416F2E
11460860    81C1 02000 add     ecx, 0x2
11460866    51         push    ecx
Return to 11416F2E (DirectAc.11416F2E)  <---- OEP !!!

Address   Hex dump     Disassembly                          Comment
11416F2E  - FF25 00200011 jmp     dword ptr [0x11002000]    mscoree._CorExeMain
11416F34  - FF25 00200011 jmp     dword ptr [0x11002000]    mscoree._CorExeMain
11416F3A    0000       add     byte ptr [eax], al
11416F3C    0000       add     byte ptr [eax], al
11416F3E    0000       add     byte ptr [eax], al
11416F40    0000       add     byte ptr [eax], al
11416F42    0000       add     byte ptr [eax], al
```

Si pasamos el RET caemos en el primero de los dos saltos a *mscoree._CorExeMain*.

Con esto nos aseguramos de que el OEP sea el correcto, tenemos dos métodos de llegar al mismo lugar: el primero nos lo brinda la rutina de la IAT donde encontramos la dirección de _CorExeMain, desde allí obtenemos la dirección de la IAT, con un BPM on Access descubrimos el OEP, y de esta otra manera: Traceando con <F7> en código propio y <F8> en llamadas a funciones de sistema.

Hecho esto, y en vista de que sólo hay una importación podemos hacer una pasada por la zona del packer y obtener los datos necesarios para el script, de todos los que necesitamos nos faltan son éstos:

1. Puntero a la Tabla de Bases de DLL
2. Puntero a la Tabla de encriptación
3. Dirección de la Tabla de semillas de excepción

El tercer valor no es necesario porque no hay virtualización pero lo buscaremos también.

Traceando la rutina de la IAT desde la dirección del *lodsd* donde lee el campo_API encontramos el valor del puntero a la tabla de Bases de DLL aquí:

```
Address   Hex dump   Disassembly                          Comment
115F5CEA   85C9       test    ecx, ecx
115F5CEC   74 51      je      short 115F5D3F
115F5CEE   8B8D 81160 mov     ecx, dword ptr [ebp+0x10041681]  // ** Puntero Tabla Bases DLL
115F5CF4   8B09       mov     ecx, dword ptr [ecx]
115F5CF6   3B8D 5A1A0 cmp     ecx, dword ptr [ebp+0x10041A5A]  // ** user32.dll ?
115F5CFC   74 10      je      short 115F5D0E
115F5CFE   3B8D 05170 cmp     ecx, dword ptr [ebp+0x10041705]  ADVAPI32.77DA0000
115F5D04   74 08      je      short 115F5D0E                   // ** advapi32.dll ?
ss:[11453695]=023E0000
ecx=00000001
```

Lo anotamos:

- ✓ Puntero a Tabla de Bases de DLL: 0x11453695h

Si traceamos la rutina unas líneas después encontramos la dirección de la tabla de semillas de excepción:

```
Address   Hex dump   Disassembly                          Comment
115F5CFE   3B8D 05170 cmp     ecx, dword ptr [ebp+0x10041705]  ADVAPI32.77DA0000
115F5D04   74 08      je      short 115F5D0E                   // ** advapi32.dll ?
115F5D06   3B8D 25140 cmp     ecx, dword ptr [ebp+0x10041425]  kernel32.7C800000
115F5D0C   75 31      jnz     short 115F5D3F                   // ** kernel32.dll ?
115F5D0E   C685 FD001 mov     byte ptr [ebp+0x101E00FD], 0x1
115F5D15   8D95 0A011 lea     edx, dword ptr [ebp+0x101E010A]
115F5D1B   8D8D E2011 lea     ecx, dword ptr [ebp+0x101E01E2]
115F5D21   39CA       cmp     edx, ecx                         mscoree.#141
Address=115F211E
edx=025501DC
```

Hagamos un *Follow in Dump -> Memory Address* sobre la instrucción para confirmarlo:

```
Address   Hex dump                                         ASCII
115F211E  81 71 F5 40 C3 C5 FE 5B AF 06 8A 0D 93 D1 86 56  .q.@...[.......V
115F212E  F7 19 7B 25 20 AC E3 39 80 24 57 6C 29 02 B8 7D  ..{% ..9.$Wl)..}
115F213E  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
115F214E  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
115F215E  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
115F216E  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
115F217E  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
115F218E  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Lo anotamos también:

- ✓ Dirección Tabla de Semillas de Excepción: 0x115F211Eh

Nos queda uno nada más: La dirección del puntero a la Tabla de encriptación.

Este puede ser el menos obvio, sabemos que tiene al menos 2 puntos distintos donde trabaja con la tabla de encriptación:

1. En la rutina de comprobación de integridad de la rutina de la IAT.
2. En la rutina de creación de semillas.

El primer punto se alcanza si el contador de ciclos de la rutina de la IAT alcanza el valor 0x64h (100d), sólo hay que tracear la rutina hasta encontrar el contador.

El segundo punto se alcanza cuando termina de procesar una DLL y va a por la siguiente, liberará la memoria de la tabla de semillas (que después reubicará) en el momento de crear la tabla de semillas para la siguiente DLL tendrá que acceder a la Tabla de Encriptación, aquí podemos encontrar el puntero.

Por facilidad nos quedamos con la primera, reiniciamos Olly y colocamos un BPM en la dirección de la comparación de ciclos 0x115F5620h, pulsamos <F9> para llegar allí:

```
Address   Hex dump    Disassembly                          Comment
115F5620  83BD 79080  cmp     dword ptr [ebp+0x10040879], 0x64  // ** alcanzados 100 ciclos ?
115F5627  0F82 0F010  jb      115F573C
115F562D  60          pushad
115F562E  E9 1000000  jmp     115F5643
115F5633  9C          pushfd
115F5634  34 64       xor     al, 0x64
115F5636  8A2F        mov     ch, byte ptr [edi]
115F5638  17          pop     ss                                Modification of segment register
```

En este momento el contador vale 0x79h (valor mayor que 0x64h) y el salto NO se produce. Pero si sólo hay una importación como es que el contador vale 0x79h? La respuesta está en la rutina de creación de semillas (justo de donde viene) en la que también utiliza este contador.

Seguimos traceando y en seguida encontramos lo que andamos buscando:

```
Address   Hex dump    Disassembly                          Comment
115F565B  8DB5 FF011  lea     esi, dword ptr [ebp+0x101E01FF]
115F5661  8DBD DBA01  lea     edi, dword ptr [ebp+0x101EA0DB]
115F5667  29F7        sub     edi, esi                     DirectAc.115F2213
115F5669  89FA        mov     edx, edi
115F566B  8BBD 604B0  mov     edi, dword ptr [ebp+0x10094B60]  // ** ptr Tabla semillas excepcion
115F5671  B9 FFFFFFFF mov     ecx, -0x1
115F5676  31C0        xor     eax, eax                     // ** chequeo CRC
115F5678  8A06        mov     al, byte ptr [esi]
ss:[114A6B74]=024D0000
edi=00009EDC
```

Lo anotamos:

- ✓ <u>Puntero a Tabla de Semillas de Excepción</u>: 0x114A6B74h

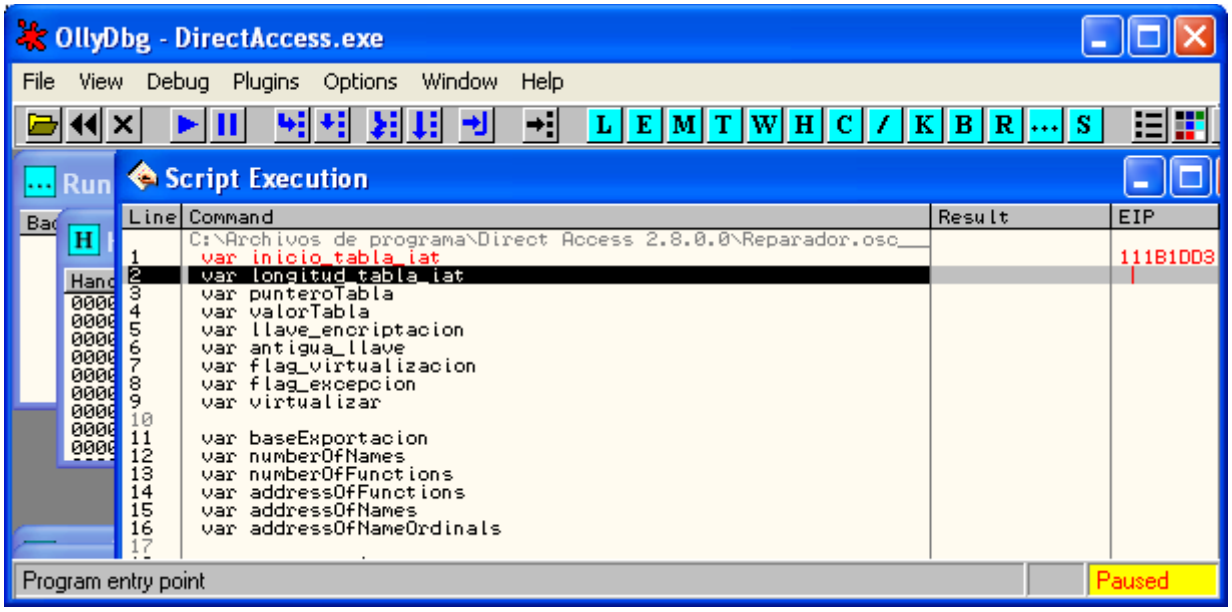
Y ahora nos hacemos una pequeña tabla con todos los valores necesarios para el script:

| Variable | | Valor |
|---|---|---|
| constante1 | == | 15E8D954 |
| constante2 | == | 453F7FFA |
| | | |
| inicio_tabla_iat | == | 0x115EF961h |
| longitud_tabla_iat | == | 0x20h |
| | | |
| direccion_lodsd | == | 0x115F577Ah |
| | | |
| puntero_tabla_bases_DLL | == | 0x11453695h |
| puntero_tabla_encriptacion | == | 0x114A6B74h |
| tabla_semillas_excepcion | == | 0x115F2E11Eh |
| | | |
| OEP | == | 0x11416F2Eh |

Colocamos estos valores en el script:

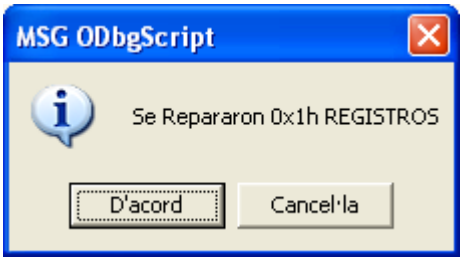
```
41 mov constante1, 15E8D954
42 mov constante2, 453F7FFA
43 mov constante3, 0
44 mov constante4, 0
45 mov constante5, 0
46 mov constante6, 0
47
48 mov inicio_tabla_iat, 115EF961
49 mov longitud_tabla_iat, 20
50 mov llave_encriptacion, 0
51 mov antigua_llave, 0
52
53 mov tabla_bases_DLL, 11453695
54 mov direccion_lodsb, 115F577A
55 mov puntero_tabla_encriptacion, 114A6B74
. . .
80 mov oep, 11416F2E
```

Le dejamos el mismo nombre *Reparador.osc* y lo cargamos en el Olly:

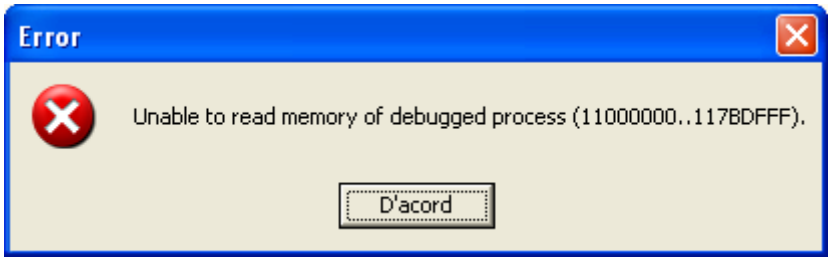



Nos aseguramos de no tener puestos ni BPs ni HBPs ni ningún BreakPoint de cualquier otro tipo y lo dejamos correr (con la Barra Espaciadora), veremos que se queda detenido en el OEP, sabremos que está trabajando porque el valor de los registros va cambiando, además desde la ventana del script (Script Window) también vemos que está en un loop (creando las semillas).

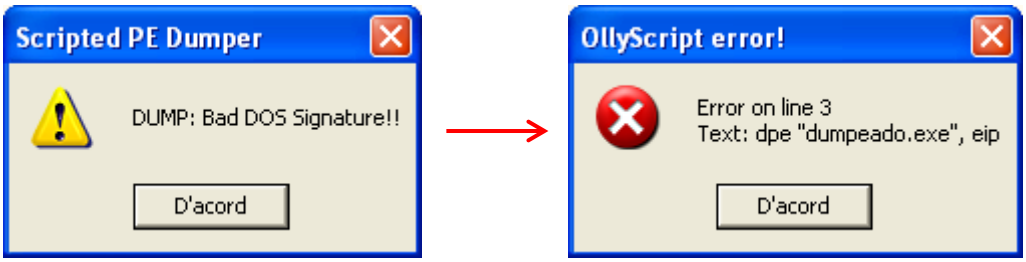
4 minutos con 17 segundos más tarde aparece este cartelito:



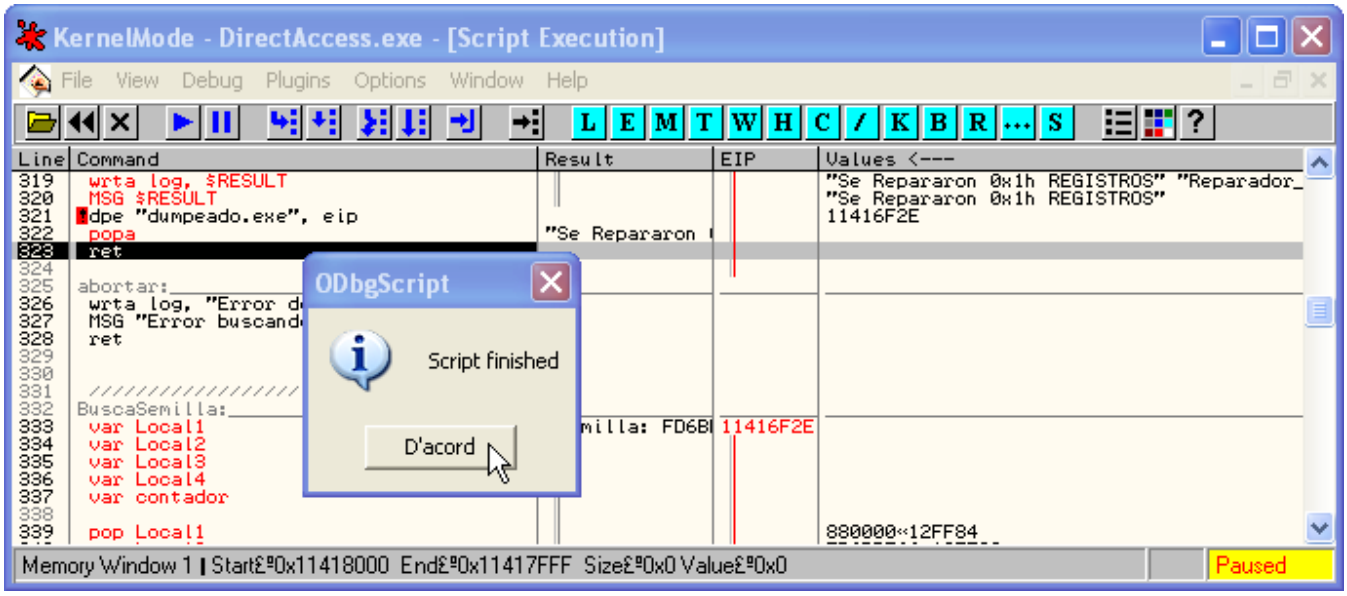
Pulsamos OK y aparece otro cartel:



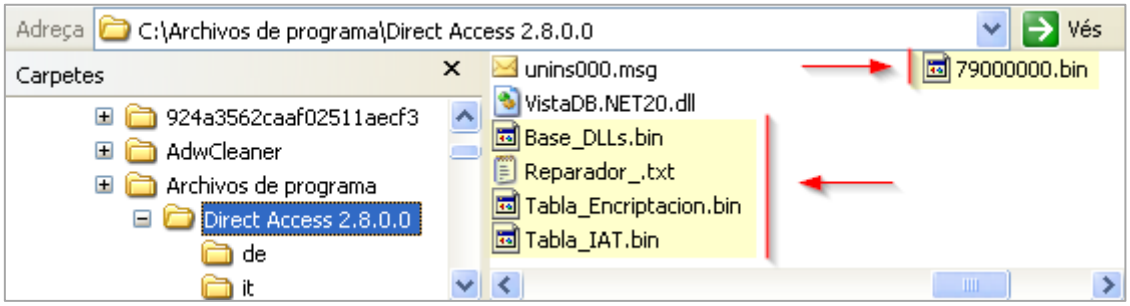

Lo cerramos y aparecen más:




El script queda pausado tras el comando *dpe* que trataba de dumpear el proceso




Pasamos la instrucción y al llegar al RET aparece el cartel de fin del script. Si miramos en el directorio de instalación encontramos las tablas pero no el dumpeado.



Tenemos un archivo *Log* donde está la información que ha ido dejando el script en el directorio de instalación como se ve en la imagen anterior (Reparador_.txt), lo abrimos con Notepad++ y lo vemos:

```
1 ///////////// **********                ****************
2 ///////////// ********** Inicio SCRIPT ****************
3  Tabla semillas 0x880000h
4  Tabla Encriptación en: 0x2530000h
5
6  Actualizando nueva DLL 0x79000000h
7  Informacion DLL
8  ----------------------
9  Tabla Exportacion : 79001268
10 Base Exportacion : 11
11 NumberOfFunctions : 7E
12 NumberOfNames : 7B
13 AddressOfFunctions : 79001290
14 AddressOfNames : 79001488
15 AddressOfNameOrdinals: 79001674
16
17 el archivo C:\Archivos de programa\Direct Access 2.8.0.0\79000000.bin NO existe
18 Creando Semillas para: 0x79000000
19 ********** Tabla Semillas creadas para DLL 0x79000000
20
21 ///////////// ********** Informacion REGISTRO 0x1h
22 punteroTabla: 115EF961
23 semilla: FD6BBE66
24 posicion semilla: 77
25 EsKernel para 79000000 termina con valor: FFFFFFFF
26 Es_Ntdll para 79004DDB termina con valor: FFFFFFFF
27 api: mscoree._CorExeMain
28 Desencripta IAT obtiene: 0x11002000h
29 Repara IAT mov [11002000], 79004DDB
30 EsCampoValido para C0082DE5 termina con valor: 1
31 Desencripta Instruccion obtiene: 0x11416F2Eh
32 opcode instruccion 0xFFh
33 EsCampoValido para AAAAAAAA termina con valor: 0
34 EsCampoValido para FFFFFFFF termina con valor: 0
35 Se Repararon 0x1h REGISTROS
```

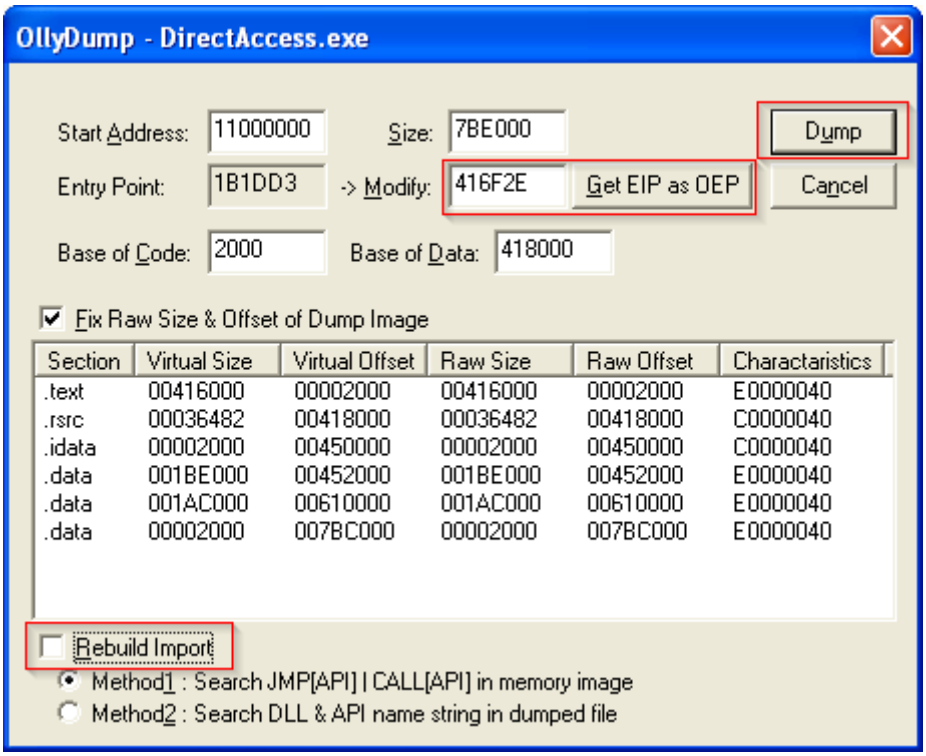
Como todo, es información que siempre nos puede valer.

Nos vamos, pues, a la siguiente sección a ver si sacamos un dumpeado válido..

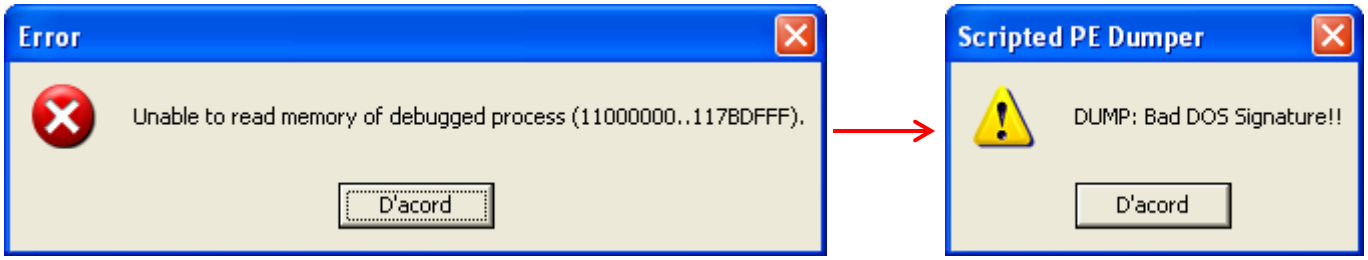
# 9. Dumpeando lo indumpeable

En esta sección veremos los aspectos a tener en cuenta a la hora de dumpear un .NET, en mi caso es el primero que intento y voy a mostrar el modo que encontré para hacerlo y los hostiazos que me di por el camino, será interesante.

Trabajamos con el proceso detenido en el OEP, primero probaremos con el plugin OllyDump con las opciones que solemos utilizar habitualmente:



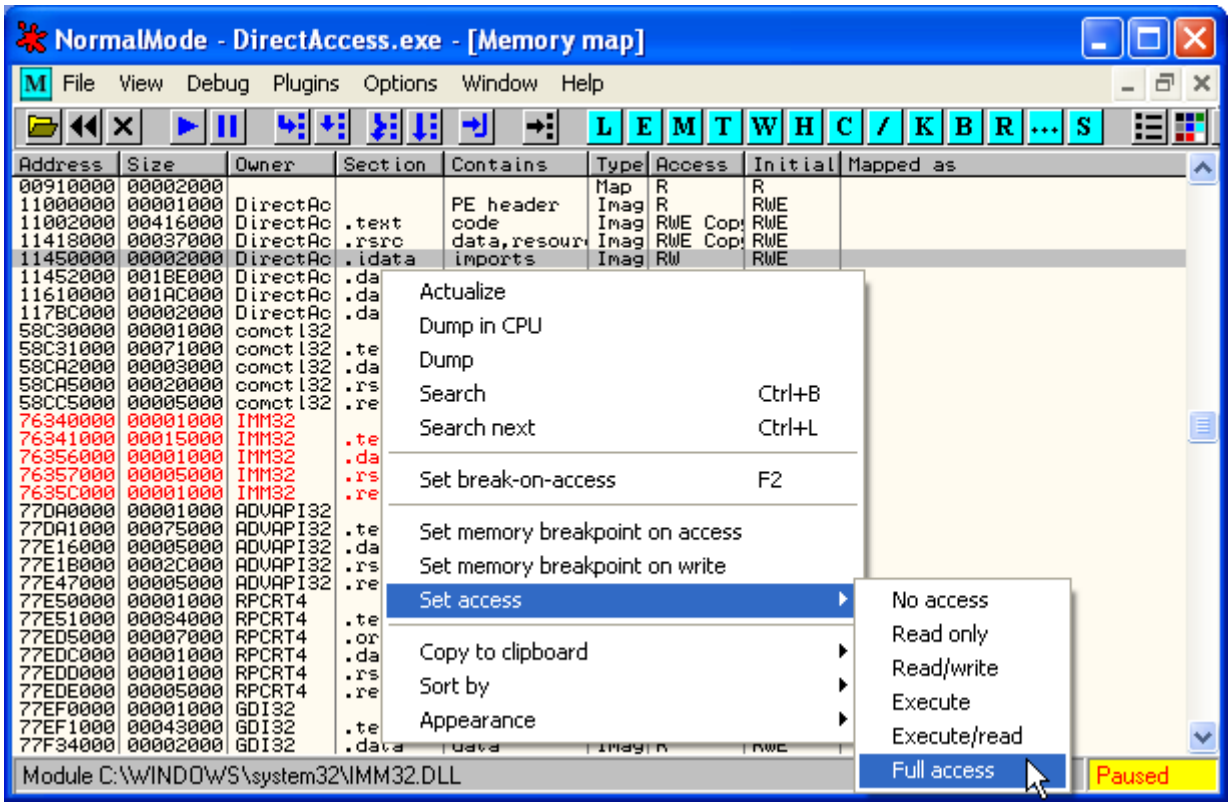

Pulsamos el botón Dump y..




Nada, no hay manera !

Aquí, como es normal me puse a investigar, primero se me ocurrió pensar que quizá fuera una protección de la memoria, hay muchos packers que lo hacen y este podría perfectamente hacerlo si se lo propusiera.

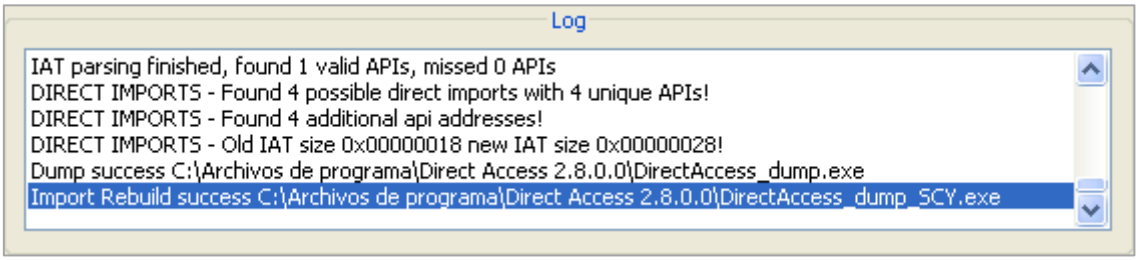
Lo que hice fue darle permiso TOTAL a todas y cada una de las secciones del ejecutable DirectAccess.exe que tenemos cargado en Olly. Yendo a la ventana de memoria <Alt+M> y cambiando los permisos tal como se muestra en la siguiente imagen:



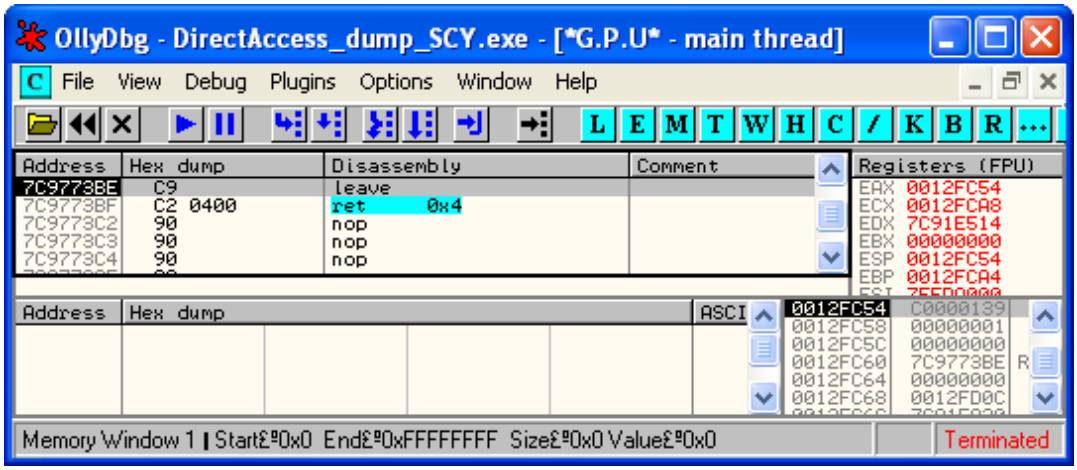

Esto muchas veces me ha funcionado, pero este NO es el caso, aparece el mismo error.

Otra posibilidad está en que alguna/s función/es de la SSDT haya/n sido interceptada/s. Lo miré con Kernel Detective 1.3.1 (que va genial para estos casos) y tampoco vi nada raro ¿???

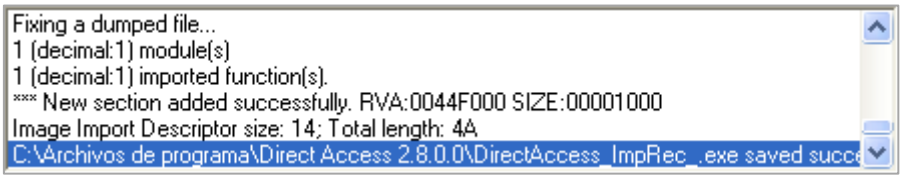
Dumpeamos con Scylla y reparamos las Imports con Scylla también:



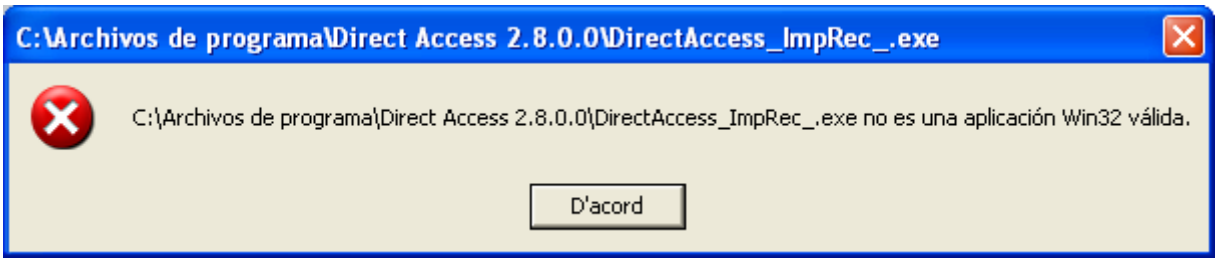

Si intentamos abrir el archivo dumpeado 'DirectAccess_dump_SCY.exe' con Olly pasa esto:



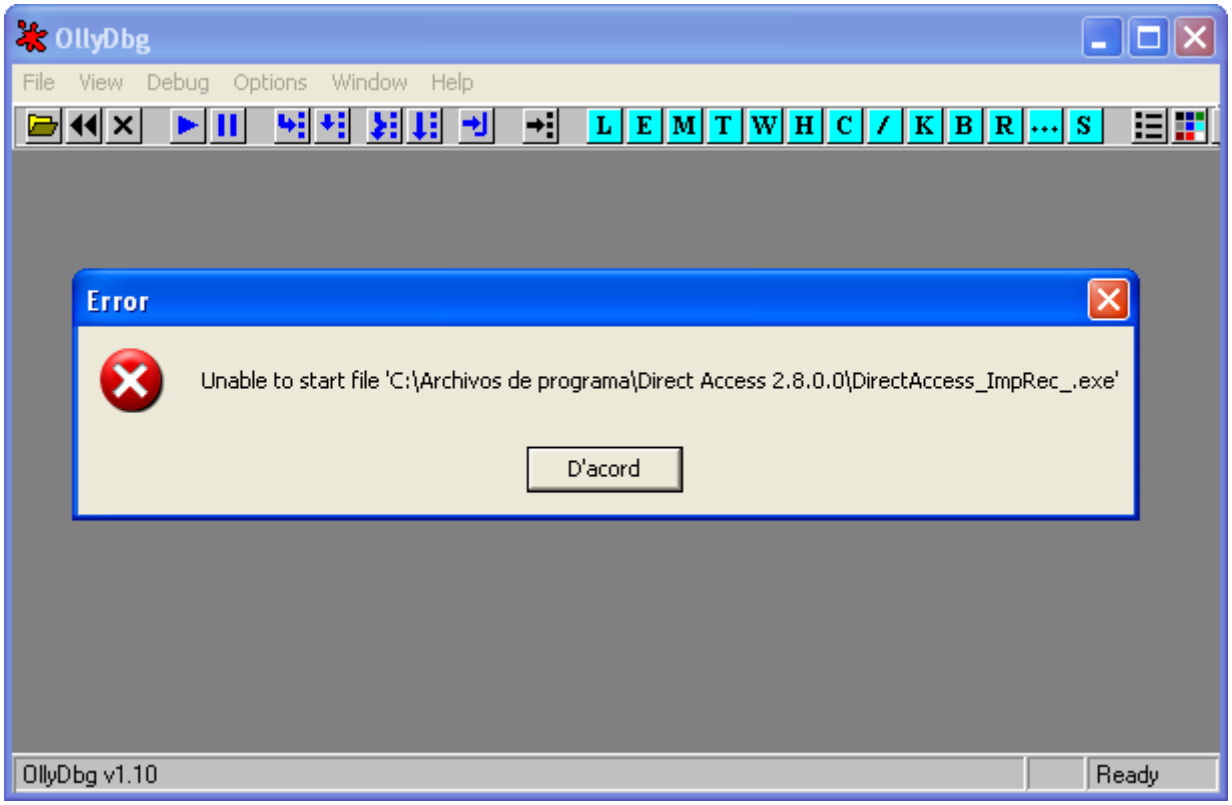
Dumpeamos y reparamos el dumpeado también con ImportReconstructor:




Lo ejecutamos y..




Si intentamos cargar el dumpeado en Olly:



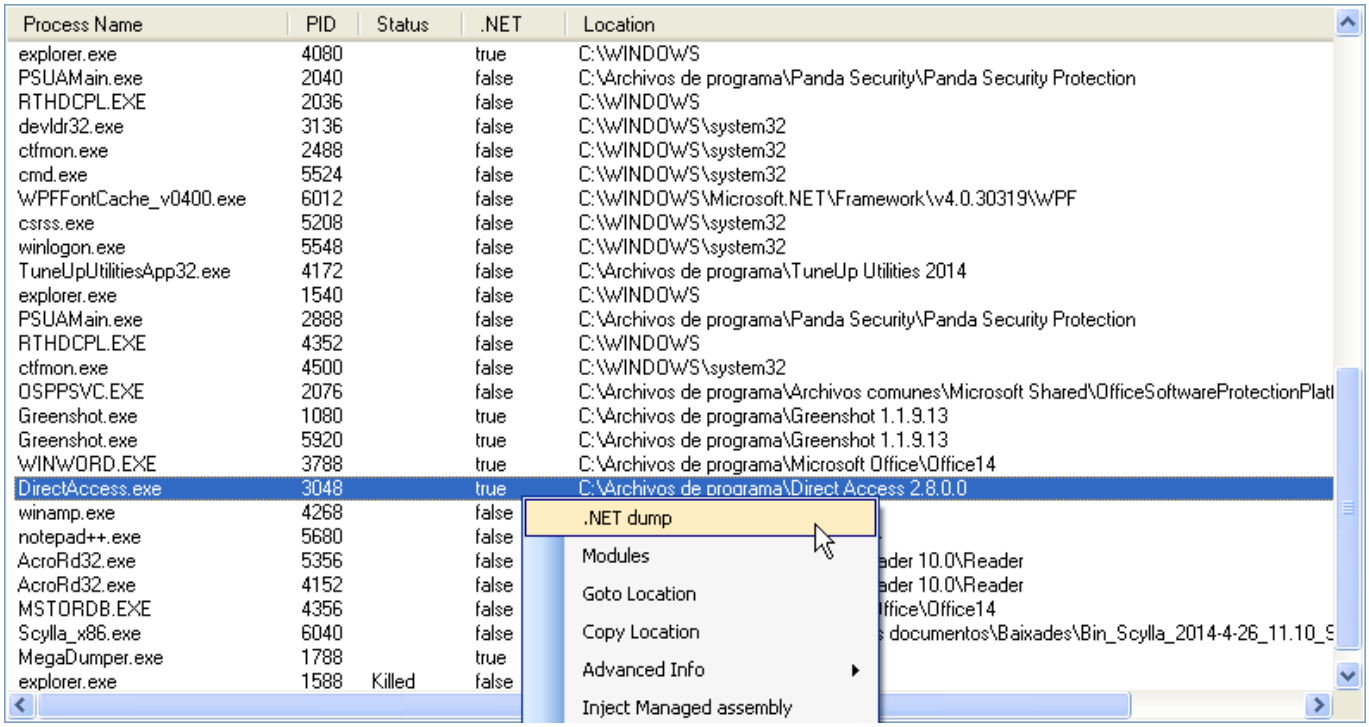

Tampoco puede !

Ya sabemos lo minucioso que es OllyDbg respecto al header del archivo, cualquier dato que no le cuadre terminamos directamente así.

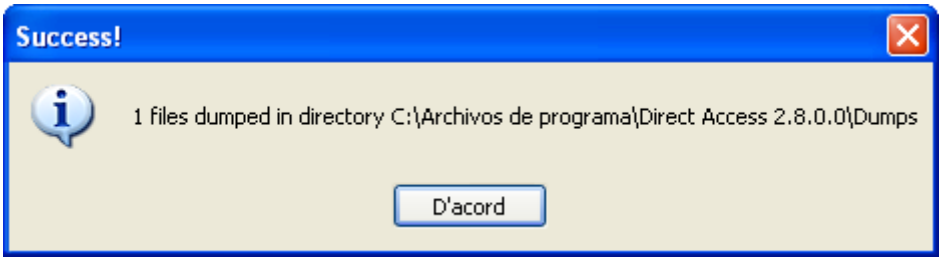
¿Qué hacer?

Cuando las cosas se ponen así no hay más remedio que hacer un pequeño parón y buscar más información al respecto, entretanto me di cuenta que en una de mis carpetas tenía un dumpeador para .NET !!

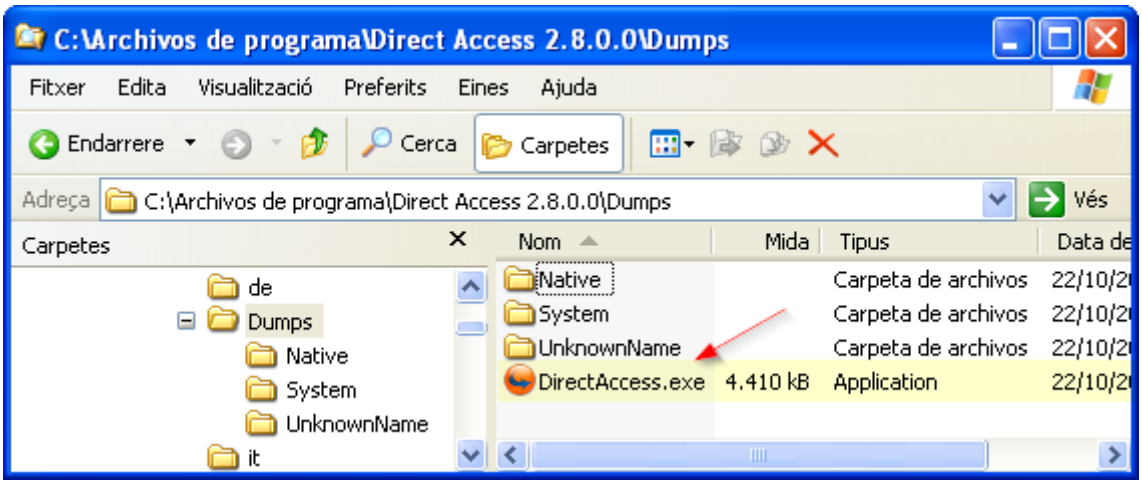
Abrimos MegaDumper 1.0 de SnD y lo dumpeamos tal que así:




A los pocos segundos aparece este cartelito:



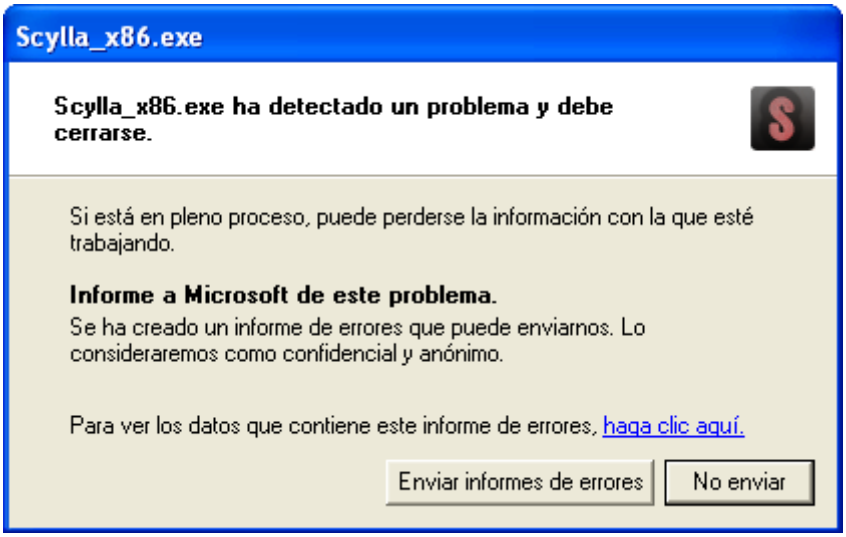

Esto nos deja un dumpeado en el directorio de instalación (subcarpeta Dumps):



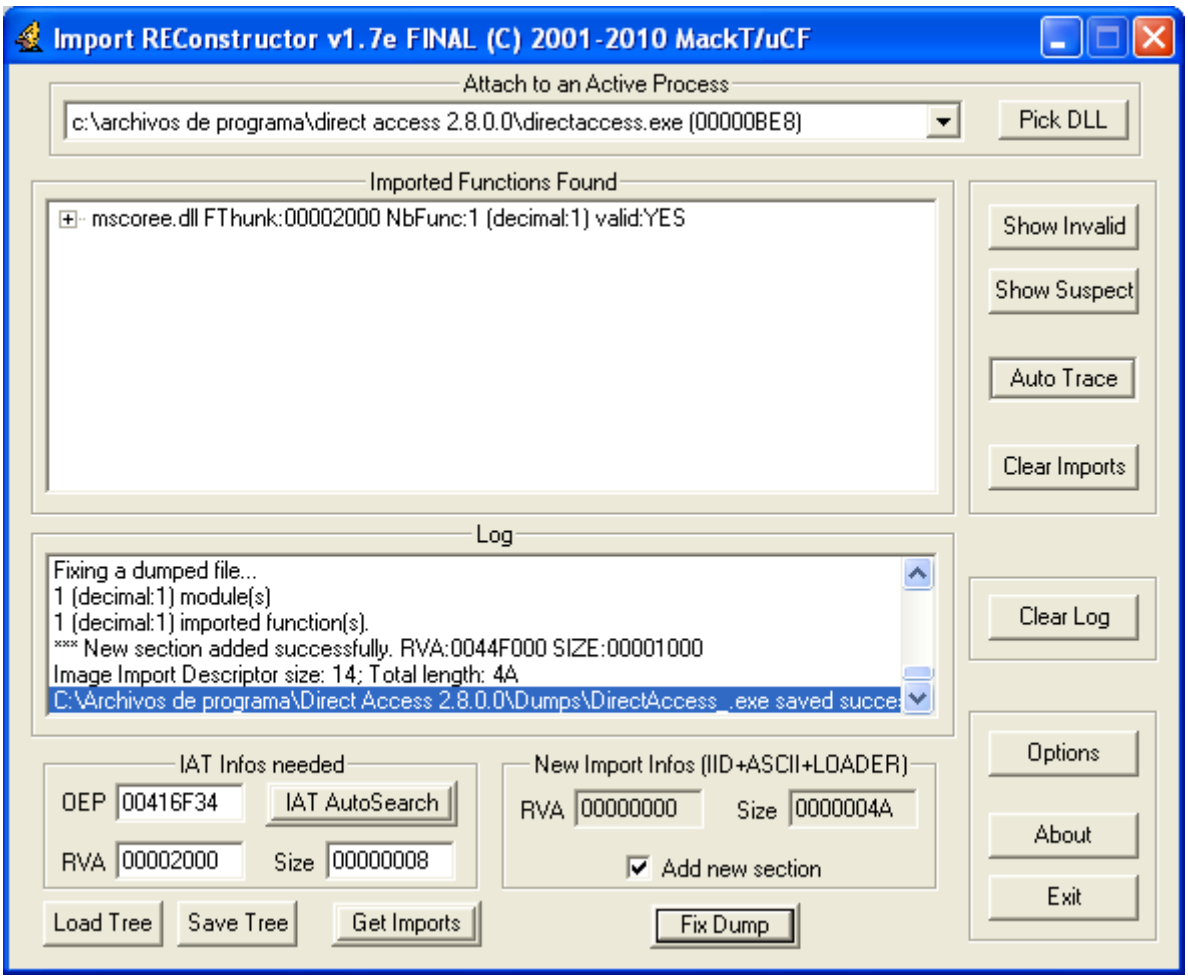

Con tamaño 4.410 Kb, muchos menos Kb que los 7.888 Kb que nos dejó el dumpeado que nos dio Scylla y el que nos dio Import Reconstructor.

Parece que vamos mejorando, si lo tratamos de ejecutar no funciona claro, no tiene Tabla de Importación todavía. Recordemos que aún tenemos el proceso original parado en el OEP (0x11416F2Eh).

Si intentamos reparar el dumpeado con Scylla ocurre esto:




Lo probamos con Import REconstructor 1.7 de MackT/uCF y parece que lo logra:

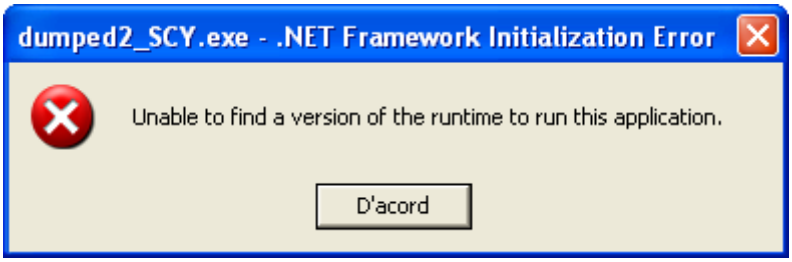



Nos ha dejado el dumpeado 'reparado' en la subcarpeta Dumps del directorio de instalación con nombre DirectAccess_.exe, si lo probamos TAMPOCO FUNCIONA ¡!!!

Lo seguimos intentando, cambiamos la combinación:

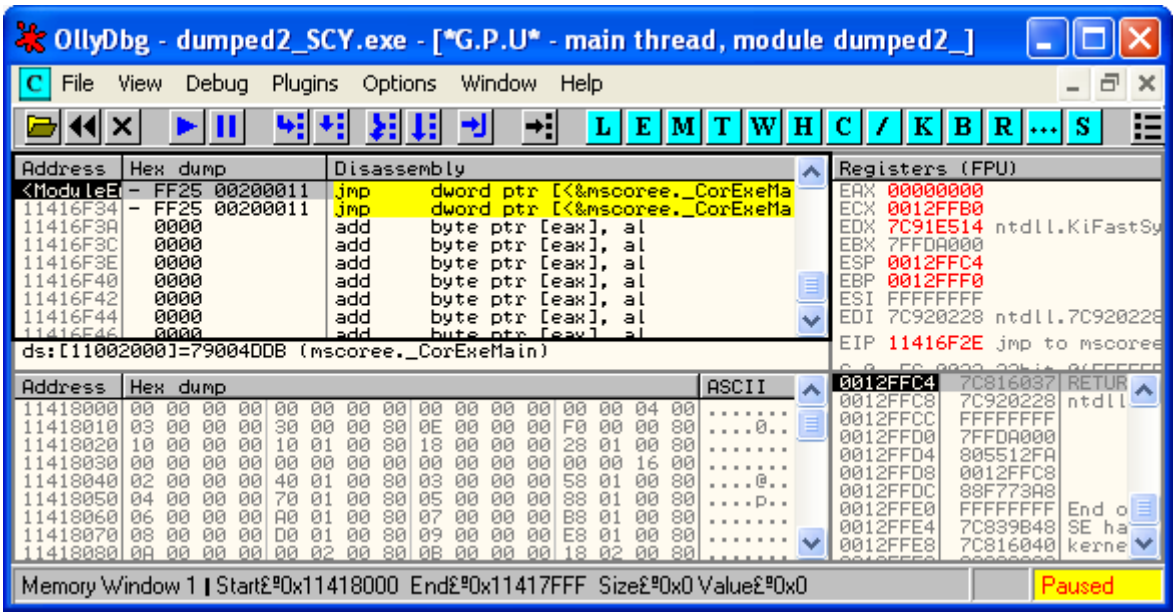
1. Dumpeamos con LordPE Deluxe con la opción IntelliDump
2. Reparamos la IAT con Scylla

Lo probamos y…




Aunque no lo parezca nos vamos acercando.

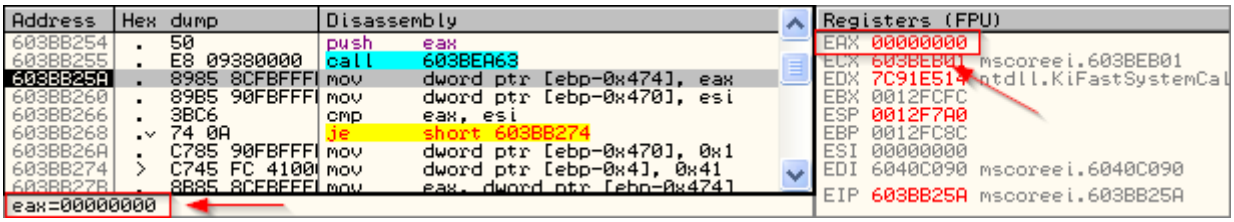
Abrimos el dumpeado dumped2_SCY.exe (el que nos saca el cartel anterior) en un segundo Olly:




Muy mala señal, sabemos que en Olly los .NET no paran en el EntryPoint y corren directamente..

Si traceamos el dumpeado y el original protegido en dos Ollys distintos (sin cerrar el primero donde está el original parado en el OEP), encontramos una gran diferencia aquí:

<u>Dumpeado</u>:



Empacado:

```
Address   Hex dump    Disassembly                          Registers (FPU)
603BB255  E8 09380000 call    603BEA63                     EAX 0017B690 UNICODE "v2.0.50727"
603BB25A  8985 8CFBFFFF mov   dword ptr [ebp-0x474], eax   ECX 603BEB01
603BB260  89B5 90FBFFFF mov   dword ptr [ebp-0x470], esi   EDX 00180002
603BB266  3BC6          cmp   eax, esi                     EBX 0012FCC4
603BB268  74 0A         je    short 603BB274               ESP 0012F768
603BB26A  C785 90FBFFFF mov   dword ptr [ebp-0x470], 0x1   EBP 0012FC54
603BB274  C745 FC 410000 mov  dword ptr [ebp-0x4], 0x41    ESI 00000000
603BB27B  8B85 8CFBFFFF mov   eax, dword ptr [ebp-0x474]   EDI 6040C090
603BB281  3BC6          cmp   eax, esi                     EIP 603BB25A
eax=0017B690, (UNICODE "v2.0.50727")
```

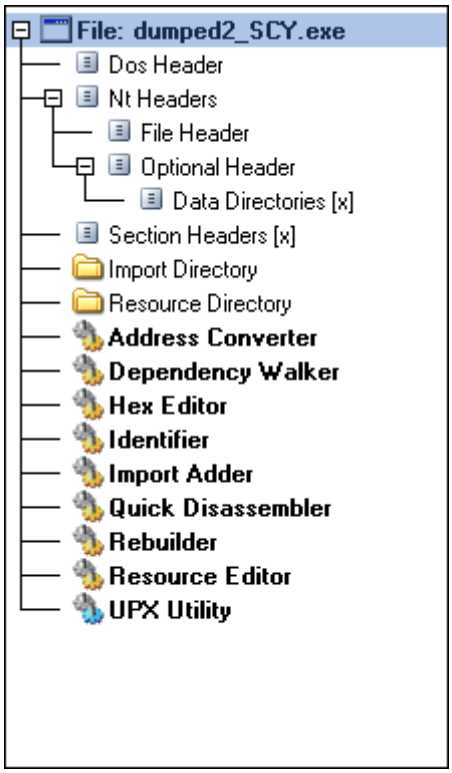
Vaya! Si reiniciamos Olly y entramos en el call anterior encontramos la diferencia:

```
Address   Hex dump   Disassembly                          Comment
603BEA84  . FF75 08  push    dword ptr [ebp+0x8]
603BEA87  . 8975 F0  mov     dword ptr [ebp-0x10], esi
603BEA8A  . E8 45FEFFFF call    GetFileVersion
603BEA8F  . 3D 7A000780 cmp  eax, 0x8007007A
603BEA94  .v 75 6E   jnz     short 603BEB04
603BEA96  . 8B45 F0  mov     eax, dword ptr [ebp-0x10]
603BEA99  . 68 18213B60 push 603B2118
603BEA9E  . 33C9     xor     ecx, ecx                      mscoreei.603BEB01
603BEAA0  . 6A 02    push    0x2
```

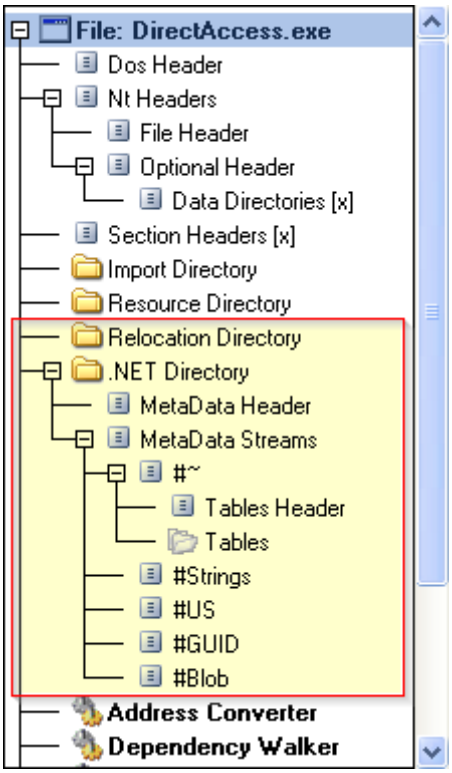
Dentro del call parece acceder al header del archivo dumpeado y no encuentra lo que busca, el retorno en EAX es igual a 0x00h y la ejecución no prosigue.

Si comparamos el dumpeado que nos ha dado LordPE reparado con Scylla (dumped2_SCY.exe) con el dumpeado que hicimos antes con MegaDumper y (sin reparar) vemos estas diferencias:

LordPE + Scylla:




MegaDumper:

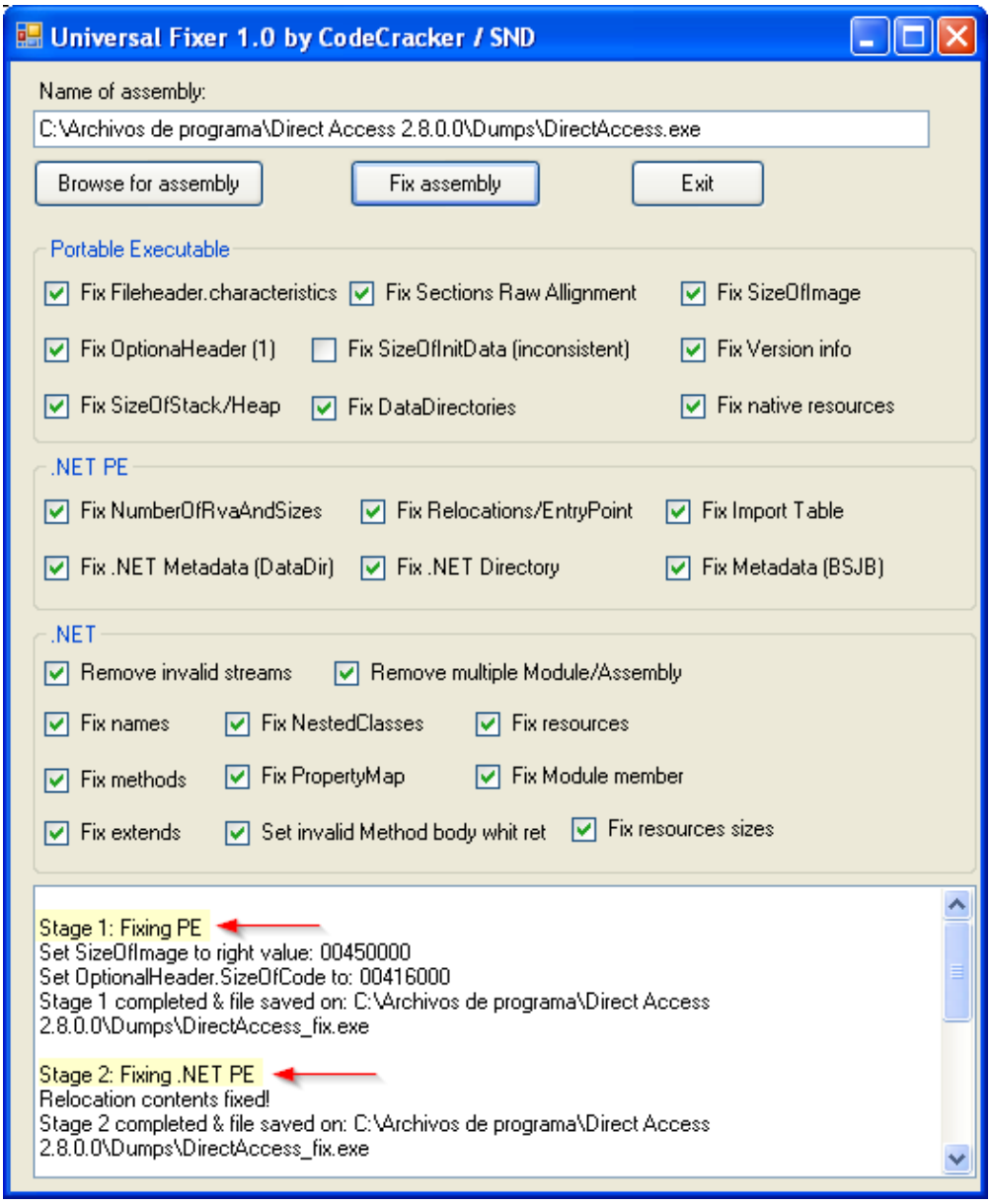



Vemos que al dumpeado que obtuvimos con MegaDumper a simple vista parece más completo, contiene dos secciones que le faltan al dumpeado que nos dio LordPE y que parecen ser fundamentales:

- ✓ Relocation Directory y .NET Directory.

En este punto acudí al grupo (el nuestro: CrackSLatinoS) describiendo el error que estaba obteniendo con la llamada a GetFileVersion (hilo: {CrackSLatinoS} GetFileVersion .NET), al que afortunadamente acudió sequeyo al rescate (un gran saludo amigo :D), me recomendaba utilizar MegaDumper + Universal Fixer. Además me dio un enlace donde las podía encontrar.

Seguí su recomendación al pie de la letra, dumpeé con MegaDumper y reparé el dumpeado con Universal Fixer, tal como se ve en la siguiente imagen:



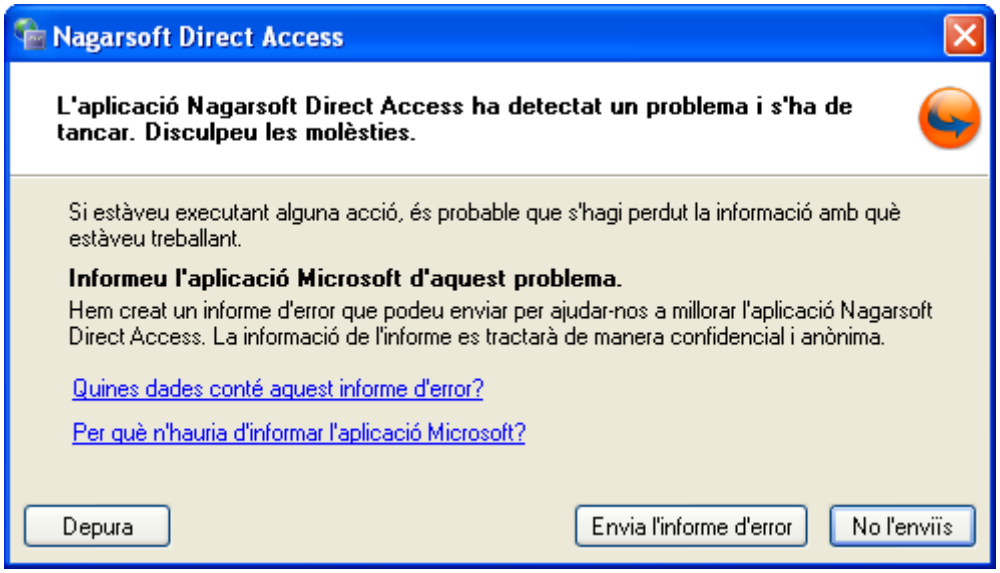

Todavía hay tres etapas más que no se ven en la imagen:

1. Removing invalid streams
2. Removing multiple definition
3. Fixing .NET

Todas ellas reportan el estado:

- ✓ completed & file saved on C:\Archivos de programa\Direct Access 2.8.0.0\Dumps\DirectAccess_fix.exe

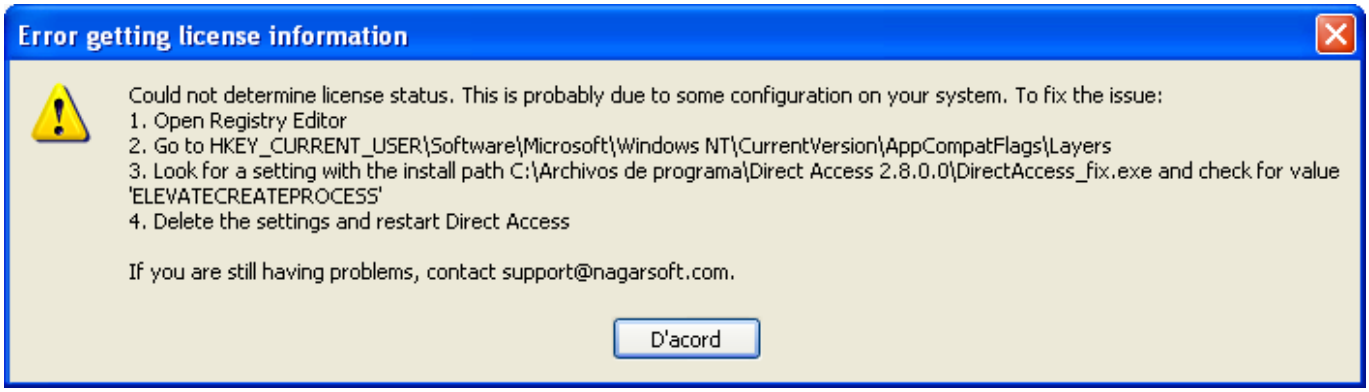
Vamos a la carpeta donde se creó y lo ejecutamos (DirectAccess_fix.exe):




Mirando el informe resulta que se trata de la dirección del kernel 0x7C812FD3h que corresponde a la función RaiseException. Otro problema del dumpeado? en absoluto ! lo que ocurre es que hay que ejecutarlo dentro del directorio de instalación, copiamos DirectAccess_fix.exe a:

- ✓ C:\Archivos de programa\Direct Access 2.8.0.0\

Lo ejecutamos y…



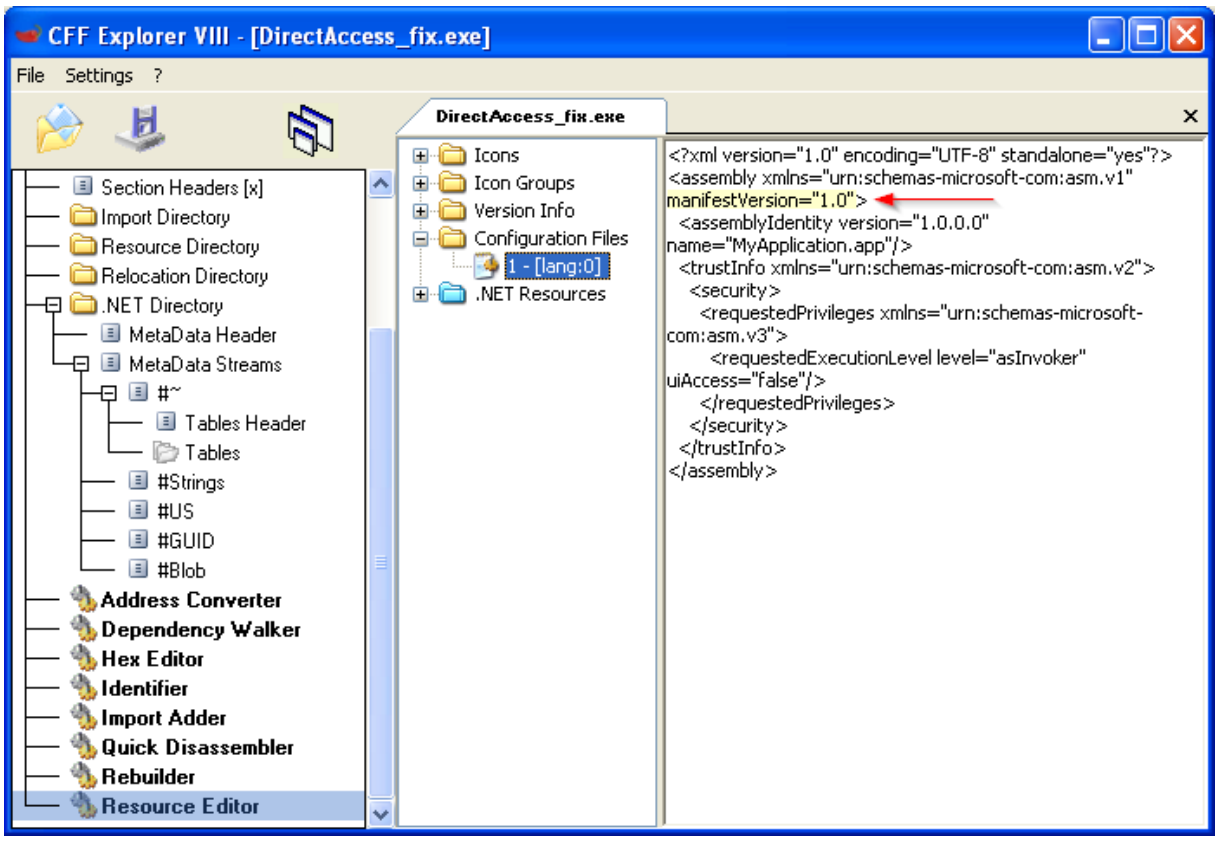

Siiiiiiiiiiiiii ¡!!!

El ejecutable funciona, por fin :DDDD

Lo que pasa es que tiene alguna comprobación adicional (anti-dumps) que trataremos en la siguiente sección del tute..

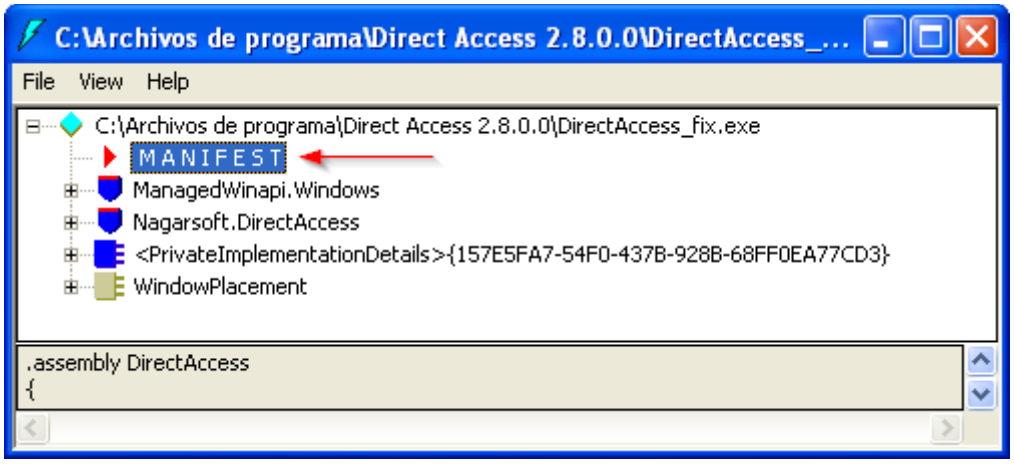
# 10. Batiendo el anti-dump

Ahora que por fin tenemos el .NET dumpeado podemos desensamblarlo con ILDasm, la idea es estudiar el ejecutable a ver dónde está la trampa. Hay que tener en cuenta que no vamos a encontrar código del que estamos acostumbrados, recordad el punto 4 (Principios Básicos). Trabajaremos con un Assembly (con sus módulos), además deberá contener el archivo MANIFEST como podemos comprobar con CFF Explorer):




Allí se ve, además mirando el .NET Directory nos damos cuenta de su importancia pues se listan TODOS los métodos, campos, constantes, miembros, etc.. asociados a éste. Es normal que GetFileVersion retornara 0x00h, lo que todavía no sé es cómo pudimos llegar tan lejos con aquél dumpeado, en fin..

Desensamblamos el assembly con ILDasm y nos muestra esto:




Hacemos doble clic sobre el MANIFEST y se abre otra ventana:

```
MANIFEST
Find  Find Next
// Metadata version: v2.0.50727
.module extern User32.dll
.module extern user32.dll
.module extern user32
.module extern gdi32.dll
.module extern kernel32.dll
.module extern shell32.dll
.module extern kernel32
.module extern MAPI32.DLL
.module extern gdiplus.dll
.module extern Kernel32.dll
.module extern Shell32
.module extern psapi.dll
.module extern ole32.dll
.module extern WinlicenseSDK.dll
.assembly extern mscorlib
{
  .publickeytoken = (B7 7A 5C 56 19 34 E0 89 )                    // .z
  .ver 2:0:0:0
}
.assembly extern System.Windows.Forms
```

Seguimos echando un vistazo y encontramos más cosas interesantes:

```
MANIFEST
Find  Find Next
.mresource public Nagarsoft.DirectAccess.frmImportingCommandsWait.resources
{
  // Offset: 0x000992A8 Length: 0x00000EFC
}
.mresource public Nagarsoft.DirectAccess.frmImportWizard.resources
{
  // Offset: 0x0009A1A8 Length: 0x00034F5B
}
.mresource public Nagarsoft.DirectAccess.frmRegistration.resources
{
  // Offset: 0x000CF108 Length: 0x0001E2A0
}
.mresource public Nagarsoft.DirectAccess.PieChart.PieChartControl.resources
{
  // Offset: 0x000ED3B0 Length: 0x000000B4
}
.mresource public Nagarsoft.DirectAccess.Properties.Resources.resources
{
  // Offset: 0x000ED468 Length: 0x000ED163
}
.mresource public Nagarsoft.DirectAccess.Configuration.Stats.frmStatistics.r
```

Hace referencia a recursos del formulario de registro.

Si seguimos mirando encontramos referencias como esta:

```
.mresource public DirectAccess.exe.licenses
{
  // Offset: 0x00333AB0 Length: 0x0000023F
}
```

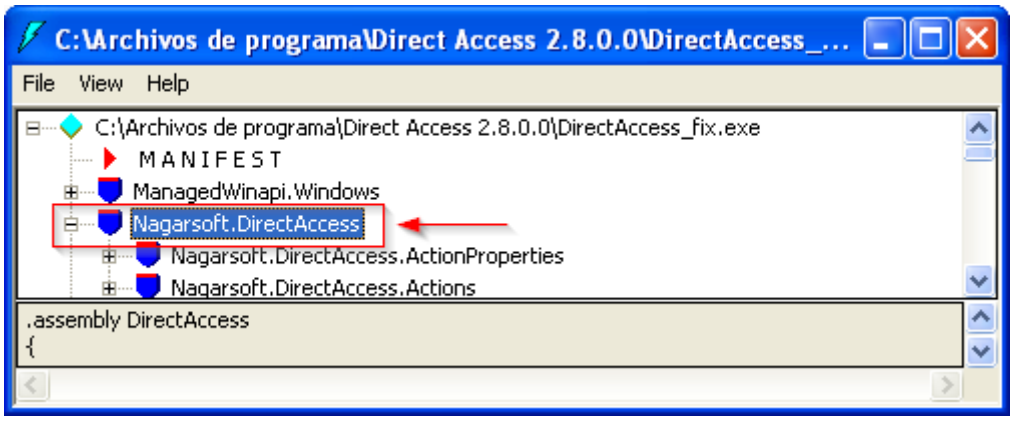
Con offsets y todo !

El archivo MANIFEST finaliza con estas líneas:

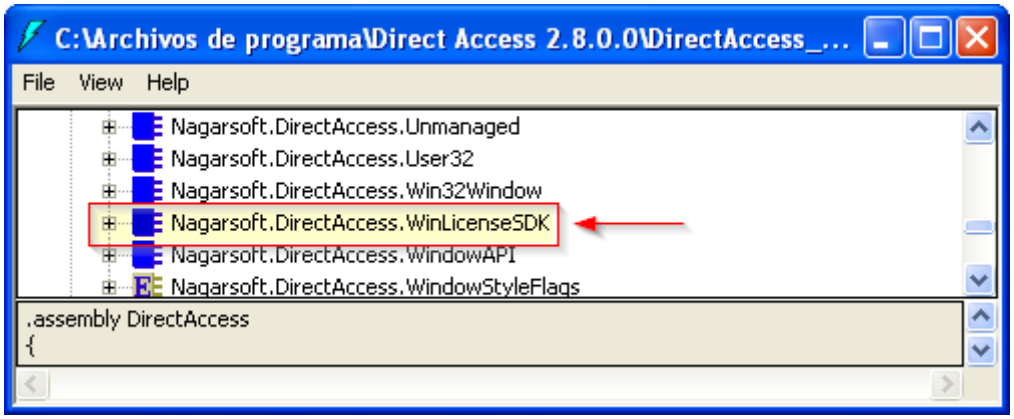
```
.module DirectAccess.exe
// MVID: {157E5FA7-54F0-437B-928B-68FF0EA77CD3}
.imagebase 0x11000000
.file alignment 0x00000200
.stackreserve 0x00100000
.subsystem 0x0002        // WINDOWS_GUI
.corflags 0x00000003     // ILONLY 32BITREQUIRED
// Image base: 0x047A0000
```

Si seguimos analizando la información en seguida nos damos cuenta que se trata de un lenguaje completamente orientado a Objetos (POO), se ven referencias a los constructores de clase, los métodos, los fields (o campos, que son las variables), tipos static, modificadores public, private, protected, lo dicho: POO. ¿Qué cómo lo sé?, fácil, me manejo medio bien en JAVA que tiene esta misma arquitectura :)

Esto nos da más alas para seguir indagando, nos vamos a mover por este árbol:




Con un simple scroll de ratón mirad qué encontramos:



Quería pasar inadvertida pero eso no es posible, estamos aquí precisamente para encontrar este tipo de cosas, hace aparición en escena el WinLicense.

Si vamos al web de oreans technologies (http://www.oreans.com/es/winlicense.php) encontramos mucha información al respecto:

WINLICENSE | RESUMEN

Protector de Software Profesional y Gestor de Licencias

**Última Versión**: 2.3 | **Última Actualización**: 2.3.8.0 | **Actualizado Día**: 21-Sep-2015

## » Introduccion

**WinLicense** combina el mismo nivel de protección que Themida y añade el poder de crear simples y complejos sistemas de licencias, a través de una tecnología bastante flexible que permite a los desarrolladores software distribuir versiones de prueba y licencias de sus aplicaciones de una forma segura.

## » WinLicense como Protector Software

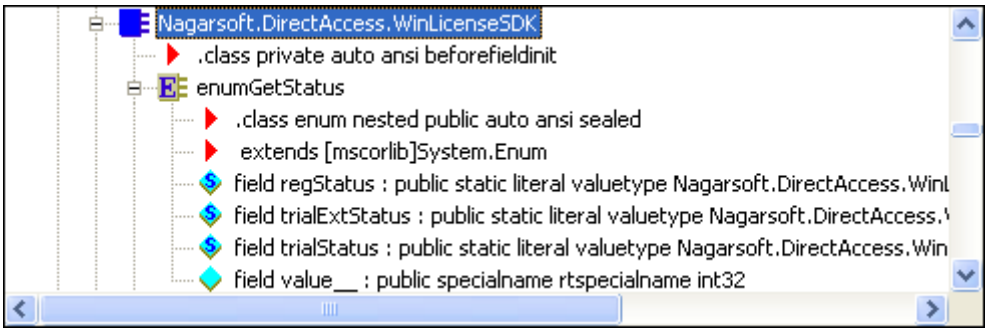
**WinLicense** es un robusto protector software diseñado para desarrolladores software, los cuáles desean proteger sus aplicaciones con las técnicas más avanzadas contra la ingeniería inversa y la piratería. Los programadores software no necesitan ningún cambio en el código fuente de sus programas ni ningún conocimiento en seguridad software para proteger sus aplicaciones con **WinLicense**.

**WinLicense** usa la tecnología de protección SecureEngine®, la cual permite la ejecución de técnicas de protección en el mismo nivel de prioridad que el sistema operativo, para implementar las técnicas de protección más avanzadas y asegurar la seguridad máxima en las aplicaciones protegidas.

A continuación mostramos una breve lista de las características de protección de **WinLicense**:

- Multitud de niveles de encriptación para una misma aplicación.
- Detección avanzada de herramientas contra la seguridad software.
- Ejecución de código en el mayor nivel de prioridad del sistema operativo.
- Ofuscación de código, datos y llamadas a funciones APIs importadas en una aplicación.
- Protección contra depuradores y desensambladores.
- SDK interna para permitir comunicación paralela con el sistema de protección desde una aplicación.
- Tecnología avanzada contra el volcado de una aplicación de memoria a disco.
- Total edición y personalización de los mensajes de protección.

Si te crees todo lo que te ponen allí terminas cerrando el Olly y comprando el producto. Como este no es nuestro estilo expandimos la clave Nagarsoft.DirectAccess.WinLicenseSDK en ILDASM y vemos esto:



Donde se da información de Clase, se muestra información de la enumeración enumGetStatus (que deriva de la Clase [mscorlib]System.Enum) y de sus *fields* (campos): regStatus, trialExtStatus, etc..

Está claro que vamos bien encaminados, sigamos bajando a ver qué más vemos. Podemos hacer un volcado del árbol completo que vemos en ILDasm desde el menú *File -> Dump TreeView* y lo podemos ir consultando con Notepad++ (o con cualquier otro editor):

```
|  |___[CLS] Nagarsoft.DirectAccess.WinLicenseSDK
|  |  |    .class private auto ansi beforefieldinit
|  |  |___[ENU] enumGetStatus
|  |  |  |    .class enum nested public auto ansi sealed
|  |  |  |     extends [mscorlib]System.Enum
|  |  |  |___[STF] field regStatus : public static literal valuetype Nagarsoft.DirectAccess.WinLicenseSDK/enumGetStatus
|  |  |  |___[STF] field trialExtStatus : public static literal valuetype Nagarsoft.DirectAccess.WinLicenseSDK/enumGetStatus
|  |  |  |___[STF] field trialStatus : public static literal valuetype Nagarsoft.DirectAccess.WinLicenseSDK/enumGetStatus
|  |  |  |___[FLD] field value__ : public specialname rtspecialname int32
|  |  |
```

Es un pequeño extracto, se ve perfecto (mejor que en ILDasm casi), se observan las etiquetas:

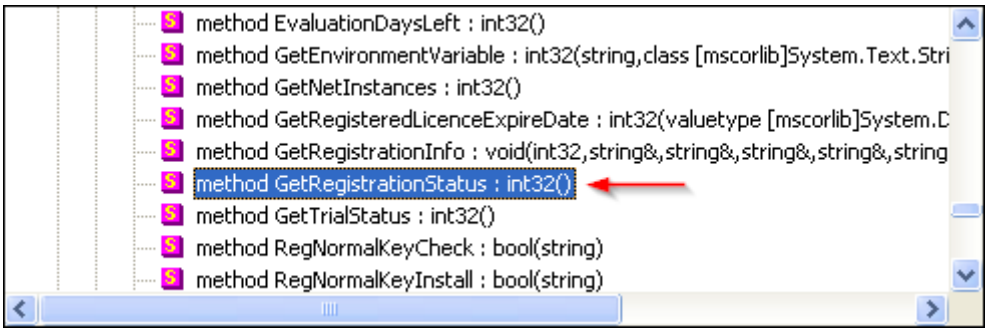
```
[CLS]: Class o Clase
[ENU]: Enumeration o Enumeración
[STF]: Static Field o Campo Static
[FLD]: Field o Campo
[NSP]: Namespace o Espacio de Nombres
[MET]: Method o Método
[PTY]: Property o Propiedad
```

Etcétera, además de ser fácilmente reconocibles se pueden consultar aquí:

- ✓ http://flylib.com/books/en/4.441.1.47/1/

Cuando se refiere a Instance [method|field] significa que es un método o campo *de Objeto* y no de clase, éstos últimos (los *de Clase*) son los que llevan el modificador Static. Un objeto es una instancia de una clase, digamos que la clase es el molde y los objetos se crean partiendo de éste. Continuando con el símil podríamos decir que la Clase es el molde para hacer flanes y los objetos son los flanes en sí. Si queremos flanes de distintos sabores solamente debemos extender (derivar) de la clase base (o superclase) molde y agregarle el sabor que queramos, con esto conseguimos desarrollar código fácilmente y en muy poco tiempo. La POO funciona así y lo que estamos viendo aquí es justo este tipo de estructuras.

Seguimos buscando y encontramos métodos muy interesantes:




Le hacemos doble clic y se abre otra ventanita:

En ella podemos ver más información acerca del método *GetRegistrationStatus*:

```
.method public hidebysig static int32  GetRegistrationStatus() cil managed
{
  // Code size       131 (0x83)
  .maxstack  3
  .locals init (class [mscorlib]System.Text.StringBuilder V_0,
           string V_1,
           int32 V_2,
           class [mscorlib]System.FormatException V_3)
  IL_0000:  ldc.i4.s   100
  IL_0002:  newobj     instance void [mscorlib]System.Text.StringBuilder::.ctor(int32)
  IL_0007:  stloc.0
  IL_0008:  ldstr      "WLRegGetStatus"
  IL_000d:  call       string [mscorlib]System.Environment::GetEnvironmentVariable(string)
  IL_0012:  stloc.1
  IL_0013:  ldstr      "WLRegGetStatus"
  IL_0018:  ldloc.0
  IL_0019:  ldc.i4.s   100
  IL_001b:  call       int32 Nagarsoft.DirectAccess.WinLicenseSDK::GetEnvironmentVariable(string,
                                                                                          class [mscorlib]System.Text.StringBuilder,
                                                                                          int32)
  IL_0020:  pop
  IL_0021:  call       class [Gurock.SmartInspect]Gurock.SmartInspect.Session Nagarsoft.DirectAccess.Logger::get_Main()
  IL_0026:  ldstr      "WLRegGetStatus"
  IL_002b:  ldloc.0
  IL_002c:  callvirt   instance string [mscorlib]System.Object::ToString()
  IL_0031:  callvirt   instance void [Gurock.SmartInspect]Gurock.SmartInspect.Session::LogString(string,
                                                                                                 string)
  IL_0036:  call       class [Gurock.SmartInspect]Gurock.SmartInspect.Session Nagarsoft.DirectAccess.Logger::get_Main()
  IL_003b:  ldstr      "WLRegGetStatus Managed"
  IL_0040:  ldloc.1
  IL_0041:  callvirt   instance void [Gurock.SmartInspect]Gurock.SmartInspect.Session::LogString(string,
                                                                                                 string)
  IL_0046:  ldloc.1
  IL_0047:  call       bool [mscorlib]System.String::IsNullOrEmpty(string)
  IL_004c:  brfalse.s  IL_0062
  IL_004e:  ldc.i4.s   -100
  IL_0050:  stloc.2
  IL_0051:  call       class [Gurock.SmartInspect]Gurock.SmartInspect.Session Nagarsoft.DirectAccess.Logger::get_Main()
  IL_0056:  ldstr      "ERROR retrieving WLRegGetStatus"
  IL_005b:  callvirt   instance void [Gurock.SmartInspect]Gurock.SmartInspect.Session::LogError(string)
  IL_0060:  br.s       IL_0081
  .try
  {
    IL_0062:  ldloc.1
    IL_0063:  call       int32 [mscorlib]System.Convert::ToInt32(string)
    IL_0068:  stloc.2
    IL_0069:  leave.s    IL_0081
  }  // end .try
  catch [mscorlib]System.FormatException
  {
    IL_006b:  stloc.3
    IL_006c:  ldc.i4.s   -100
```

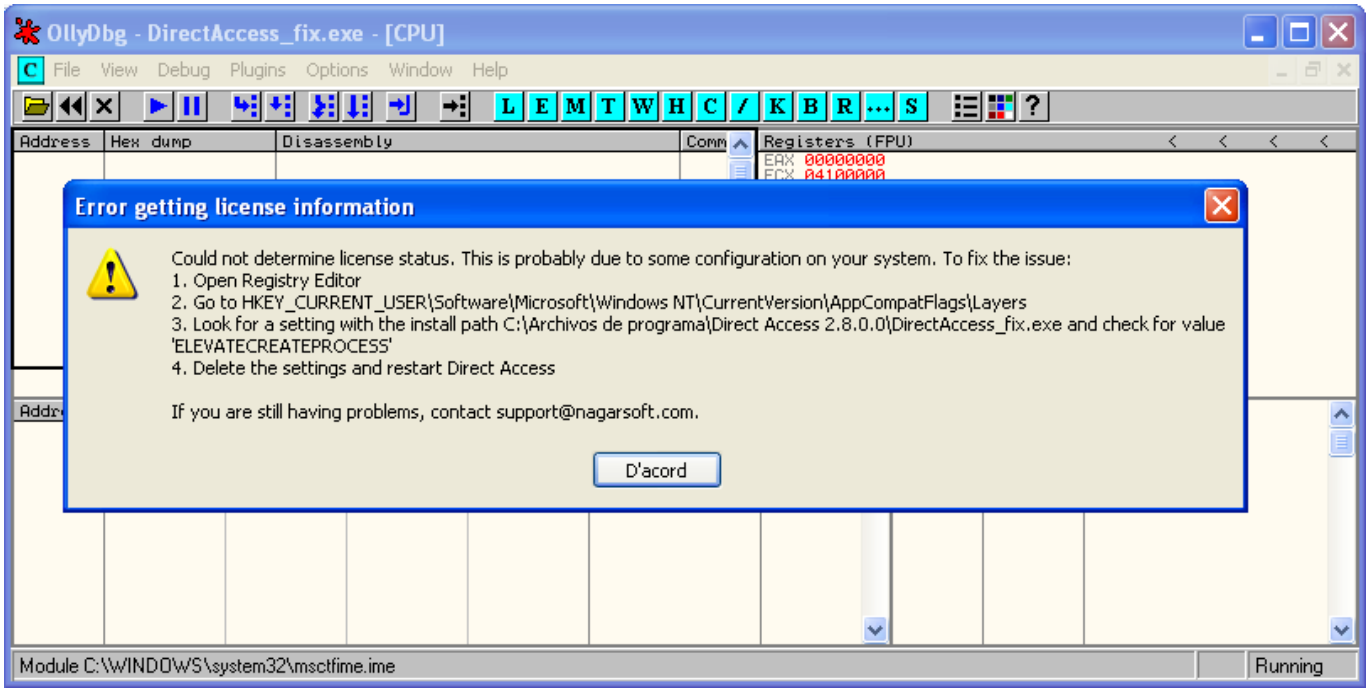
Más que interesante, podemos ver una llamada a GetEnvironmentVariable con el más que sospechoso parámetro "WLRegGetStatus", después se ve otra llamada a GetEnvironmentVariable, pero no apuntan a la misma función, fijémonos que cambia el espacio de nombres siendo System.Environment en el primer caso (dentro de la librería mscorlib) y Nagarsoft.DirectAccess.WinLicenseSDK en el segundo caso (propia del WinLicense). El espacio de nombres, bajo mi punto de vista, viene a ser algo similar a los *packages* de java, puedes duplicar el nombre de un método o incluso Clase siempre y cuando forme parte de un package distinto, hay packages de java que vienen con Clases con mismo nombre, por ejemplo:

- ✓ java.awt.List
- ✓ java.util.List

Para diferenciarlas se antepone el nombre del package (evitando ambigüedades). Lo mismo sucede en la imagen anterior, para diferenciar la llamada a *GetEnvironmentVariable* antepone el nombre del espacio de nombres en que se ubica, además se observa también que justo antes del nombre está el carácter de dos puntos dobles :: llamado operador de ámbito (no se utiliza en java pero si en C++). Son estructuras fácilmente reconocibles, es más, aprendiendo a programar por ejemplo en java no se estría muy lejos de poder escribir un programa en .NET (cambiará la sintaxis y algunos detalles más, fácilmente solventables).

Dicho esto sigamos a lo nuestro, el programa parece que va a obtener información del estado de registro a través de la llamada a GetEnvironmentVariable con el parámetro WLRegGetStatus.

Para cerciorarnos cargamos el dumpeado DirectAccess_fix.exe en Olly:

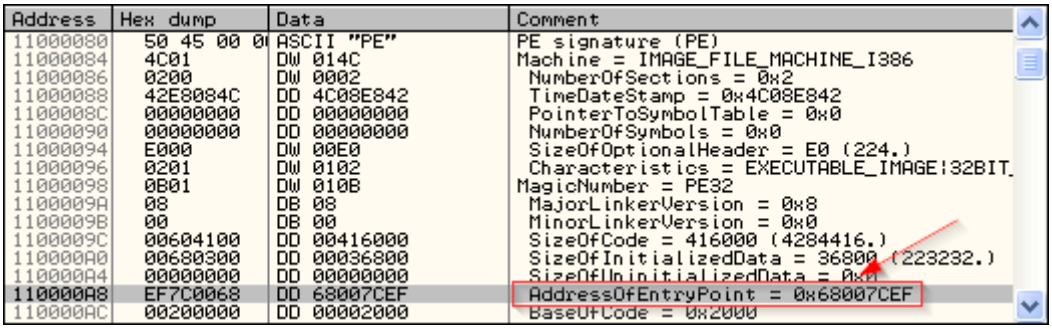



No para en el EntryPoint, aunque no lo parezca esto es muy buena señal, si queremos parar antes debemos configurar Olly para que se detenga en el *System breakpoint*, pero antes hagamos una pequeña comprobación que me tiene intrigado.

Según la cabecera del ejecutable el EntryPoint está en el offset 0x416F34h, al que agregándole el ImageBase se obtiene el Virtual Address del mismo:

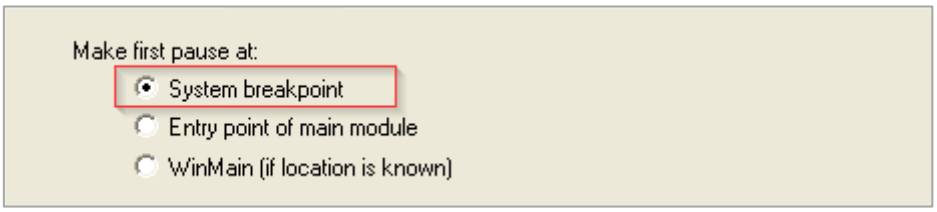
```
va = 0x416F34h + 0x11000000h == 0x11416F34h
```

Esta es la dirección que obtuvimos antes. Si miramos el valor del EntryPoint en Olly ahora que está el proceso en ejecución vemos que ha cambiado ¿???

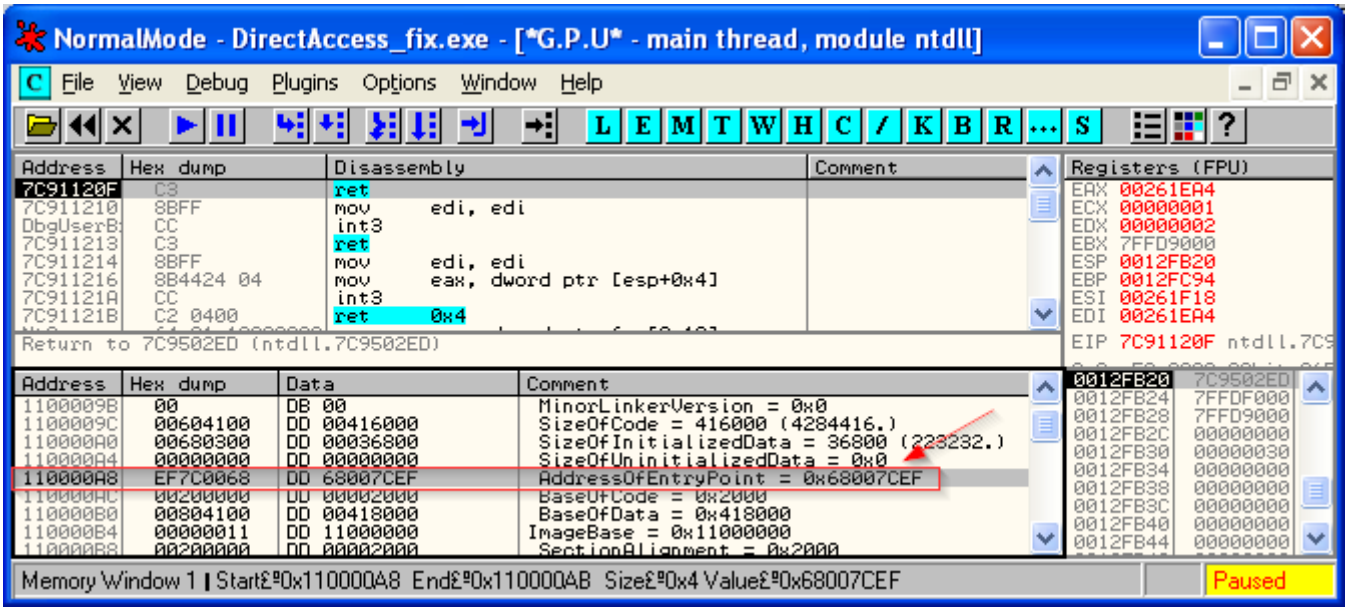


| Address | Hex dump | Data | Comment |
|---|---|---|---|
| 11000080 | 50 45 00 00 | ASCII "PE" | PE signature (PE) |
| 11000084 | 4C01 | DW 014C | Machine = IMAGE_FILE_MACHINE_I386 |
| 11000086 | 0200 | DW 0002 | NumberOfSections = 0x2 |
| 11000088 | 42E8084C | DD 4C08E842 | TimeDateStamp = 0x4C08E842 |
| 1100008C | 00000000 | DD 00000000 | PointerToSymbolTable = 0x0 |
| 11000090 | 00000000 | DD 00000000 | NumberOfSymbols = 0x0 |
| 11000094 | E000 | DW 00E0 | SizeOfOptionalHeader = E0 (224.) |
| 11000096 | 0201 | DW 0102 | Characteristics = EXECUTABLE_IMAGE!32BIT_ |
| 11000098 | 0B01 | DW 010B | MagicNumber = PE32 |
| 1100009A | 08 | DB 08 | MajorLinkerVersion = 0x8 |
| 1100009B | 00 | DB 00 | MinorLinkerVersion = 0x0 |
| 1100009C | 00604100 | DD 00416000 | SizeOfCode = 416000 (4284416.) |
| 110000A0 | 00680300 | DD 00036800 | SizeOfInitializedData = 36800 (223232.) |
| 110000A4 | 00000000 | DD 00000000 | SizeOfUninitializedData = 0x0 |
| 110000A8 | EF7C0068 | DD 68007CEF | AddressOfEntryPoint = 0x68007CEF |
| 110000AC | 00200000 | DD 00002000 | BaseOfCode = 0x2000 |

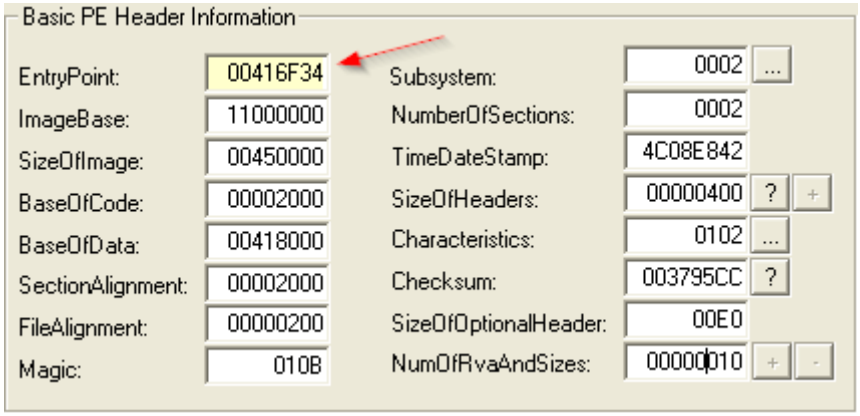
Allí debería estar el valor 0x416F34h. Configuremos Olly para que se detenga en el *System breakpoint*:



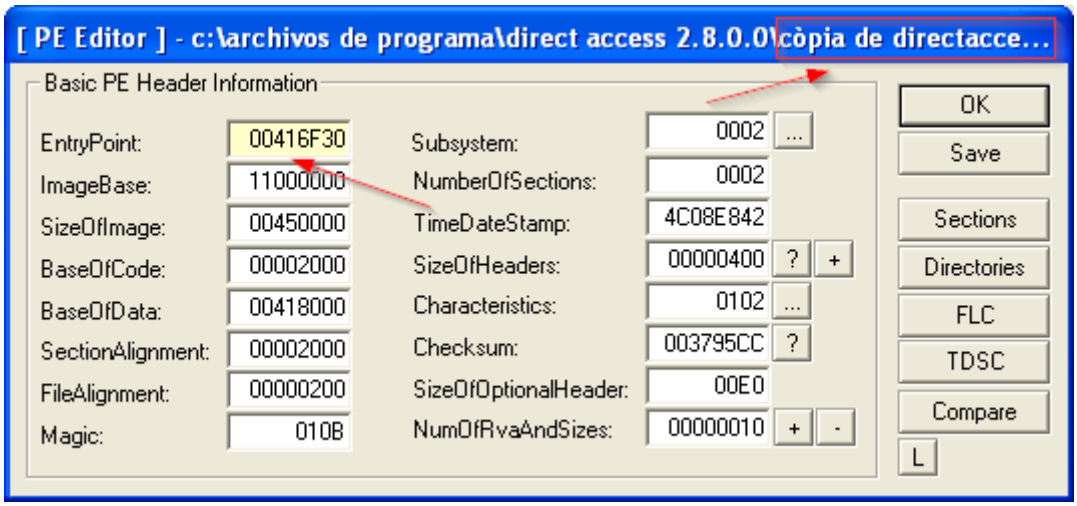
Reiniciamos Olly <Ctrl + F2> y miramos el valor del EntryPoint desde el *System breakpoint*:




Sigue teniendo ese mismo valor, si cargamos el EXE en LordPE vemos que allí está el nuestro:

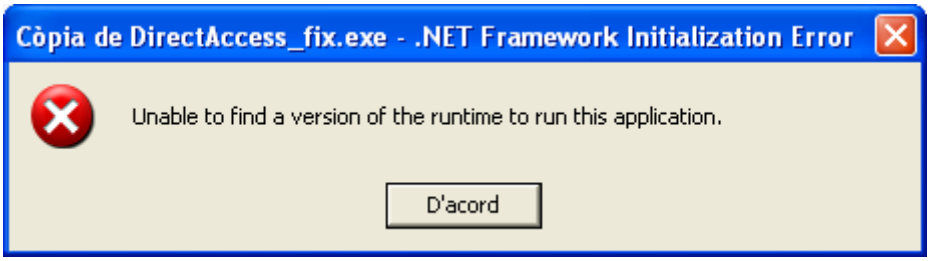



Salgamos de este entuerto, hagamos una copia del ejecutable DirectAccess_fix.exe y lo cargamos en el LordPE, modificamos el valor del EntryPoint arbitrariamente tal que así:



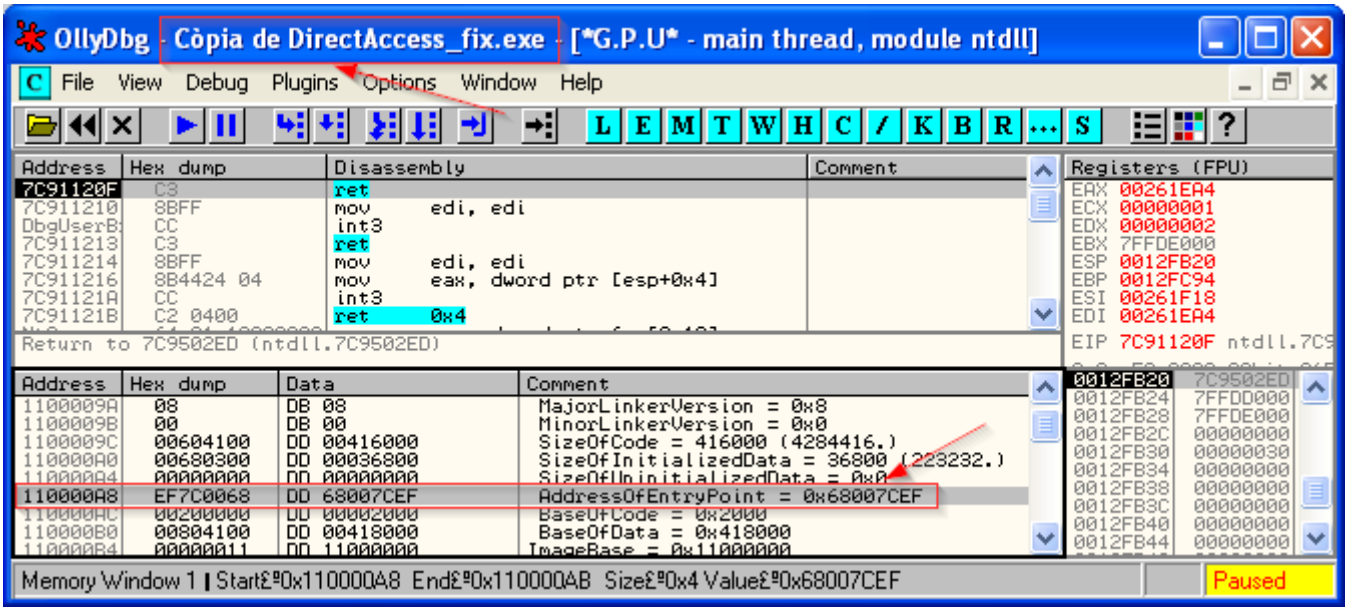

Pulsamos el botón *Save* y después *OK*.

Ejecutamos la copia (con el EntryPoint modificado) y..




Eppppppppppps.. No se ejecuta ¡!!!

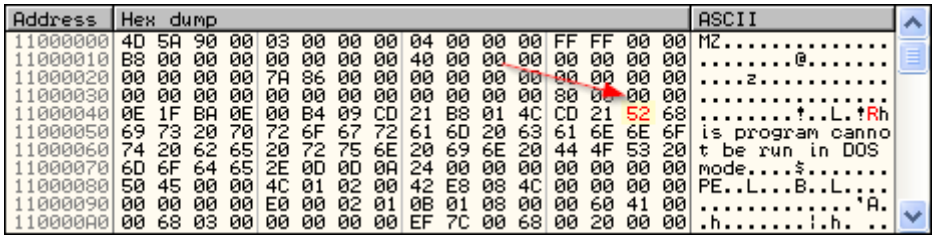
Como estamos aquí para aprender seguiremos investigando, cargamos la copia con el EntryPoint modificado en el Olly y desde el *System breakpoint* miramos el valor del AddressOfEntryPoint:




El mismo ¡!!!

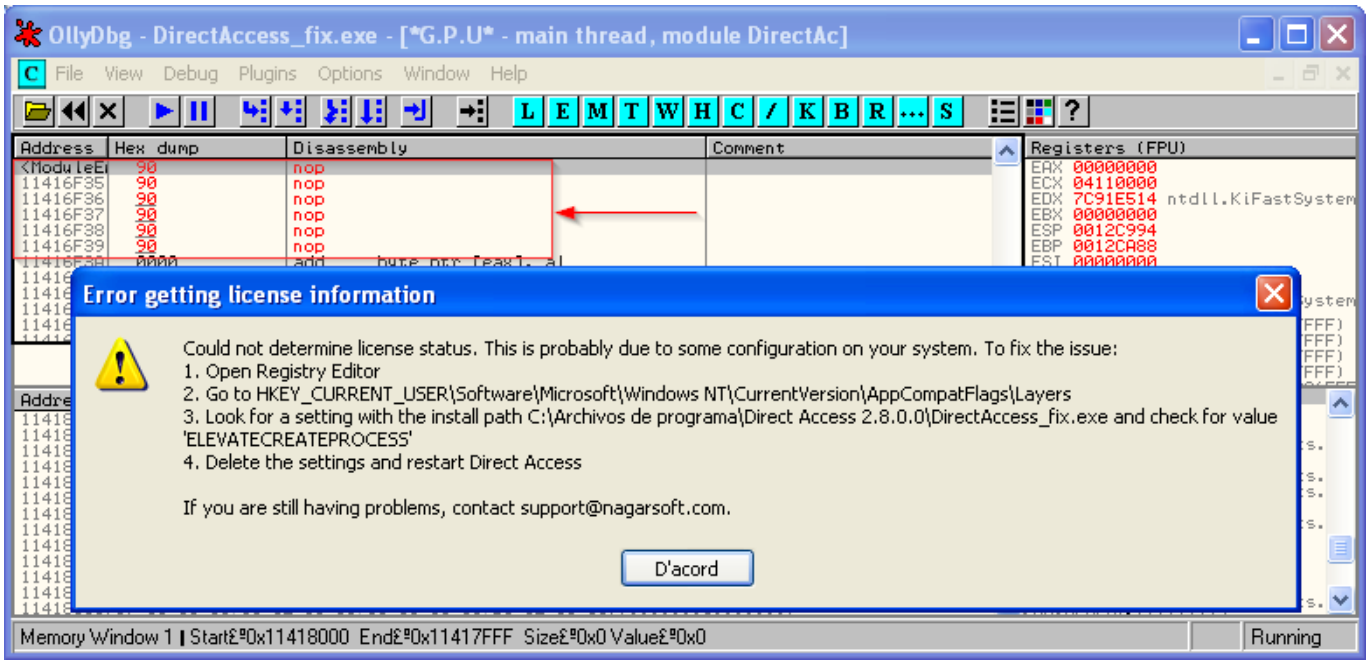
Si abrimos dos Ollys distintos, cargamos el dumpeado en uno y su copia con el EP modificado en el otro y copiamos el header completo del dumpeado (normal) al modificado tampoco corre ¿???

Por si fuera algún control de integridad del archivo modificamos un byte del header que no influya en la carga del archivo, por ejemplo esta cadena:




Forma parte del *DOS Stub*, por lo que el EXE no debería verse afectado, y de hecho, si lo cargamos en Olly con esta modificación (y fuera de Olly igual), lo dejamos correr funciona bien. Deducimos, pues que no se le ha aplicado ninguna firma con StrongNames.

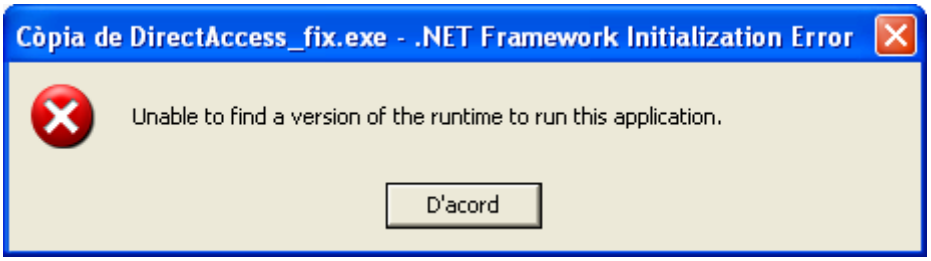
Para más inri, si cargamos DirectAccess_fix.exe (el dumpeado SIN modificar) en Olly, vamos a la dirección del OEP 0x11416F34h y NOPeamos el JMP a _CorExeMain tal que así:




El programa corre, si y sólo si lo modificamos después del System Breakpoint, si salvamos los cambios al disco tendremos dos problemas:

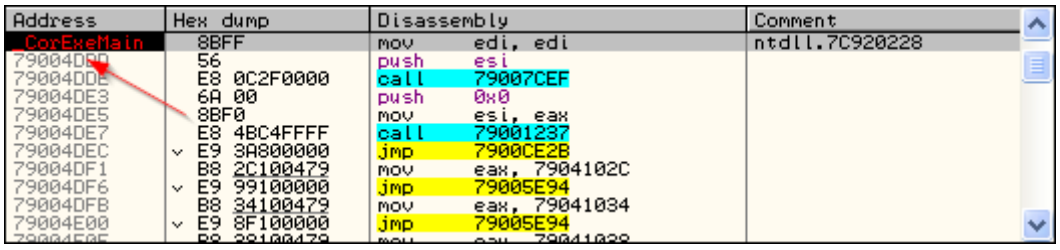
1. Los fixups del ejecutable: son los 0x90h que se ven subrayados en la imagen.
2. El proceso de carga no finaliza y el proceso no corre.

Hagamos una prueba y salvamos únicamente la modificación de los dos primeros bytes 0xFF25h que no tienen fixups a disco con nombre DirectAccess_fix_NOP.exe, lo ejecutamos y…




Está claro que aunque NO ejecute la instrucción SI la necesita para cargar. Pero si no para ? es cierto que no se detiene en el salto al OEP, esto por qué ? Traceando el dumpeado (sin modificaciones) en Olly se descubre la zona en la que salta al OEP.

Cargamos el dumpeado sin modificar en Olly y ponemos un BP en _CorExeMain, damos RUN y para:



Sabemos que si o si, debe parar en *_CorExeMain*, esta librería se encargará de la traducción del código IL o CIL a código máquina entendible por nuestra CPU (según la teoría), para saber desde donde se hizo la llamada miramos la pila:

```
0012FFC4  7C816037  RETURN to KERNEL32.7C816037
0012FFC8  7C920228  ntdll.7C920228
0012FFCC  FFFFFFFF
0012FFD0  7FFDA000
0012FFD4  00000640
0012FFD8  0012FFC8
0012FFDC  88E7C020
0012FFE0  FFFFFFFF  End of SEH chain
0012FFE4  7C839B48  SE handler
0012FFE8  7C816040  KERNEL32.7C816040
0012FFEC  00000000
0012FFF0  00000000
0012FFF4  00000000
0012FFF8  79004DDB  mscoree._CorExeMain
```

Nos ponemos sobre esta línea y pulsamos <Enter>, esto nos lleva a la ventana de código:

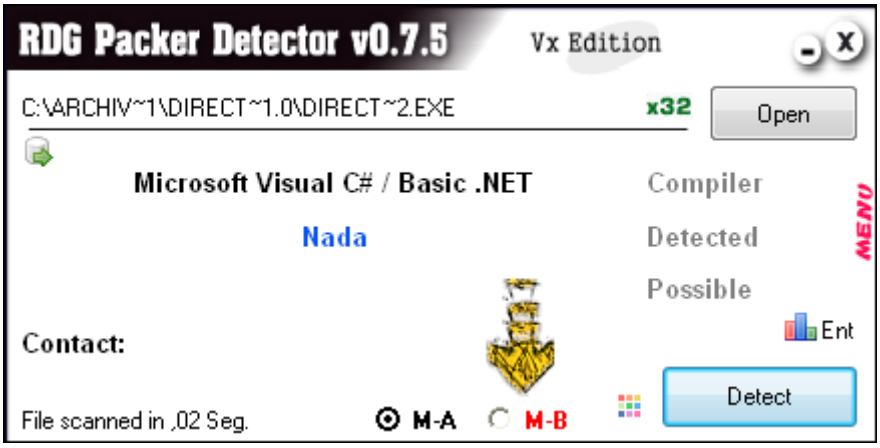
```
Address   Hex dump    Disassembly                    Comment
7C81602A  6A 09       push    0x9
7C81602C  6A FE       push    -0x2
7C81602E  FF15 B013807C  call  dword ptr [<&ntdll.NtSe  ntdll.ZwSetInformationThread
7C816034  FF55 08     call    dword ptr [ebp+0x8]      mscoree._CorExeMain
7C816037  50          push    eax
7C816038  E8 BB60FFFF call    ExitThread
7C81603D  90          nop
7C81603E  90          nop
7C81603F  90          nop
```

Reiniciando Olly y colocando un BP en el push 0x9h (parámetro ThreadInformationClass) podemos tracear la rutina y nos damos cuenta que, efectivamente, el *call dword ptr [ebp+0x8h]* nos lleva al OEP.

Es decir, el salto al OEP que hay en la dirección 0x11416F34h no se utiliza pero si se necesita para cargar el ejecutable, esto es que parece contradictorio tiene su sentido, desde el punto de vista del packer (Themida en este caso) debe, en algún momento, transferir el control de ejecución al ejecutable protegido, aquí sí tiene sentido el JMP a *_CorExeMain*, lo que es menos entendible que después este salto no se pueda modificar cuando no se utiliza.

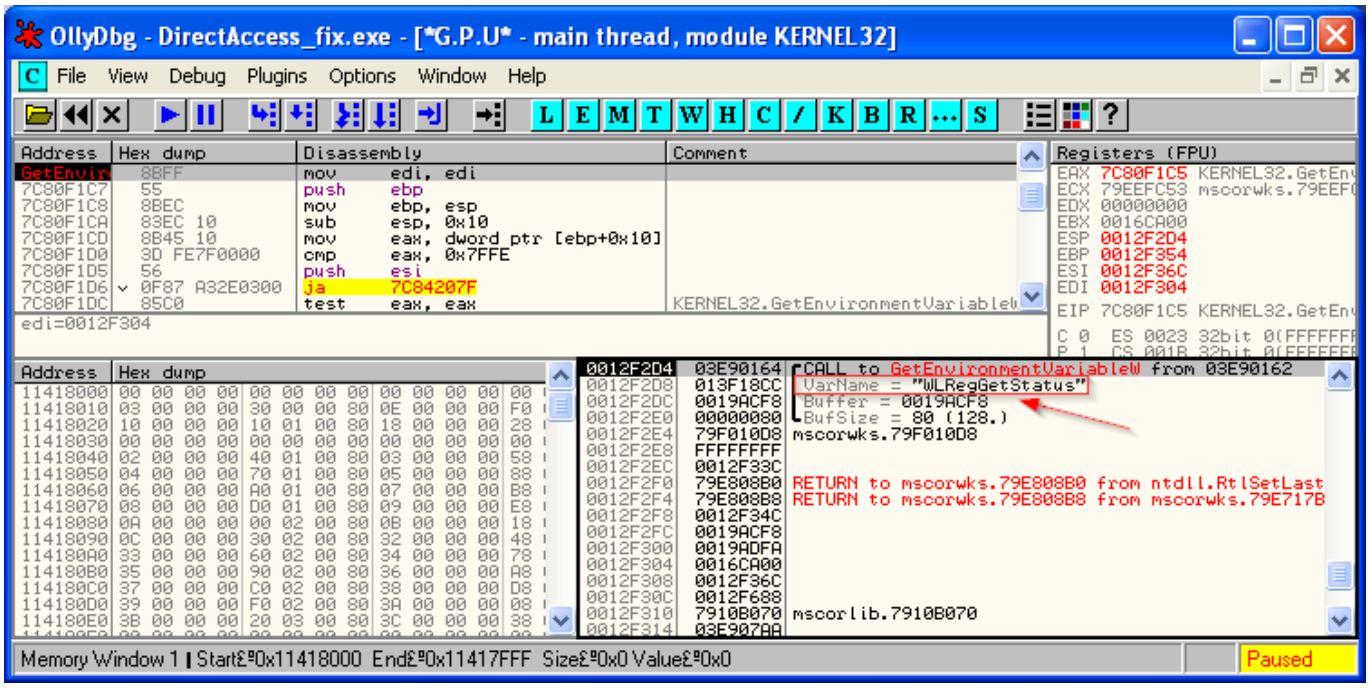
Todo esto no es necesario para resolver el dumpeado, pero sí está muy bien empezar a comprender cómo funcionan los .NET, este es mi primer acercamiento a esta plataforma (por así decirlo) y toda información que pueda extraer es poca.

Por cierto, si pasamos RDG Packer al dumpeado, ahora sí nos muestra que es un .NET:

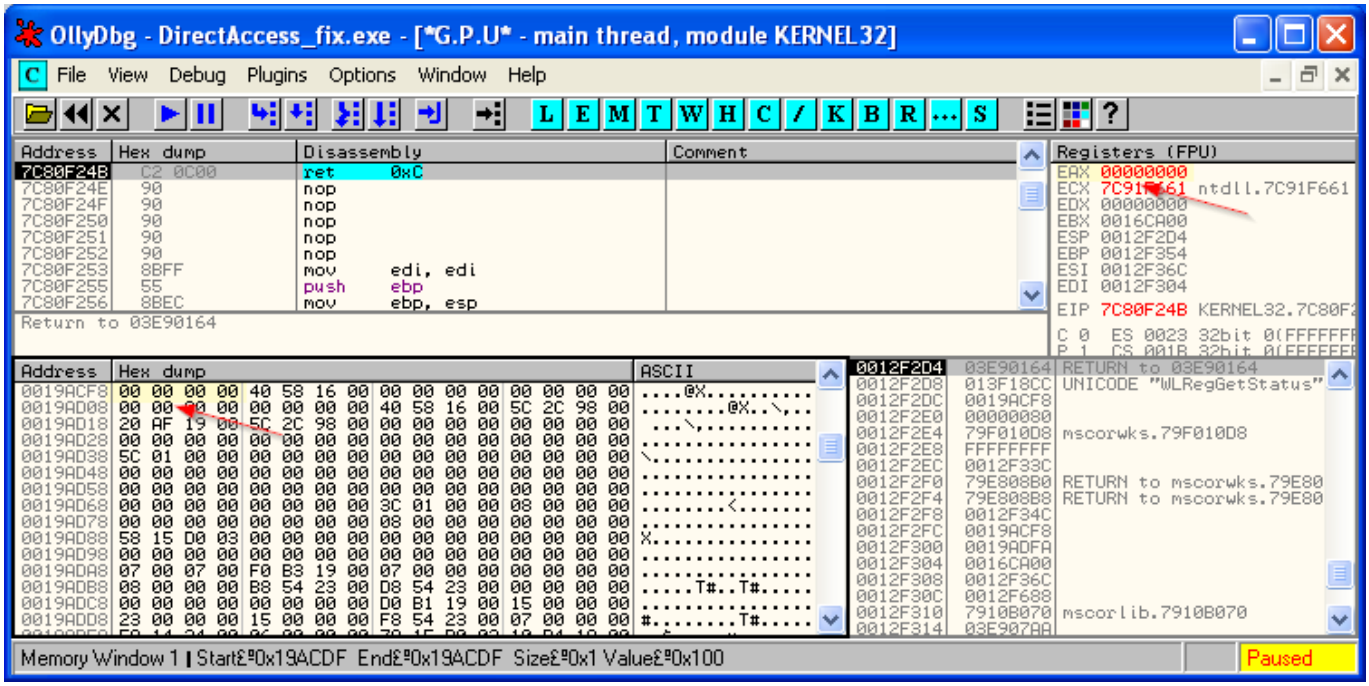



Esto está bien.

Lo siguiente que haremos será tratar de que corra el dumpeado. El cartelito que aparece en pantalla lo controla el WinLicense (está en el otro lado :D), sabemos que en algún momento hará uso de la función nativa GetEnvironmentVariable, abrimos DirectAccess_fix.exe en Olly y colocamos un BP en esta función, en ambas variantes GetEnvironmentVariableA/W (no sabemos si utilizará el formato ASCII o UNICODE), vamos pulsando <F9> cuando vaya parando hasta encontrar lo que buscamos en la pila:



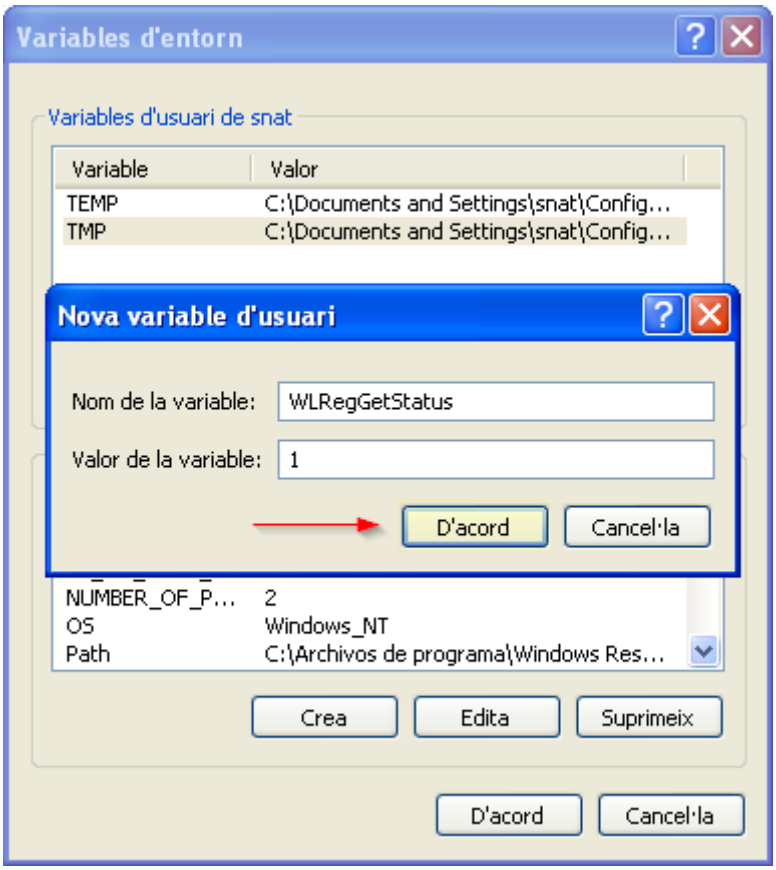

La liebre saltó por GetEnvironmentVariableW (modo UNICODE), vamos a la dirección que marca el buffer 0x0019ACF8h, pasamos el call con <Ctrl + F9> y miramos cómo quedó:



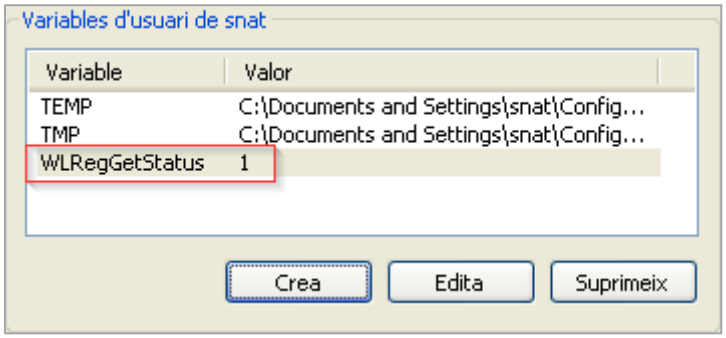

A cero: tanto el buffer como EAX.

Lo siguiente que haremos será crear la variable de entorno nosotros mismos para que la encuentre.

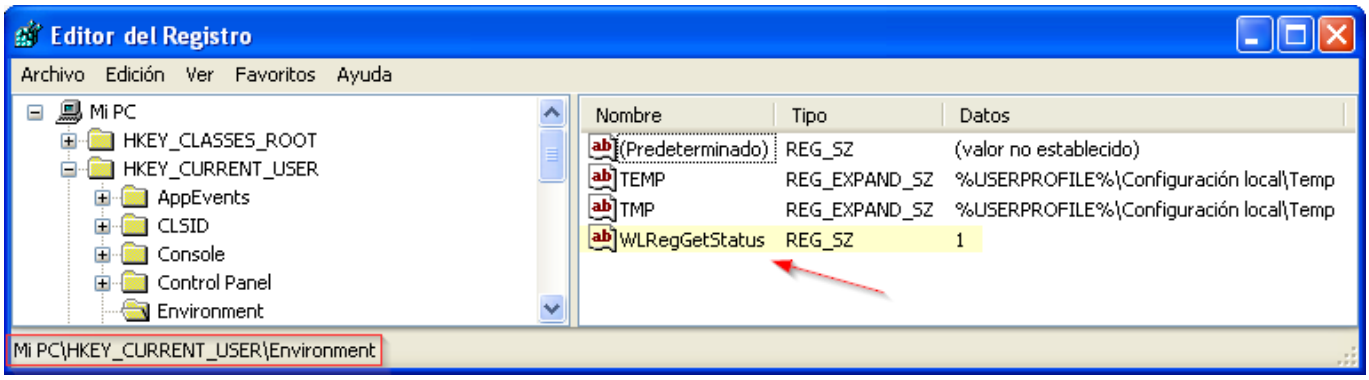
Esto se puede hacer de varias formas, desde las propiedades de MiPc, pestaña Avanzado, botón Variables de Entorno, desde allí la definimos:



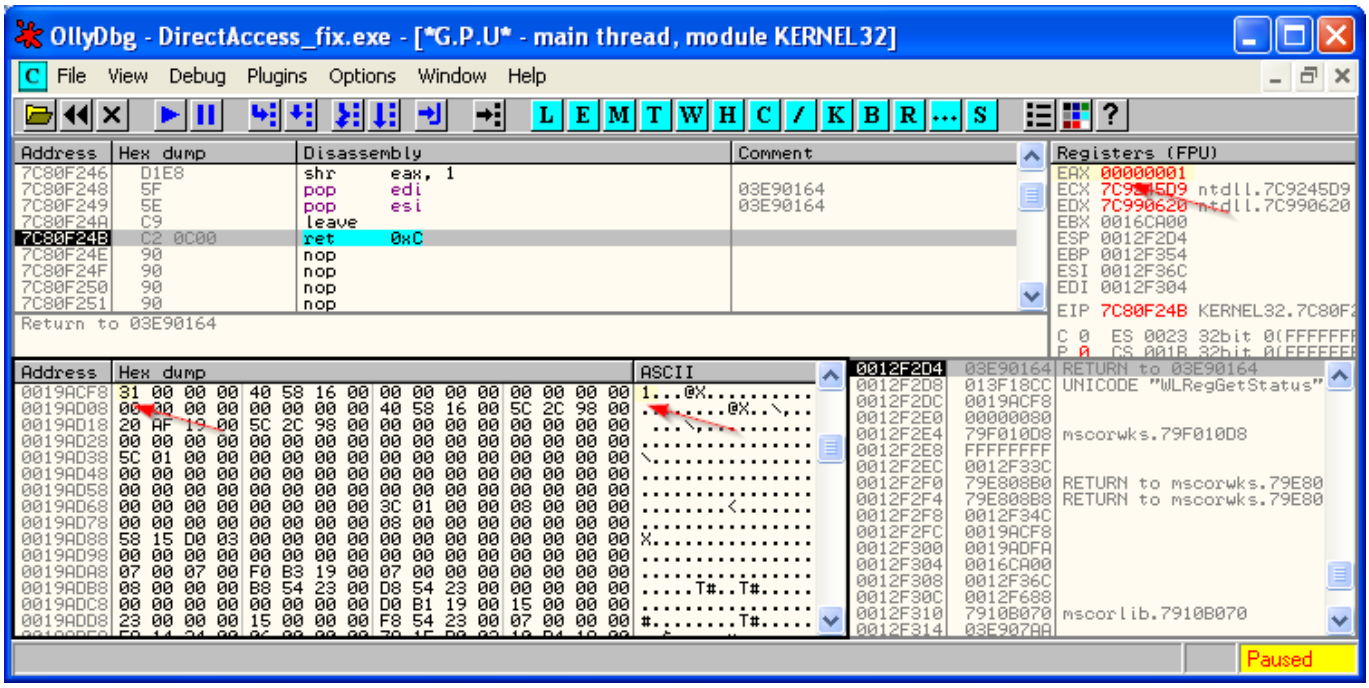

una vez que pulsemos el botón OK ésta se habrá creado:



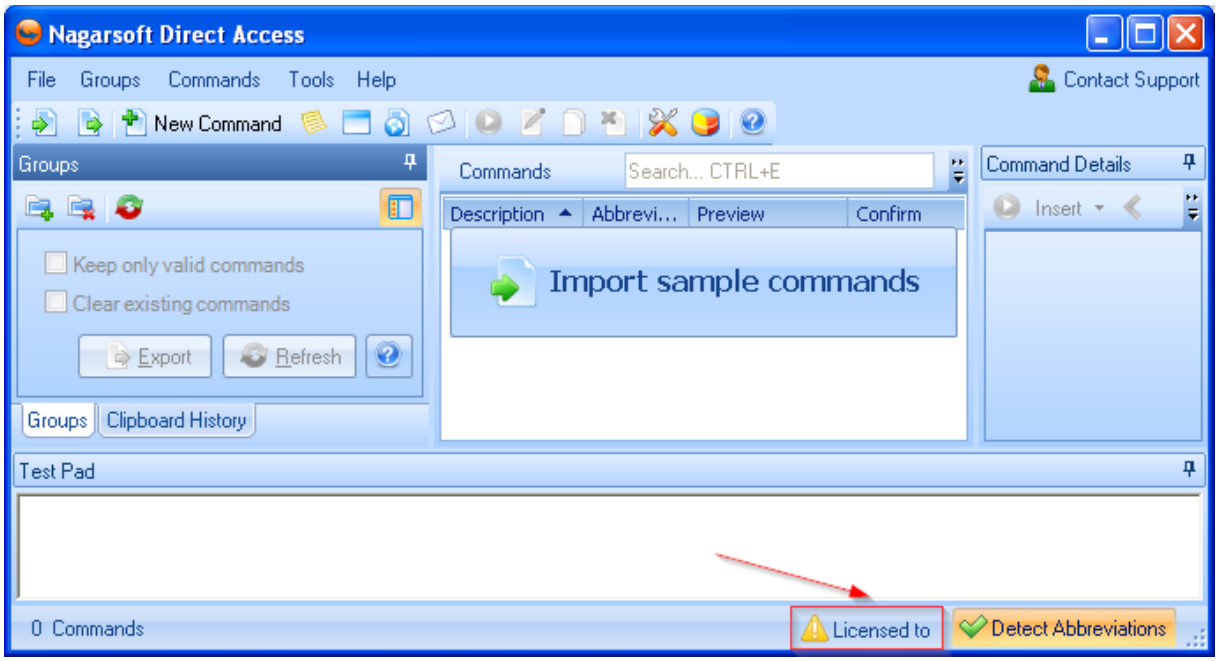

La hemos creado para el usuario actual, ésta se guarda en el registro bajo esta clave:



Reiniciamos Olly y colocamos un BP en GetEnvironmentVariableW, lo dejamos correr y cuando pare con el parámetro WLRegGetStatus pasamos la llamada con <Ctrl + F9> y miramos el buffer y EAX:




De momento la ha encontrado, quitamos el BP, pulsamos <F9> y…




Eiiiiiiiiiiiiiii ¡!!! Aparece la app en el escritorio xDDD

Bueno, ya sabemos qué busca y cómo dárselo, pero estamos con una variable de entorno externa al propio programa, si tratamos de ejecutar el dumpeado en otra máquina NO FUNCIONARÁ puesto que esta variable seguramente no este establecida. Necesitamos encontrar un método con el cual la variable se establezca en cada inicio del programa.

Esto nos lleva al siguiente punto..

# 11. Fijando la variable

Pues sí, vamos a mirar de encontrar la manera de establecer ese WLRegGetStatus a 1 antes de que el programa comience a ejecutarse para que nuestro dumpeado corra.

Esto a priori es bastante trivial en un ejecutable compilado en VB, Delphi, C/C++, ASM,.., pero los principios allí aplicados aquí no valen, ya hemos visto como salta al principio de la función _CorExeMain directamente desde el kernel sin pasar por el ejecutable. Si tratamos de injertar simplemente nuestro injerto será ignorado, o lo que es peor, quizá el proceso ni tan solo se ejecutará.

Entonces, Cómo hacerlo ? Una manera que se me ocurrió fue estudiar la posibilidad de crear un Loader que cree la variable de entorno desde código propio y después lance un proceso para DirectAccess_fix.exe (nuestro dumpeado). Puede servir pensé.

Cómo hacer el Loader? hay muchas posibilidades, de entre todas hay una que era la más apetecible para mí: En ASM con RadASM, este editor se utilizaba ampliamente en otros tiempos, bastaba el más mínimo pretexto para sacar RadASM y ponerte a codificar como loco cualquier rutina en ASM, a lo mejor era más fácil hacerlo en C/C++ pero daba igual, lo interesante era utilizar RadASM. Sólo hay que echar un vistazo a las Teorías y Colaboraciones del grupo para encontrar muchísimos textos donde este editor es utilizado.

Con el paso del tiempo RadASM fue quedando a un lado con la aparición de WinASM (otra variante), pero al fin y al cabo el escribir en ASM estaba a la orden del día: CrackMe's, KeyGenMe's, Loaders, Loaders-Debuggers, KeyGens y un largo etcétera.

Decididamente, esto es lo que aquí haremos, desempolvar los manuales de RadASM y codificar el Loader en ASM, como en los viejos tiempos.

Como es un editor muy poco común en la actualidad lo adjuntaré al tutorial, NO haré lo mismo con MASM32 (que utilizaremos para compilar nuestro pequeño proyecto) debido al peso del mismo.

Antes de empezar a escribir código debemos (y esto es una máxima) estudiar lo que queremos hacer (análisis) y posteriormente trazar las líneas maestras de cómo lo vamos a hacer (diseño), llegando al nivel de detalle que sea necesario.

El análisis lo tenemos prácticamente hecho al estudiar lo que necesita el dumpeado, esto es bien sencillo: la variable de entorno WLRegGetStatus con valor igual a 1. Para pasarle este valor únicamente hay que hacer una llamada a la función SetEnvironmentVariable, su sintaxis es esta:

C++

```
BOOL WINAPI SetEnvironmentVariable(
  _In_     LPCTSTR lpName,
  _In_opt_ LPCTSTR lpValue
);
```

Recibe dos argumentos (parámetros): un puntero al nombre de la variable a establecer y un puntero al valor que tendrá esta variable. A veces cuando busco la especificación de cierta función abro cualquier ejecutable en un Olly y coloco un BP en ella, cuando para analizo la pila y así obtengo los parámetros de ésta. En este caso acudimos al MSDN de Microsoft:

- ✓ https://msdn.microsoft.com/en-us/library/windows/desktop/ms686206%28v=vs.85%29.aspx

Para nada es obligatoria esta parada, basta con conocer la función.

Para darle un poco de emoción a la cosa nuestro cargador hará lo siguiente:

1. Comprobar si la variable de entorno existe.
2. Si no existe la crea con el valor apropiado.
3. Si existe seguirá sin hacer nada.
4. Revisar si el dumpeado existe en el directorio actual (donde correrá el loader).
5. Si no existe lo creará.
6. Si existe se saltará este paso.
7. Creará un proceso para el dumpeado (existente o recién creado).
8. Esperará a que el proceso termine.
9. Si el dumpeado fue creado lo elimina
10. Si no fue creado no hace nada más.

El dumpeado tendrá un nombre específico que le vamos a asignar ahora mismo: DirectAccess_CLS.exe en honor a nuestro grupo y será cargado COMO RECURSO desde el propio cargador, es decir, este dumpeado se cargará con la función LoadResource y existirá como tal en la sección de recursos .rsrc del ejecutable Cargador.exe (nuestro Loader).

Para esta tarea podemos utilizar herramientas externas tipo XN Resource Editor o similar, pero también se puede hacer más artesanalmente, asimismo RadASM nos da la opción de incorporar estos recursos. Sobre la marcha iremos viendo como lo haremos. Por ahora estamos definiendo las líneas generales del proyecto. Más adelante lo refinaremos.

Una vez que tengamos el recurso cargado en memoria se creará el dumpeado (con atributo oculto) en el directorio de instalación, para esto nos podemos ayudar de la función CreateFile doblemente, puesto que la utilizaremos además para comprobar si el dumpeado existe, en ese caso no habría que hacer nada. En el supuesto de que no exista tras crear el archivo (vacío: 0 Kb) habrá que completar su contenido, esto lo haremos con la api WriteFile, con ello conseguiremos un dumpeado funcional, liberaremos el manejador de éste con la función CloseHandle y crearemos el proceso que lo lanzará con CreateProcess.

A partir de aquí el .NET correrá en un segundo proceso y nuestro cargador quedará a la espera del cierre del .NET para terminar su trabajo (eliminar el archivo dumpeado si y sólo si lo tuvo que crear). He de decir que en un primer momento me decanté más por la opción Loader-Debugger, con unos mínimos ajustes se consigue, modificando el parámetro CreationFlags en la llamada a CreateProcess estableciéndolo a DEBUG_PROCESS+DEBUG_ONLY_THIS_PROCESS y quedando en un loop en WaitForDebugEvent (parámetro -0x1h equivalente a INFINITO) y procesando los eventos del proceso depurado, todo ello está muy bien detallado en las teorías del grupo.

Pero este esquema no me funcionó en el .NET, resulta que se generan tres excepciones (quitando la primera que es la parada en el System BreakPoint) las otras dos impiden que corra el proceso, probé DE

TODO pero no hubo manera, dudaba del Loader-debugger así que lo probé con el UnpackMe que utilizamos en la parte1 de este texto (el de VB) y funcionaba bien, cosas del .NET :(

En este punto me detuve a estudiar qué posibilidades nos quedaban, la idea era quedar a la espera de la finalización del proceso para borrar el ejecutable dumpeado del directorio de instalación, en el tutorial de Armadillo (Automation Anywhere 6.1) lo conseguimos interceptando la api user32.ExitProcess, pero como no me gusta repetir los procedimientos, sino más bien lo contrario, abrir nuevas líneas, lo intenté mediante la función WaitForSingleObjet, cuya sintaxis es la siguiente:

C++

```
DWORD WINAPI WaitForSingleObject(
  _In_ HANDLE hHandle,
  _In_ DWORD  dwMilliseconds
);
```

Necesita el manejador del objeto en cuestión y el tiempo de espera (especificado en milisegundos). El tiempo de espera será infinito (-0x1h) y el manejador lo podemos obtener de la estructura PROCESS_INFORMATION que le pasamos a la función CreateProcess, en concreto el campo *hProcess*:

```
push -0x1h
push processInformation.hProcess
call WaitForSingleObject
```

Hay que definir previamente la variable processInformation como estructura PROCESS_INFORMATION para que la llamada funcione.

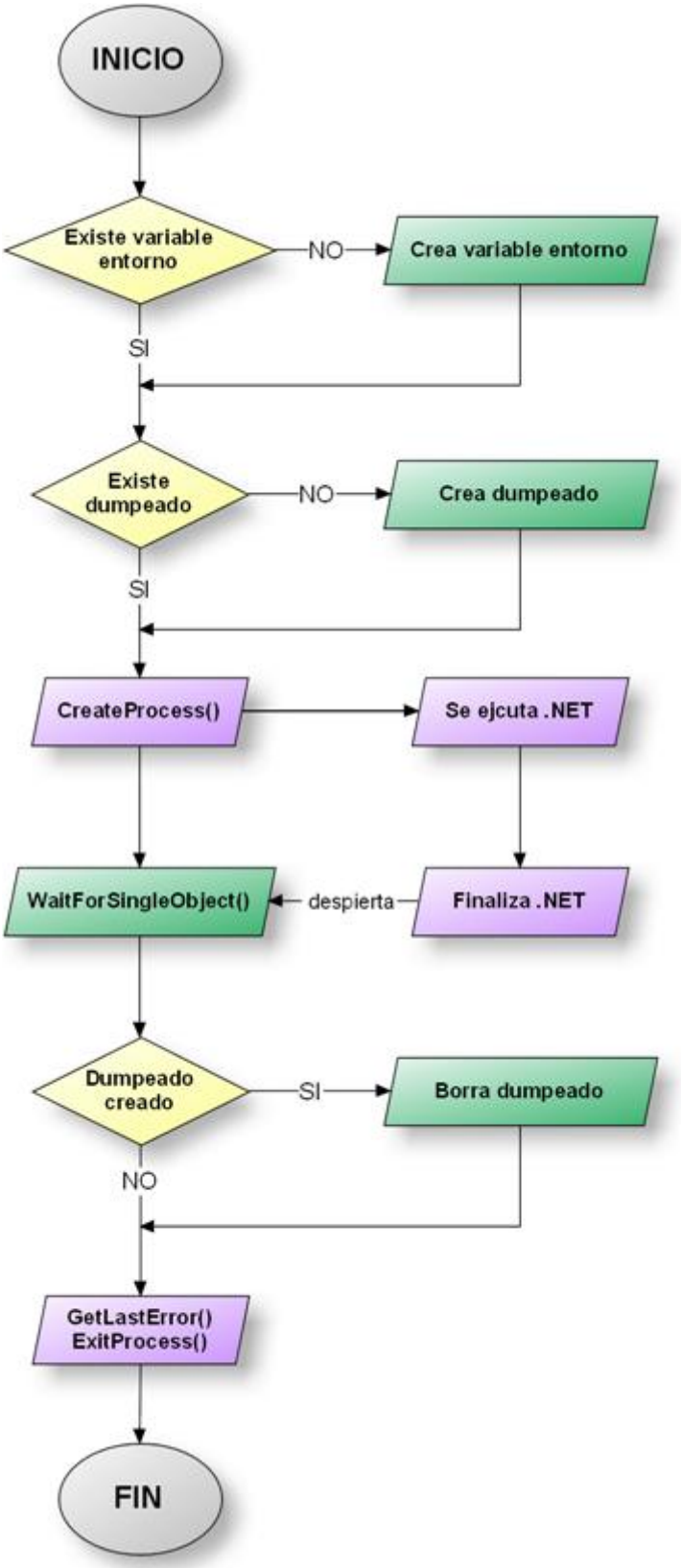
Cuando despierte (lo que significa que el proceso del .NET terminó) proseguirá su ejecución y revisará la variable booleana existencia que mantiene información acerca de si se tuvo que crear el dumpeado (CreateFile + WriteFile) en el directorio de instalación, si tuvo que hacerlo lo eliminará y si no lo tuvo que hacer no hará nada al respecto y terminará su ejecución.

He de añadir que hice una aproximación tratando de modificar la llamada a CreateFile, parámetro Attributes tratando de crear el archivo de manera que se eliminara automáticamente al cerrar el proceso, lo intenté con Attributes=FILE_ATTRIBUTE_HIDDEN+FILE_FLAG_DELETE_ON_CLOSE y se creaba el archivo bien, pero fallaba la llamada a CreateProcess porque el archivo no existía, se eliminaba tras la llamada a CloseHandle, cosa que es normal, el archivo se elimina cuando se cierra el último manejador a éste. Si no cerraba el manejador la llamada a CreateProcess también fallaba. Al final por esto me decanté por la opción de WaitForSingleObject arriba mostrada y con la que trabajaremos.

Como se puede observar, detrás de un tutorial hay muchísimos hostiazos que muchas veces no mostramos con tal de escribir el texto rápido y publicarlo. Este no es un buen estilo, es preferible dejar este tipo de documentación escrita ya que es tan o más válida que la que te lleva directamente a la línea de meta.

Como dijo un sabio: *de los errores se aprende*.

Veamos la lógica del programa en un diagrama de flujo:



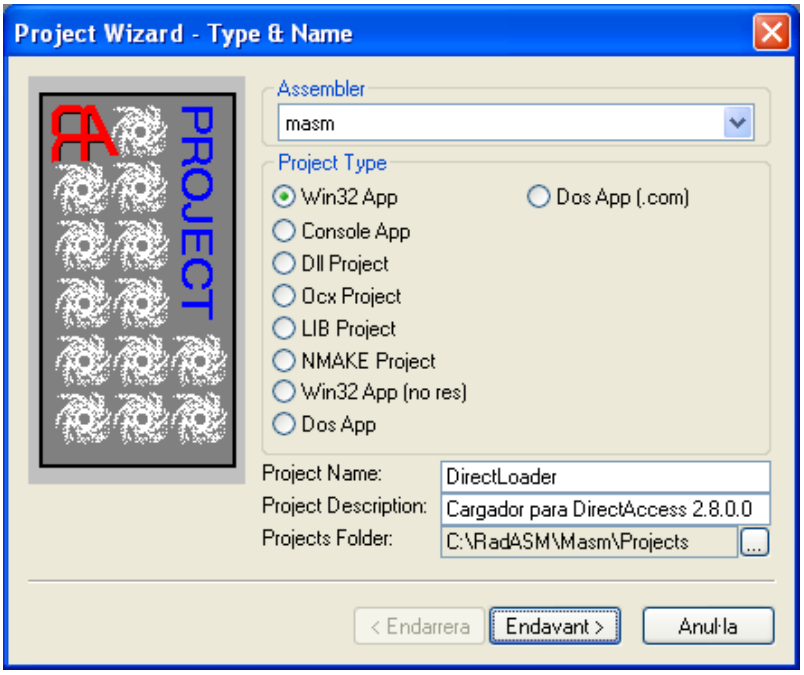

Sólo nos queda escribirlo, vamos a la siguiente etapa..

# 12. RadAsm + MASM32

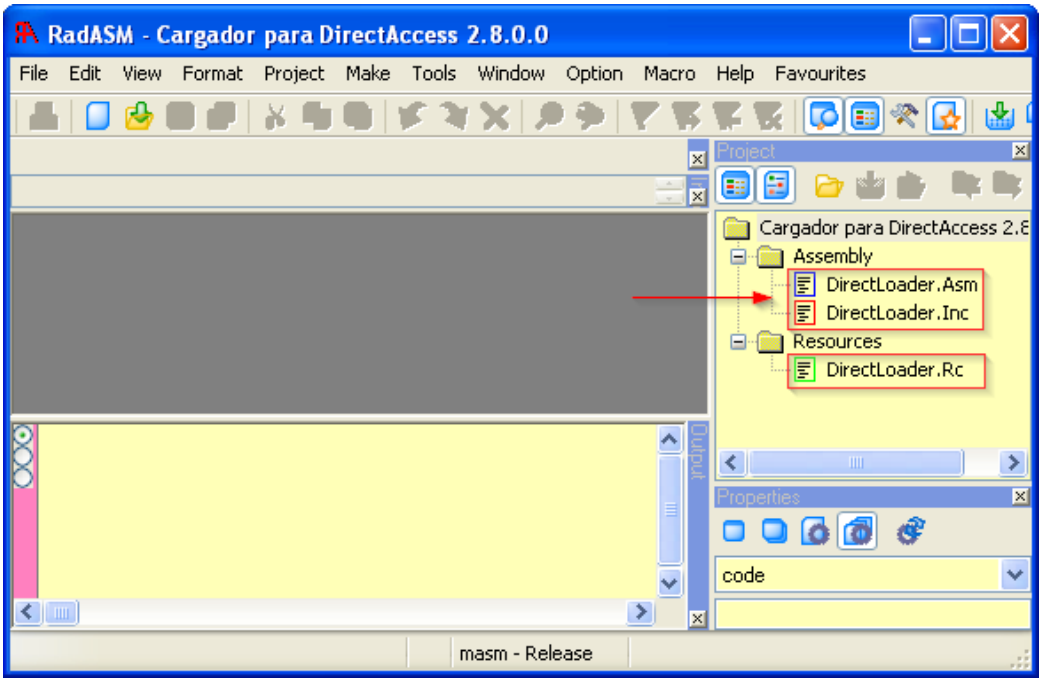
Pues sí, ha llegado el momento de implementar esta función (programa).

Antes de nada copiamos la carpeta donde están los binarios de MASM32 a C:\ quedando C:\MASM32, y aunque no adjuntaré el compilador (pesa bastante ~30 MB el conjunto) es muy fácil de obtener.

Hecho esto, ahora así abrimos RadASM (en mi caso está en C:\ pero lo puedes ubicar donde prefieras, siéntete libre.), vamos a *File -> New Project* y le ponemos un nombre al mismo:




Dejamos marcada la opción *Win32 App* como en la imagen, después pulsamos Siguiente en las sucesivas ventanitas que vayan saliendo, al final se abre el editor donde podemos ver nuestro proyecto (vacío):



En el marco derecho se ven referencias a tres archivos:

1. DirectLoader.Asm: Contendrá el código ejecutable del proyecto.
2. DirectLoader.Inc: Donde se definirán las constantes y variables.
3. DirectLoader.Rc: Hará referencia a los recursos que incluyamos en el proyecto.

En el fichero DirectLoader.Asm escribimos el código:

```
1   .386
2   .model flat, stdcall
3   option casemap :none
4   include DirectLoader.inc
5
6   .code
7
8  Inicio:
9
10  invoke GetEnvironmentVariable,addr nombre_variable,addr buffer_variable,TAMANO_BUFFER
11  test eax, eax
12  jne sigue
13  invoke SetEnvironmentVariable,addr nombre_variable,addr valor_variable
14
15 sigue:
16  invoke CreateFile,addr offset_nombre,GENERIC_READ,FILE_SHARE_READ,NULL,\
17                    OPEN_EXISTING,FILE_ATTRIBUTE_HIDDEN,0
18  inc eax
19  test eax, eax
20  jne existe
21
22  mov existencia, FALSE
23  invoke GetModuleHandle,NULL
24  mov manejador_proceso, eax
25
26  invoke FindResource,manejador_proceso,DUMPEADO,RT_RCDATA
27  mov manejador_recurso, eax
28
29  invoke SizeofResource, manejador_proceso, eax
30  mov tamano_recurso, eax
31
32  invoke LoadResource, manejador_proceso, manejador_recurso
33  mov direc_recurso, eax
34
35  invoke CreateFile,addr offset_nombre,GENERIC_WRITE,FILE_SHARE_WRITE+FILE_SHARE_DELETE,\
36                    NULL,CREATE_ALWAYS,FILE_ATTRIBUTE_HIDDEN,0
37  mov manejador_archivo, eax
38  invoke WriteFile,eax,direc_recurso,tamano_recurso,addr bytes_escritos,NULL
39  invoke CloseHandle,manejador_archivo
40
41 existe:
42  invoke CreateProcess,addr offset_nombre,NULL,NULL,NULL,NULL,NULL,NULL,NULL,\
43                       addr StartupInfo,addr ProcessInformation
44  test eax,eax
45  jz Problema_CreateProcess
46
47  invoke WaitForSingleObject,ProcessInformation.hProcess, INFINITE
48
49  cmp existencia, TRUE
50  je Salida
51
52  invoke CloseHandle,ProcessInformation.hProcess
```

```
53  invoke Sleep,5000
54  invoke DeleteFile,addr offset_nombre
55  jmp Salida
56
57 Problema_CreateProcess:
58  invoke MessageBoxA,NULL,offset error_cProcess,offset Titulo, MB_OK+MB_ICONEXCLAMATION
59  jmp Salida
60
61 Problema_ExeNoEncontrado:
62  invoke MessageBoxA,NULL,offset error_exe,offset Titulo, MB_OK+MB_ICONEXCLAMATION
63  jmp Salida
64
65 Salida:
66  invoke GetLastError
67  invoke ExitProcess,eax
68
69 END Inicio
```

Este sería el código *base* de nuestro proyecto, pero si tratamos de compilar dará error, necesitamos agregar las definiciones de constantes y variables que vamos a utilizar, así como incluir los archivos .inc y .lib para poder hacer uso de las funciones nativas de sistema (MessageBox, etc.).

El contenido del archivo DirectLoader.Inc será el siguiente:

```
1  ; ##########################################################################
2
3   include windows.inc
4   include kernel32.inc
5   include user32.inc
6
7   includelib kernel32.lib
8   includelib user32.lib
9
10 ; ##########################################################################
11 .const
12  DUMPEADO           equ 5
13  TAMANO_BUFFER      equ 4
14
15 .data
16  offset_nombre      db 'DirectAccess_CLS.exe',0
17  error_salida       db 'Termina ejecución del Loader',0
18  Titulo             db 'Loader para Direct Access 2.8.0.0',0
19  error_exe          db 'Ejecutable no encontrado!',0
20  error_cProcess     db 'Error CreateProcess!',0
21  nombre_variable    db 'WLRegGetStatus',0
22  valor_variable     db '1',0
23  manejador_archivo  dd ?
24  manejador_recurso  dd ?
25  manejador_proceso  dd ?
26  tamano_recurso     dd ?
27  direc_recurso      dd ?
28  buffer_variable    dd ?
29  bytes_escritos     dd ?
30  existencia         BOOL TRUE
31
32 .data?
33  StartupInfo        STARTUPINFO <>
34  ProcessInformation PROCESS_INFORMATION <>
```

De este archivo decir varias cosas, en el primer bloque debemos agregar las directivas include y includelib de aquéllos archivos de los cuales queremos utilizar funciones, si queremos utilizar por ejemplo *GetModuleHandle* debemos agregar include kernel32.inc y includelib kernel32.lib, esto será necesario para todas las librerías de las cuáles vayamos a invocar funciones.

Esto por un lado, por otra parte viene después la definición de constantes con directiva .const, en ella deberemos colocar el nombre de la constante seguido de su valor (ese *equ* significa igualdad, *equals*), en el texto *DUMPEADO equ 5* significa que declaramos la constante DUMPEADO al que se le asigna el valor 5.

La siguiente directiva es .data, en ella definimos el nombre de las variables que vayamos a necesitar, seguido de su tipo (db, dd, BOOL, etc..) y opcionalmente, de su valor en caso de querer inicializar la variable, en caso de no querer hacerlo se pondrá el signo ? tras el tipo de variable, así por ejemplo:

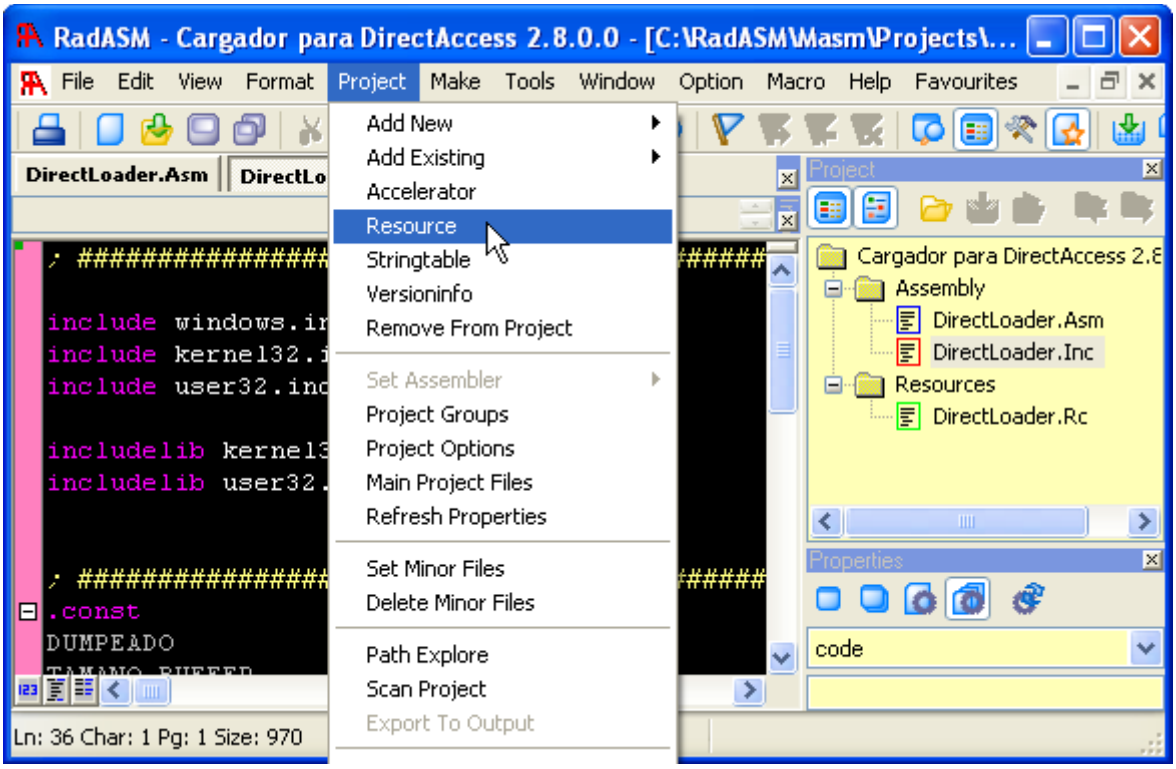
```
valor_variable      db '1',0
```

Estamos indicando al compilador que declaramos una variable con nombre valor_variable, que será tipo byte (db) y se inicializará con valor 1.

Por último está la directiva .data?, como se ve es igual que la anterior pero con el signo ? al final, aquí debemos declarar las variables que NO van a ser inicializadas, normalmente definiremos variables a las que se le asignan ciertas estructuras como las que allí vemos: STARTUPINFO, PROCESS_INFORMATION, CONTEXT, etc.., no se inicializan precisamente porque lo hará la API de turno que invoquemos y que recibirá esta estructura como parámetro.

Hasta aquí creo que se entiende.

Lo siguiente que nos quedaría será agregar los recursos que queramos que acompañen a nuestro pequeño ejecutable (ya veréis que el código ASM genera unos exes realmente pequeños). RadASM incorpora esta opción:

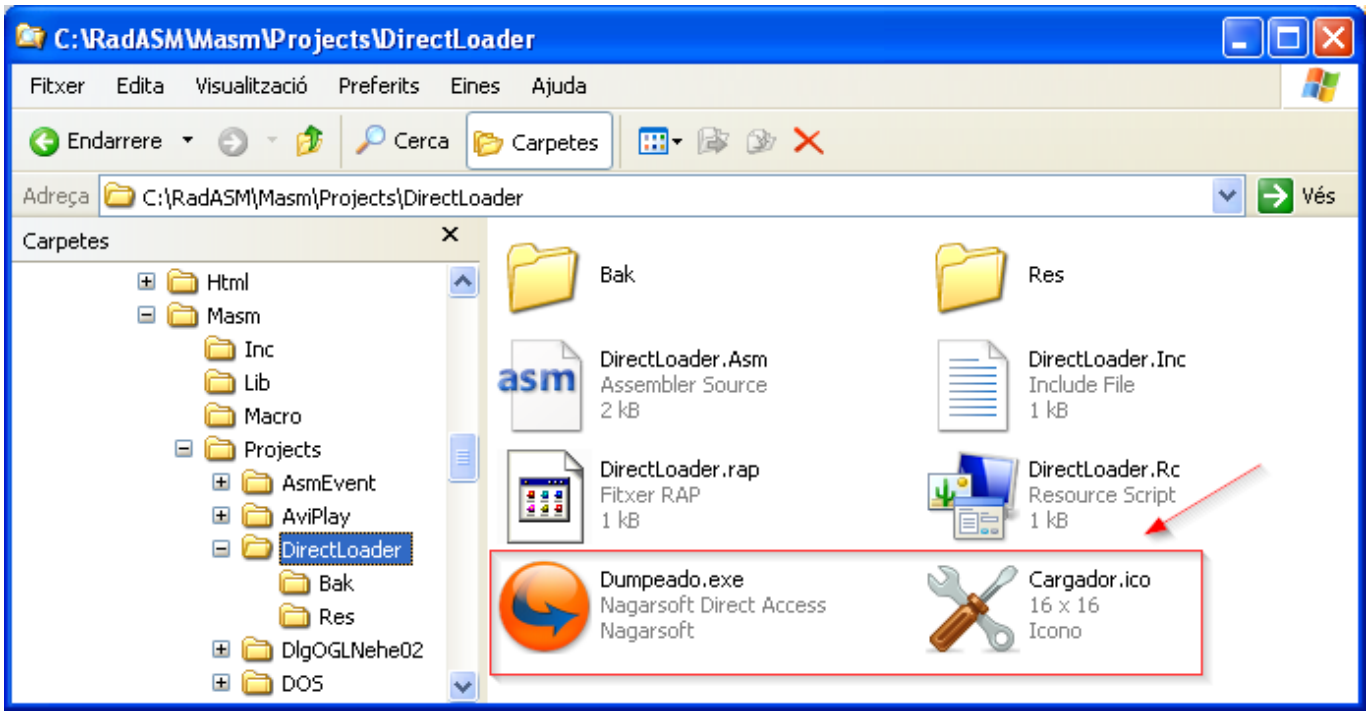


Esto abrirá una ventanita desde donde agregar los recursos. Antes de nada necesitamos un icono muy chulo que vamos a poner y que obtendremos por la vía rápida: bajado de internet o bien con ResourceHacker extraído de cualquier otro ejecutable, ahora veréis el que hemos puesto: una llave inglesa y un tornavís, que tiene que ver con los Loaders ¿?? nada, pero igual lo ponemos :=)

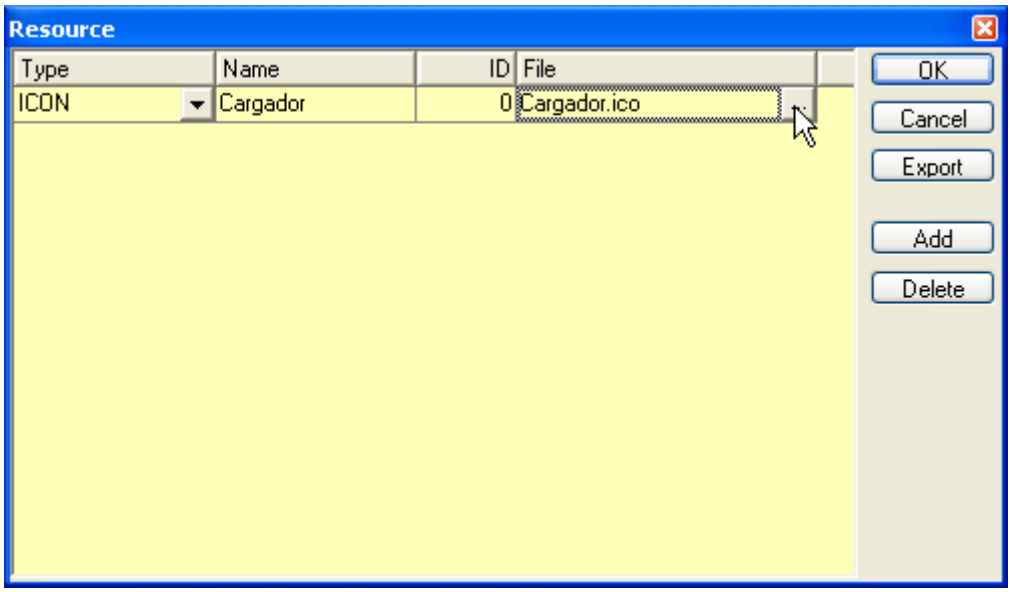
Copiamos el icono (cada uno que ponga el que vea conveniente) al directorio del proyecto, en mi caso:

- ✓ C:\RadASM\Masm\Projects\DirectLoader

Lo copio allí. Hago lo mismo con el dumpeado DirectAccess_fix.exe que nos dejó MegaDumper + UniversalFixer en la etapa 9 (Dumpeando lo indumpeable) que nos recomendó sequeyo. Lo renombramos como Dumpeado.exe, nos debe de quedar así:




Agregamos el icono desde *RadASM -> Project -> Resource* y botón *Add*. Lo dejamos así:



En Type hemos puesto ICON, en Name uno cualquiera, el ID hemos dejado el 0 y en File hay que pinchar el botón de los puntos suspensivos (donde está el cursor en la imagen) y seleccionar el icono que acabamos de copiar en el directorio del proyecto (Cargador.ico).

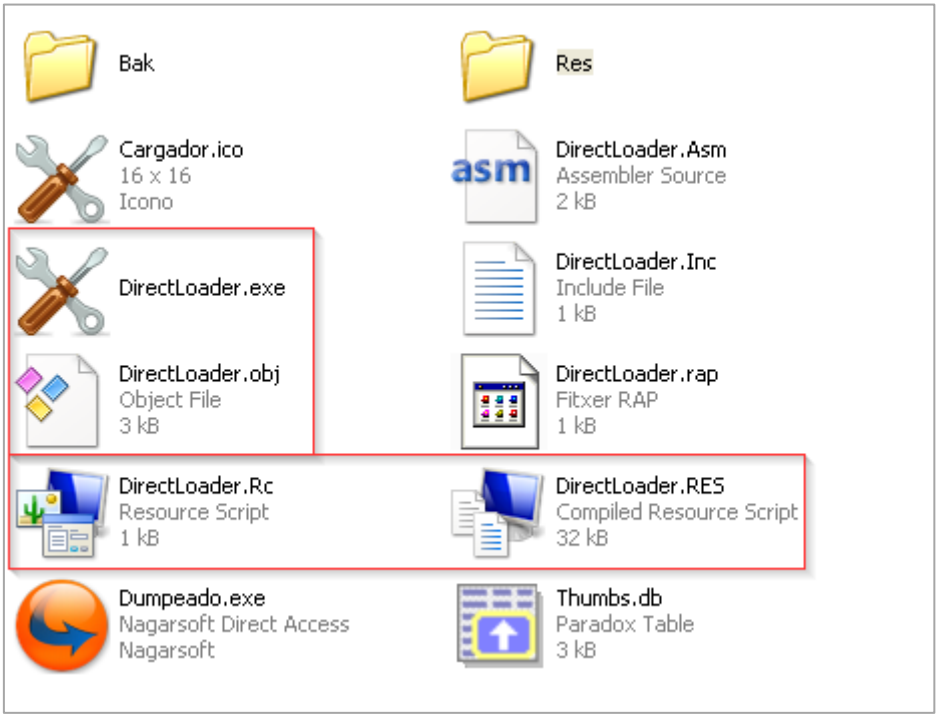
Pulsamos OK y el recurso habrá sido agregado, para confirmarlo podemos mirar el archivo DirectLoader.rc con cualquier editor de textos (notepad o el que sea) y veremos esta línea:

```
1  #include "Res/DirectLoaderRes.rc"
```

Hace referencia a un segundo archivo llamado DirectLoaderRes.rc y que está en el subdirectorio /Res dentro del directorio del proyecto. Si abrimos este archivo con un editor de texto veremos su contenido:

```
1 Cargador ICON DISCARDABLE "Cargador.ico"
```

Aquí está la información que necesita para cargar el recurso, si compilamos el proyecto desde el botón y miramos el directorio del proyecto nuevamente veremos más detalles:




Han aparecido más archivos: DirectLoader.obj que contiene el código objeto, los archivos Rc y Res que contienen información acerca de los recursos y sobretodo el DirectLoader.exe que es el ejecutable generado durante las fases de compilación y de enlace (compiling & linking phases), el compilador traduce el código fuente a código objeto (generando el archivo DirectLoader.obj) y el enlazador o linker como su propio nombre indica toma este código fuente y lo enlaza a las librerías (dlls) referidas en el código, en esta segunda fase se genera el ejecutable DirectLoader.exe

Lo siguiente que vamos a hacer es agregar el Dumpeado como un recurso más del proyecto, esto se hace completamente igual que para el Icono salvo el Tipo que es RCDATA como en la imagen:

| Type | Name | ID | File |
|---|---|---|---|
| ICON | Cargador | 0 | Cargador.ico |
| RCDATA | Dumpeado | 5 | Dumpeado.exe |

Notad que el ID hemos colocado 5, si miramos el archivo DirectLoader.inc encontramos la constante:

```
11 .const
12  DUMPEADO            equ 5
13  TAMANO_BUFFER       equ 4
```

La hemos definido a conciencia para poder hacer referencia al mismo desde el código ejecutable que contiene el archivo DirectLoader.Asm como se puede comprobar:

```
25
26  invoke FindResource,manejador_proceso,DUMPEADO,RT_RCDATA
27  mov manejador_recurso, eax
28
```

Si miramos el archivo DirectLoaderRes.rc encontraremos otra diferencia:

```
1 #define Dumpeado 5
2 Cargador ICON DISCARDABLE "Cargador.ico"
3 Dumpeado RCDATA DISCARDABLE "Dumpeado.exe"
```

Se ha agregado contenido, justo lo que le pedimos: incluir Dumpeado.exe como recurso, además hace alusión al ID con valor 5 que le pusimos.

Si miramos el directorio del proyecto veremos que el tamaño del archivo DirectLoader.RES ha aumentado bastante, tanto como 4.410 Kb, justo el tamaño que tiene el dumpeado:

| Nom | Mida | Tipus |
|---|---|---|
| Bak | | Carpeta de archivos |
| Res | | Carpeta de archivos |
| Cargador.ico | 32 kB | Icono |
| DirectLoader.Asm | 2 kB | Assembler Source |
| DirectLoader.exe | 4.444 kB | Application |
| DirectLoader.Inc | 1 kB | Include File |
| DirectLoader.obj | 3 kB | Object File |
| DirectLoader.rap | 1 kB | Fitxer RAP |
| DirectLoader.Rc | 1 kB | Resource Script |
| DirectLoader.RES | 4.442 kB | Compiled Resource Script |
| Dumpeado.exe | 4.410 kB | Application |
| Thumbs.db | 3 kB | Paradox Table |

Podemos entonces, agregar recursos manualmente con sólo modificar estos ficheros. Hay otra manera de hacerlo que es incluir los recursos uno a uno, cada recurso tendrá asociado un archivo .Res y se agregarán tantas directivas include en DirectLoader.Rc como sean necesarias, como se puede ver hay muchas formas distintas de llegar al mismo resultado. Compilamos nuevamente el proyecto:

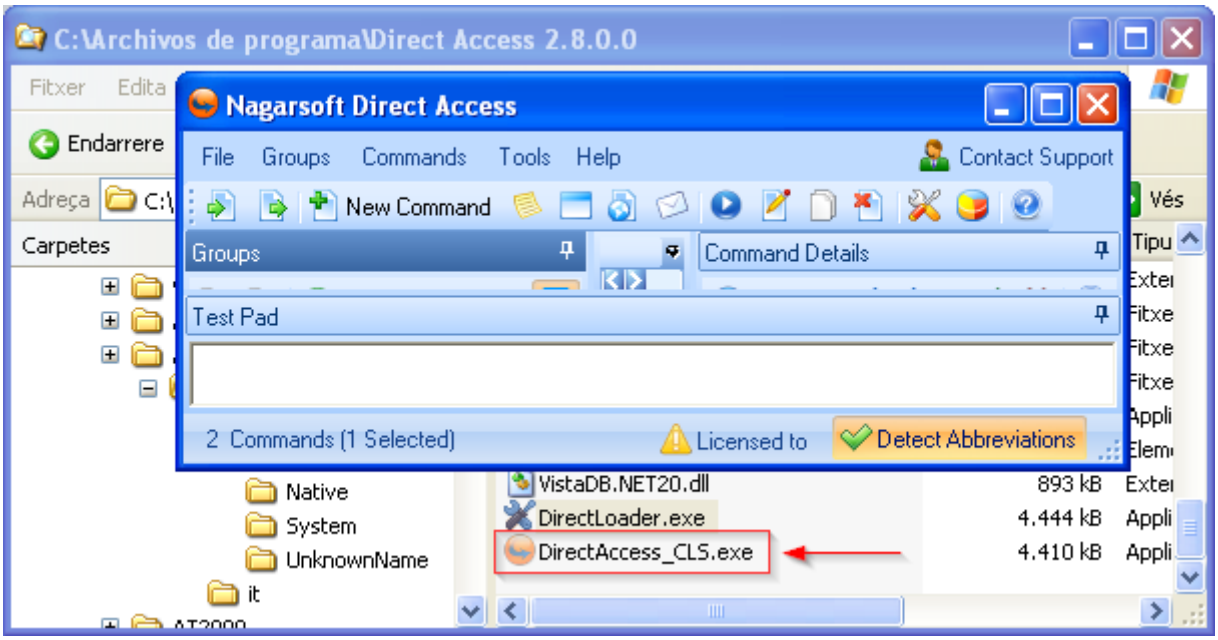
```
Copyright (C) Microsoft Corp 1992-1998. All rights reserved.


Make finished.
Total compile time 157 ms
```

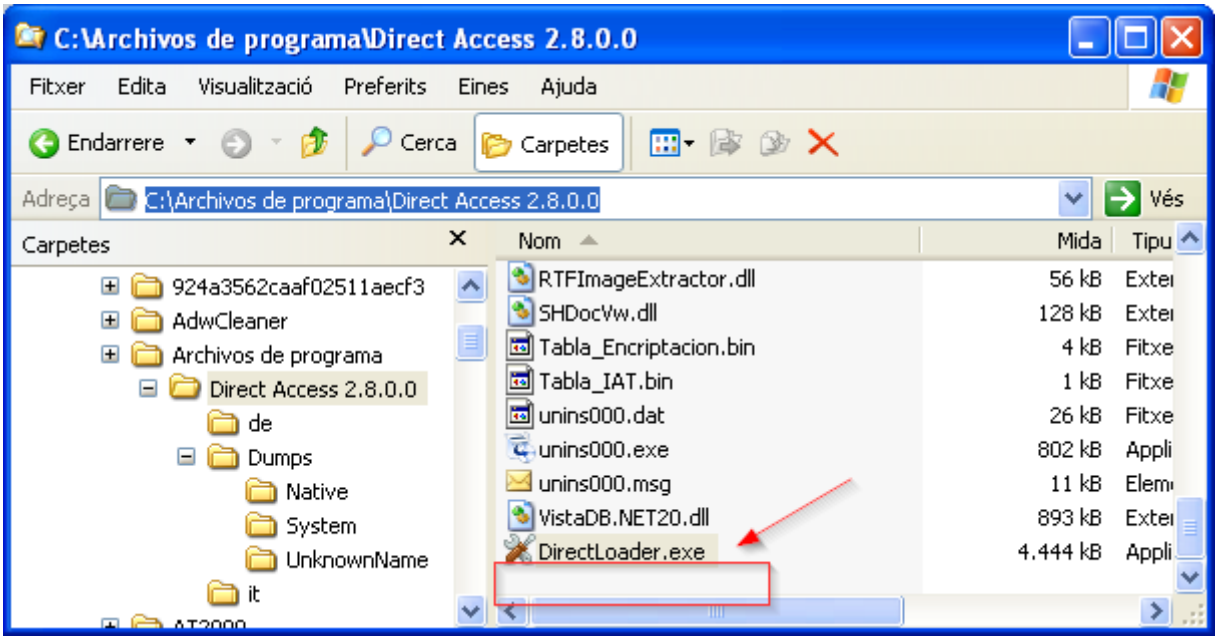
Copiamos el ejecutable DirectLoader.exe desde:

- origen: C:\RadASM\Masm\Projects\DirectLoader
- destino: C:\Archivos de programa\Direct Access 2.8.0.0

Y lo probamos:



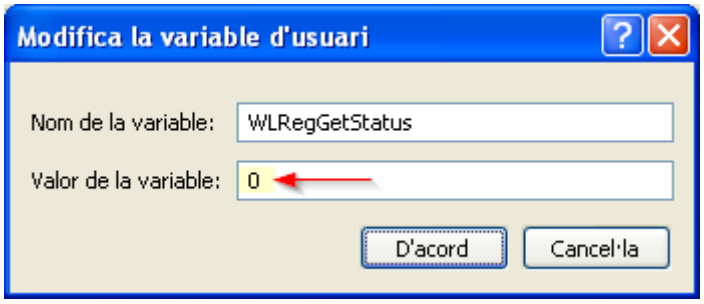

Cerramos el programa y miramos otra vez:



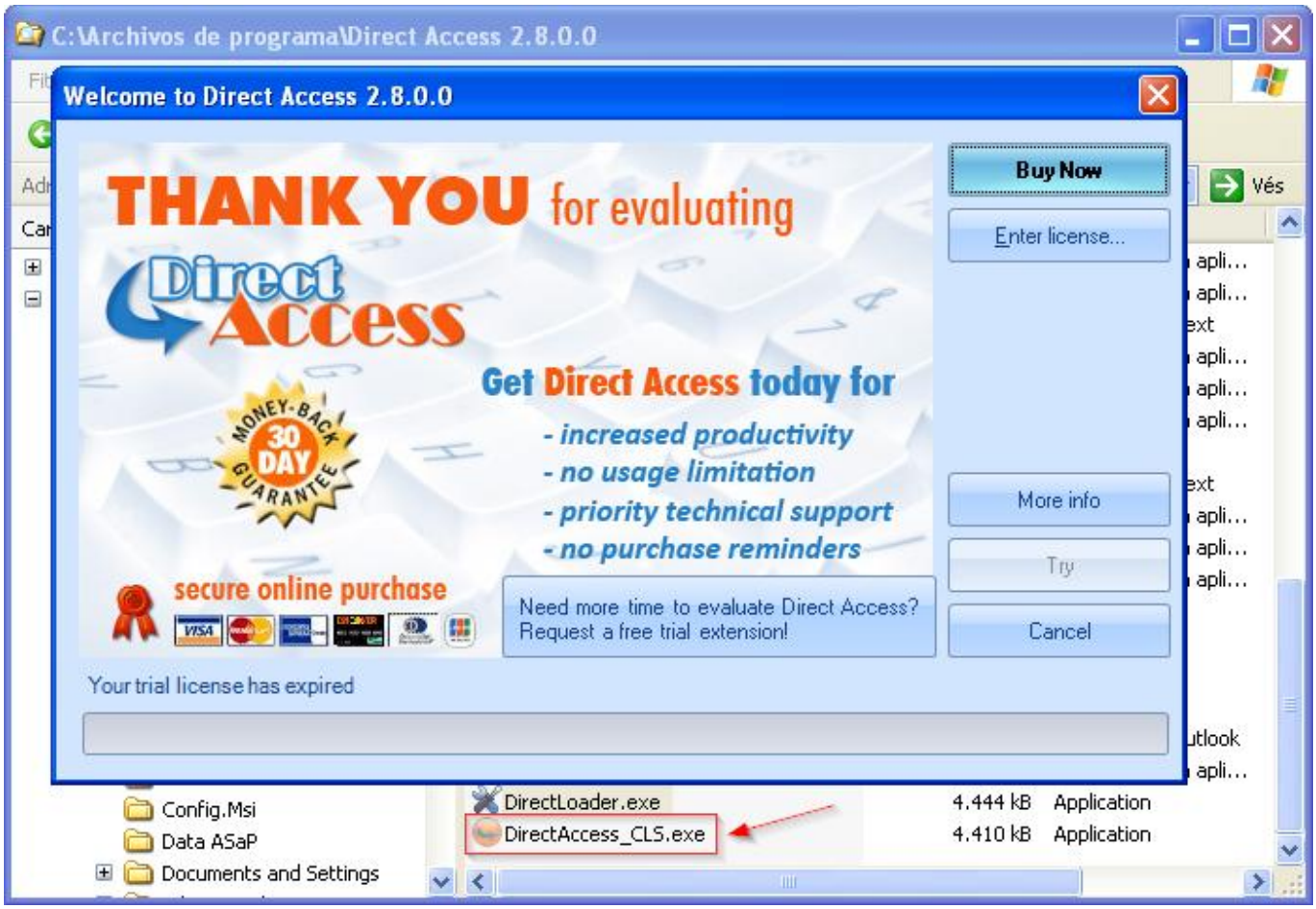

Flap ¡!!! Se borró :)

Si lo ejecutamos sin variable de entorno funciona perfecto puesto que la crea el propio cargador. Si lo ejecutamos con variable de entorno a 1 como hasta ahora todo bien igual. Pero hay un supuesto que todavía no habíamos previsto: Que la variable de entorno exista y NO tenga valor 1.

Hagamos la prueba, la modificamos para que tenga cualquier valor diferente de 1:

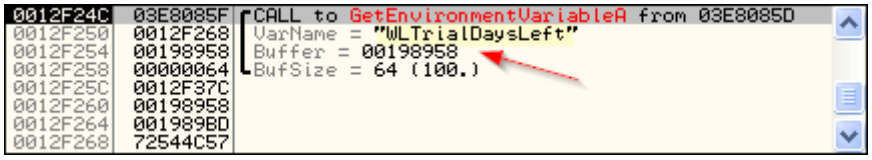



Ejecutamos nuestro cargador y…



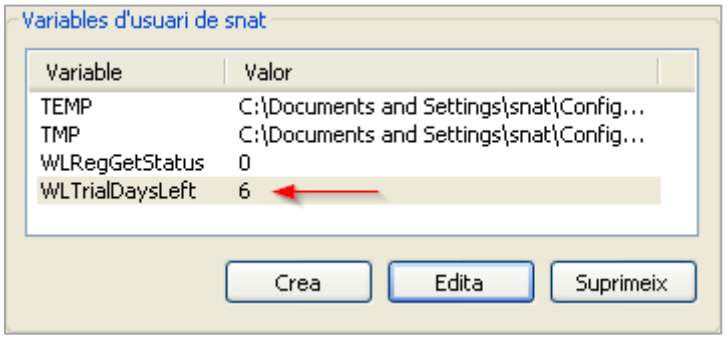

El programa arranca EXPIRED :(

Esto no lo habíamos previsto. Busquemos soluciones, carguemos el ejecutable DirecAccess_CLS.exe en OllyDbg y coloquemos un HBP en el par GetEnviromentVariableA/W, damos RUN y Olly va parando, esperamos a que aparezca algo llamativo como esto:



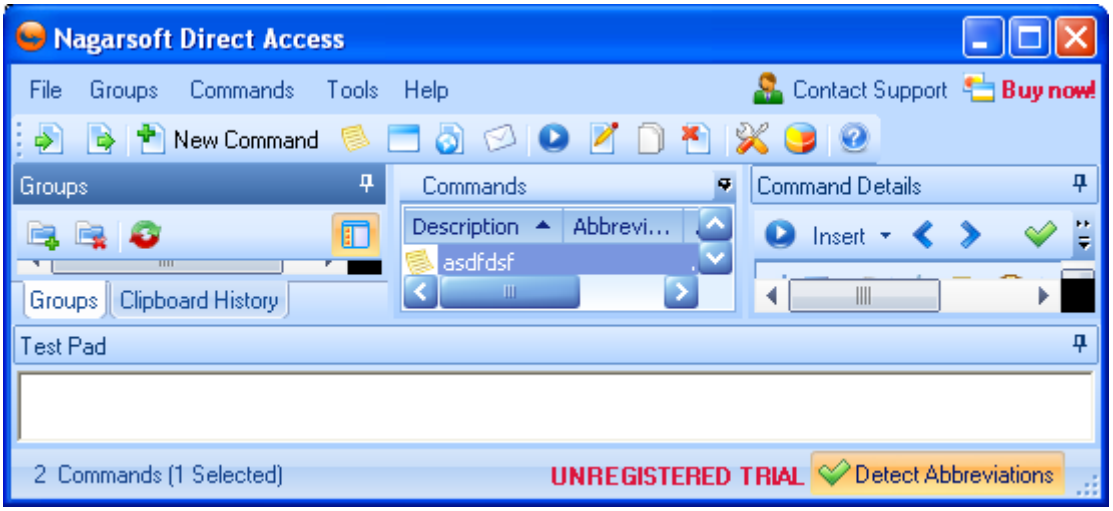

Está cotejando otra variable WLTrialDaysLeft, genial, hagamos otra prueba:

Creemos esta variable y le damos cualquier valor (mayor que o claro):




Ejecutemos nuevamente nuestro dumpeado:

Si pulsamos el botón *Try* el programa realmente se ejecuta:



El modo más sencillo que encuentro de superar este obstáculo es creando SIEMPRE la variable (exista o no exista), basta con eliminar este salto en el código fuente y recompilar:

```
 8 Inicio:
 9
10  invoke GetEnvironmentVariable,addr nombre_variable,addr buffer_variable,TAMANO_BUFFER
11  test eax, eax
12  jne sigue
13  invoke SetEnvironmentVariable,addr nombre_variable,addr valor_variable
14
```

Otra opción sería comprobar el valor de la variable SI existe y si no es igual a 1 entonces crearla. hay muchísimas opciones.

Por ejemplo con esta modificación también valdría:

```
 8 Inicio:
 9
10  invoke GetEnvironmentVariable,addr nombre_variable,addr buffer_variable,TAMANO_BUFFER
11  test eax, eax
12  je sigue
13   invoke CompareString,NULL,NULL,addr buffer_variable,eax,addr valor_variable,1
14  dec eax
15  dec eax
16  test eax, eax
17  je sigue
18  invoke SetEnvironmentVariable,addr nombre_variable,addr valor_variable
```

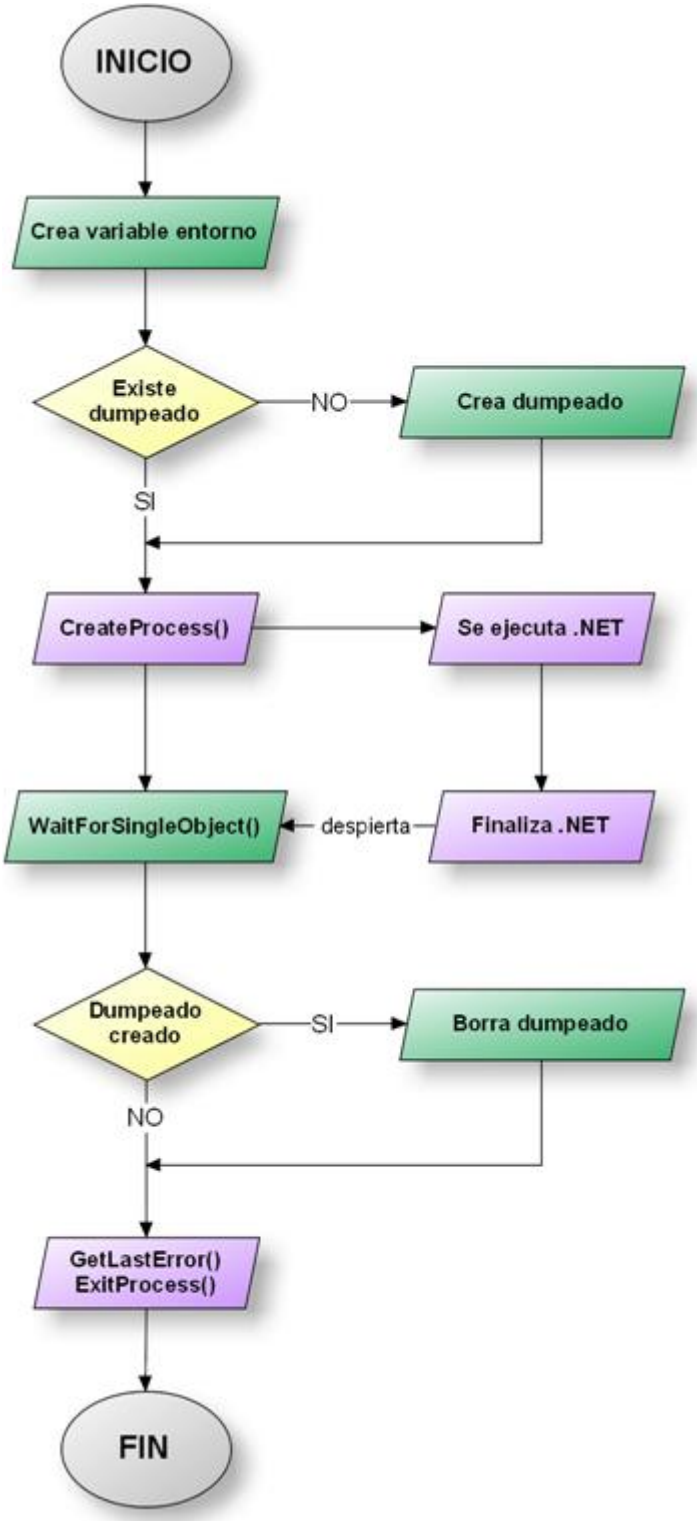
Pero como no me fio mucho optaremos por crear SIEMPRE la variable, no por ser el camino más fácil ni mucho menos, cuestión de confianza, parece más segura.

Dejamos el archivo DirectLoader.Asm tal como estaba salvo esta modificación:

```
 1 .386
 2 .model flat, stdcall
 3 option casemap :none
 4 include DirectLoader.inc
 5
 6 .code
 7
 8 Inicio:
 9
10 invoke SetEnvironmentVariable,addr nombre_variable,addr valor_variable
11
12 sigue:
13 invoke CreateFile,addr offset_nombre,GENERIC_READ,FILE_SHARE_READ,NULL,\
14 OPEN_EXISTING,FILE_ATTRIBUTE_HIDDEN,0
15 inc eax
16 test eax, eax
17 jne existe
18
19 mov existencia, FALSE
20 invoke GetModuleHandle,NULL
21 mov manejador_proceso, eax
22
23 invoke FindResource,manejador_proceso,DUMPEADO,RT_RCDATA
...
```

Se llama directamente a SetEnvironmentVariableA (sin comprobación), de esta forma nos aseguramos de crear siempre la variable.

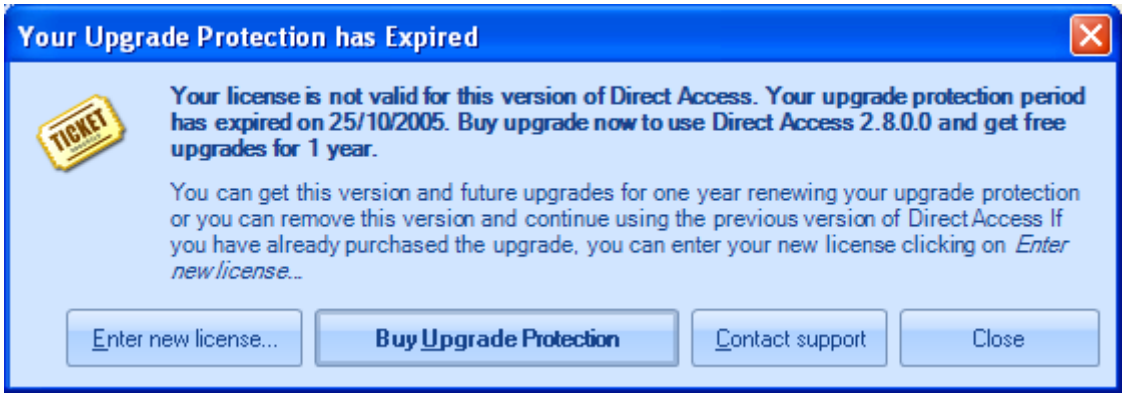
El flowchart también se ve afectado y quedará así:




Si ejecutamos el Loader ahora el programa arranca perfecto, lo hemos conseguido !

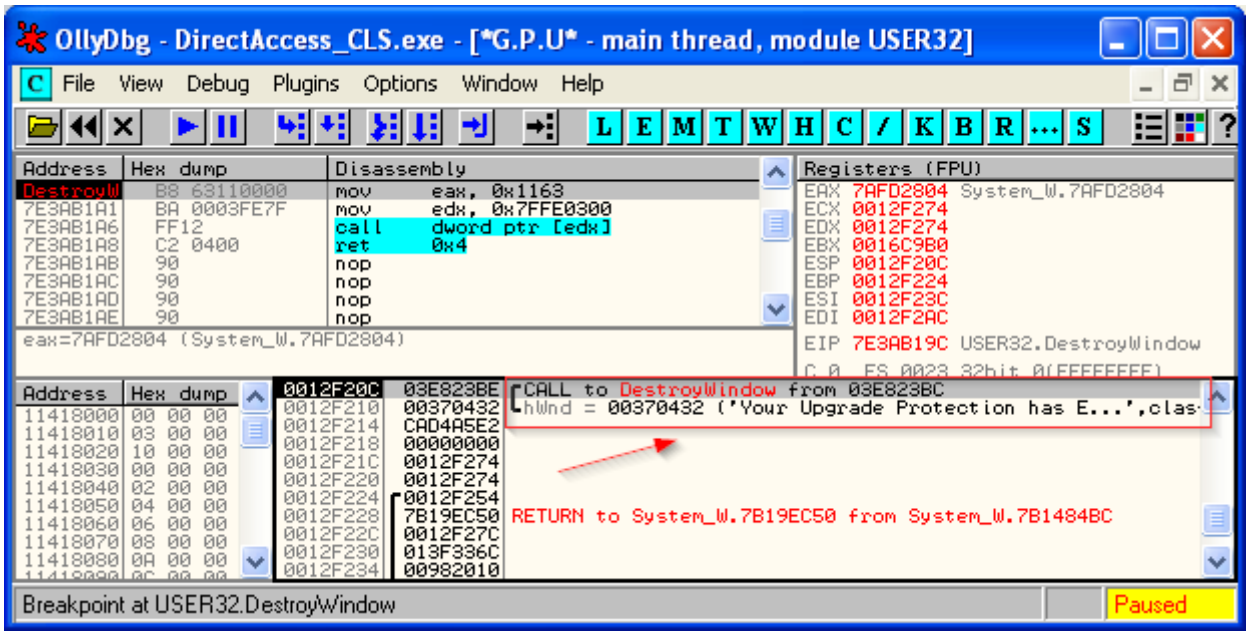
# 13. WinLicense al contraataque

Ahora que parecía que había terminado el tutorial (ya tenía escritas las Notas Finales incluso), me da por hacer una última prueba, retraso el reloj de sistema al año 2005 y zasca..

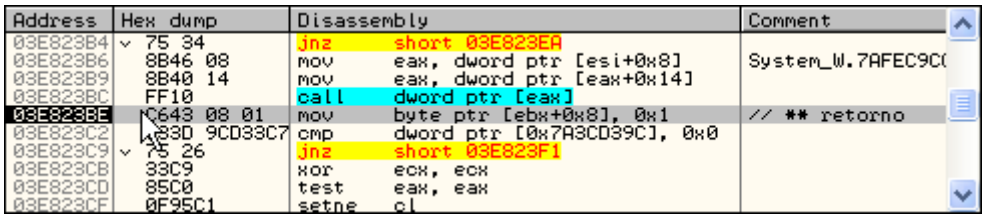



Otra vez ¿??? Parece imposible que todavía nos pille, ahora que teníamos el texto terminado :(

Esta vez nos vamos a poner un poco más serios, abrimos el dumpeado en un Olly y lo dejamos correr, cuando aparezca esta nagscreen en pantalla ponemos un BreakPoint con <F2> en la función DestroyWindow y pulsamos el botón <Close> de la nag, Olly para en el BP:




Pasamos el RET de la función y caemos aquí:



Seguimos con <F8> y vamos superando los RET que encontremos, estamos dentro de la librería System.Windows.Forms.ni.dll, responsable de mostrar las ventanas, al final llegaremos a este código:

```
Address   Hex dump   Disassembly                          Comment
00F804EF  33D2       xor     edx, edx
00F804F1  E8 FAA02F78 call   mscorlib.7927A5F0
00F804F6  8BCB       mov     ecx, ebx
00F804F8  FF15 A078E30 call  dword ptr [0x3E378A0]
00F804FE  8D4D C4    lea     ecx, dword ptr [ebp-0x3C]
00F80501  FF15 B43E980 call  dword ptr [0x983EB4]        // ** Nagscreen
00F80507  85C0       test    eax, eax
00F80509  75 26      jnz     short 00F80531
00F8050B  803D 5C3C980 cmp   byte ptr [0x983C5C], 0x0
00F80512  74 05      je      short 00F80519
```

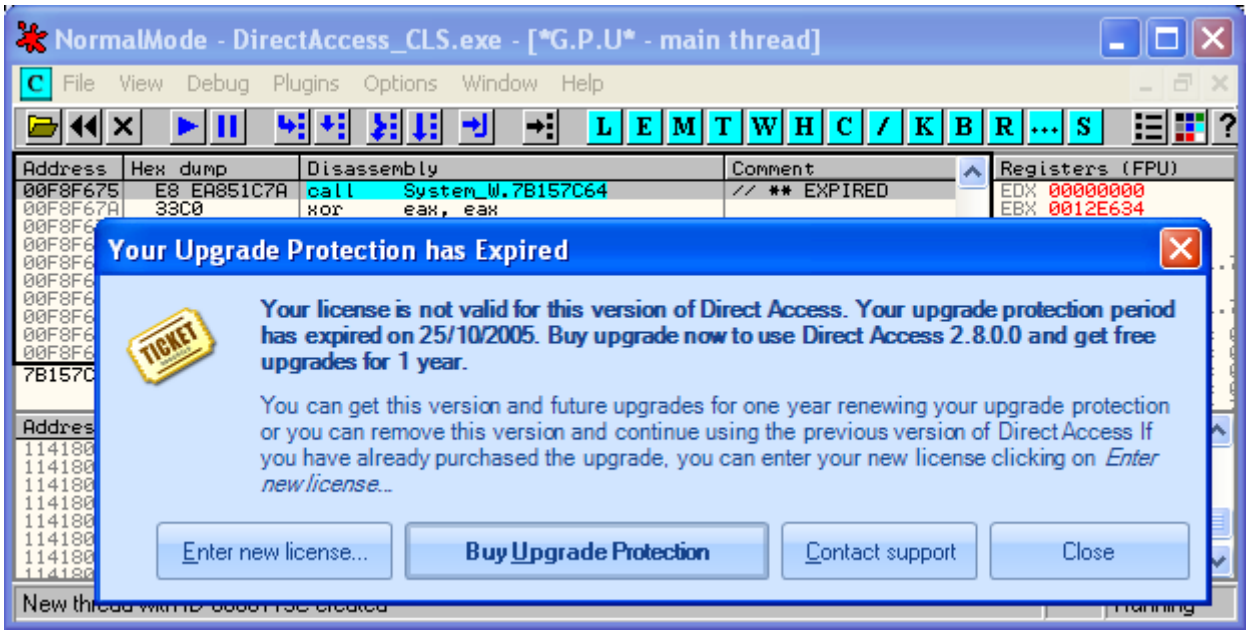
Ese call de 0xF80501h es el responsable de la ventana. Colocamos un Hardware BreakPoint sobre él y reiniciamos Olly, lo dejamos correr y paramos justo sobre este call, entramos con <F7> a ver qué vemos:

```
Address   Hex dump   Disassembly                          Comment
0098C0A5  B0 62      mov     al, 0x62
0098C0A7  EB E8      jmp     short 0098C091
0098C0A9  B0 65      mov     al, 0x65
0098C0AB  EB E4      jmp     short 0098C091
0098C0AD  0000       add     byte ptr [eax], al
0098C0AF  00E8       add     al, ch
0098C0B1  05 5D4E795E add    eax, 0x5E794E5D
0098C0B6  0000       add     byte ptr [eax], al
0098C0B8  E0 3D      loopdne short 0098C0F7
0098C0BA  98         cwde
```

Código muy raro, sigamos traceando con <F8>, tras varios JMP caemos aquí:

```
Address   Hex dump   Disassembly                          Comment
003D1EF0  8973 0C    mov     dword ptr [ebx+0xC], esi
003D1EF3  68 5DFBD61A push   0x1AD6FB5D
003D1EF8  56         push    esi
003D1EF9  E8 C6E8AA79 call   mscorwks.79E807C4
003D1EFE  897B 0C    mov     dword ptr [ebx+0xC], edi
003D1F01  8B4E 08    mov     ecx, dword ptr [esi+0x8]
003D1F04  8946 08    mov     dword ptr [esi+0x8], eax
003D1F07  8BC1       mov     eax, ecx
003D1F09  83C4 04    add     esp, 0x4
003D1F0C  5A         pop     edx                          0012F408
```

Hemos cambiado de sección, si seguimos traceando con <F8> encontramos el subcall culpable de la nag:




Lo tenemos en la dirección 0xF8F675h, esta dirección NO pertenece a nuestro Assembly, quizá esté cargando algún Módulo en memoria, esto todavía no lo sabemos.

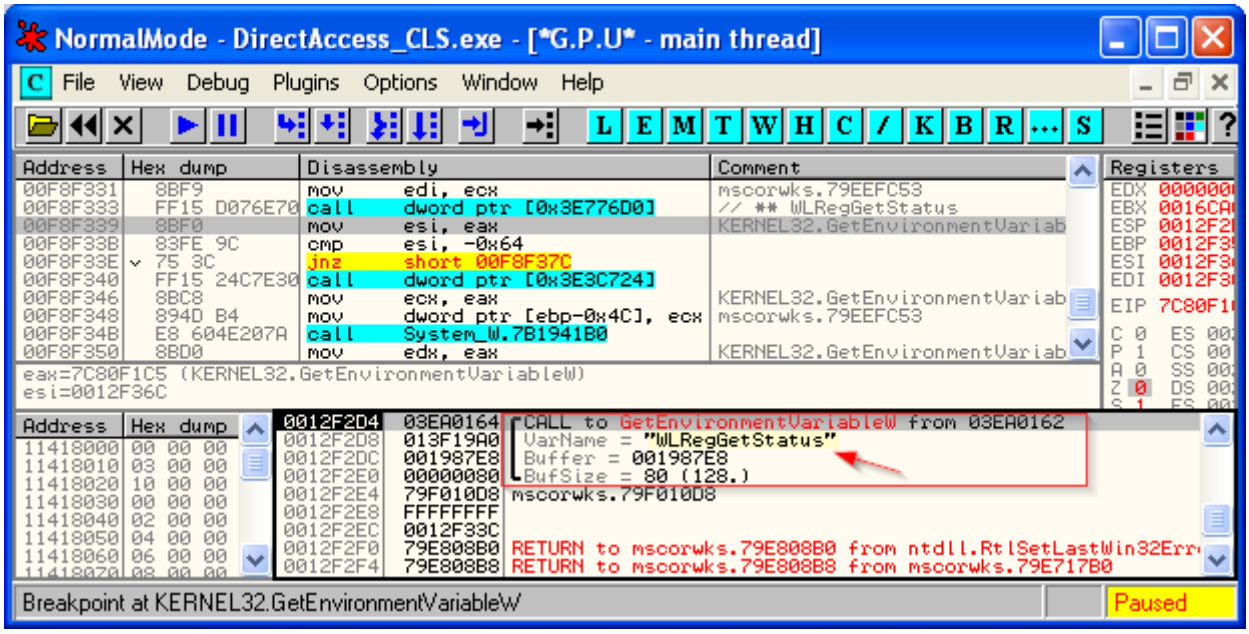
Reiniciamos Olly y comenzamos de nuevo, ya hemos identificado el call de la nag, a ver qué más vemos (si miramos la ventana de hilos (*threads*) veremos que hay bastantes:

| Ident | Entry | Data block | Last error | Status | Priority | User time | System time |
|---|---|---|---|---|---|---|---|
| 00000898 | 11416F34 | 7FFDE000 | ERROR_SUCCESS (00000000) | Active | 32 + 0 | 0.9218 s | 0.2343 s |
| 00001014 | 7C810729 | 7FFDB000 | ERROR_SUCCESS (00000000) | Paused | 32 + 0 | 0.0000 s | 0.0000 s |
| 00001024 | 7C810729 | 7FFD9000 | ERROR_SUCCESS (00000000) | Paused | 32 + 0 | 0.0000 s | 0.0000 s |
| 00001124 | 7C810729 | 7FFD6000 | ERROR_SUCCESS (00000000) | Paused | 32 + 0 | 0.0000 s | 0.0000 s |
| 0000113C | 7C810729 | 7FFD4000 | ERROR_SUCCESS (00000000) | Active | 32 + 0 | 0.0156 s | 0.0000 s |
| 00001310 | 7C810729 | 7FFDC000 | ERROR_SUCCESS (00000000) | Paused | 32 + 2 | 0.0000 s | 0.0000 s |
| 00001598 | 7C810729 | 7FFD7000 | ERROR_SUCCESS (00000000) | Paused | 32 + 0 | 0.0000 s | 0.0000 s |
| 00001690 | 7C810729 | 7FFDA000 | ERROR_NO_TOKEN (000003F0) | Paused | 32 + 0 | 0.0000 s | 0.0000 s |
| 000016FC | 7C810729 | 7FFDD000 | ERROR_SUCCESS (00000000) | Paused | 32 + 0 | 0.0000 s | 0.0000 s |

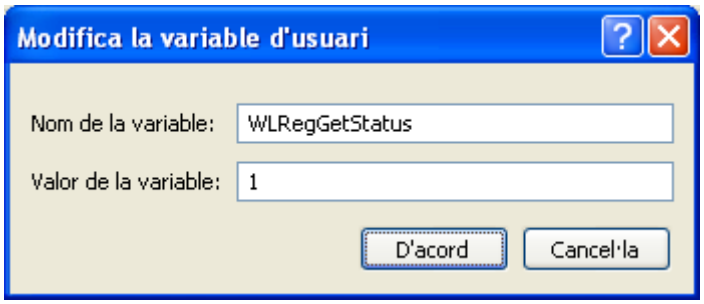
Traceamos la subrutina desde el *push ebp* inicial:

| Address | Hex dump | Disassembly | | Comment |
|---|---|---|---|---|
| 00F8F318 | 55 | push | ebp | // ** Inicio subrutina |
| 00F8F319 | 8BEC | mov | ebp, esp | |
| 00F8F31B | 57 | push | edi | |
| 00F8F31C | 56 | push | esi | |
| 00F8F31D | 53 | push | ebx | |
| 00F8F31E | 83EC 40 | sub | esp, 0x40 | |
| 00F8F321 | 8BF1 | mov | esi, ecx | |
| 00F8F323 | 8D7D B4 | lea | edi, dword ptr [ebp-0x4C] | |
| 00F8F326 | B9 10000000 | mov | ecx, 0x10 | |
| 00F8F32B | 33C0 | xor | eax, eax | |

No hay que tracear mucho para encontrar algo:

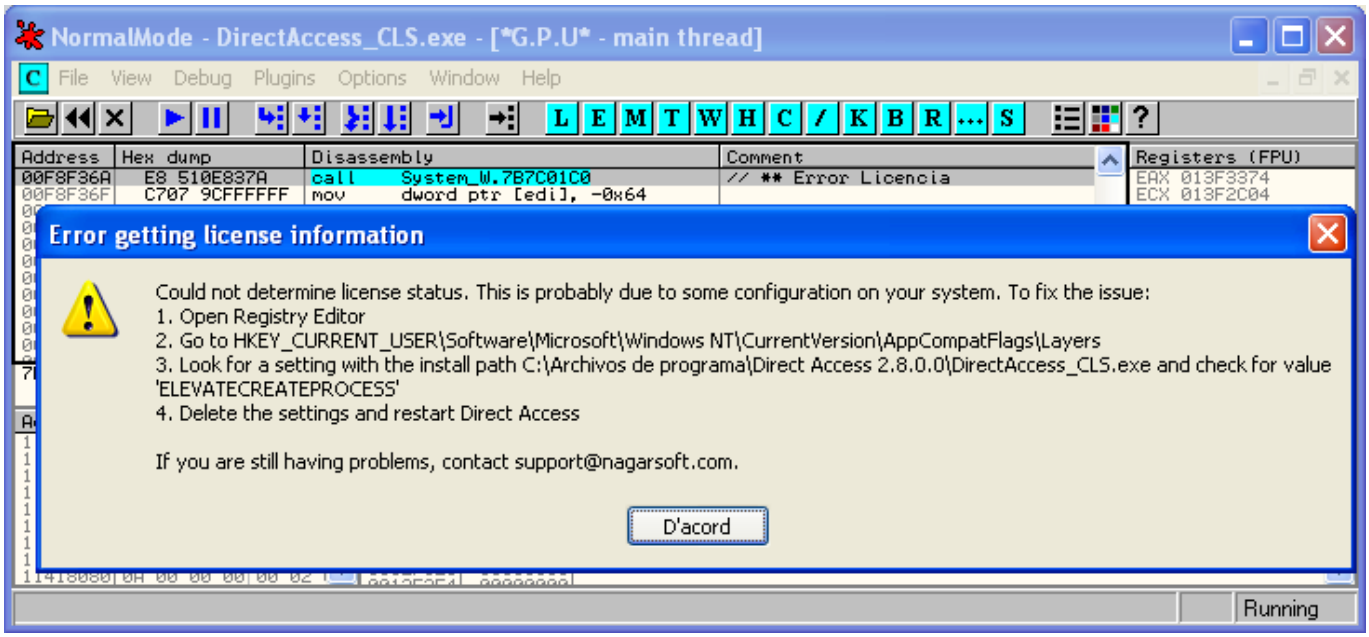



Ese call de 0xF8F339h hace una primera llamada a GetEnvironmentVariableW para consultar el valor de la variable WLRegGetStatus (si existe), por tanto la creamos temporalmente a mano como hicimos antes:

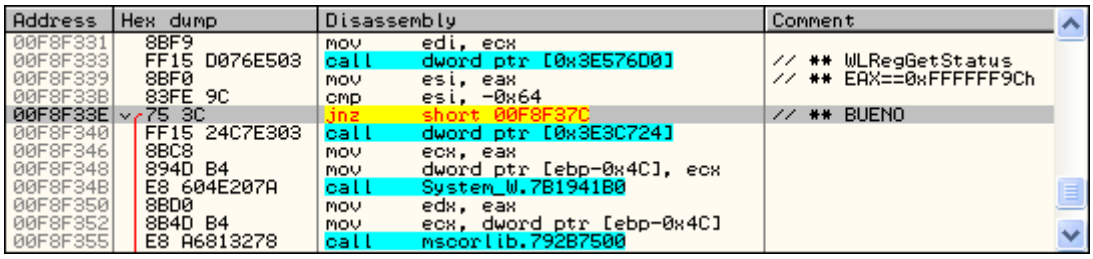



Esto por ahora nos mantendrá a salvo.

Tened en cuenta que estamos ejecutando el dumpeado SIN el Loader, lo que significa que esta variable de entorno NO existe (la crea el Loader precisamente). Por tanto la dejamos creada, esto nos evita este cartel:

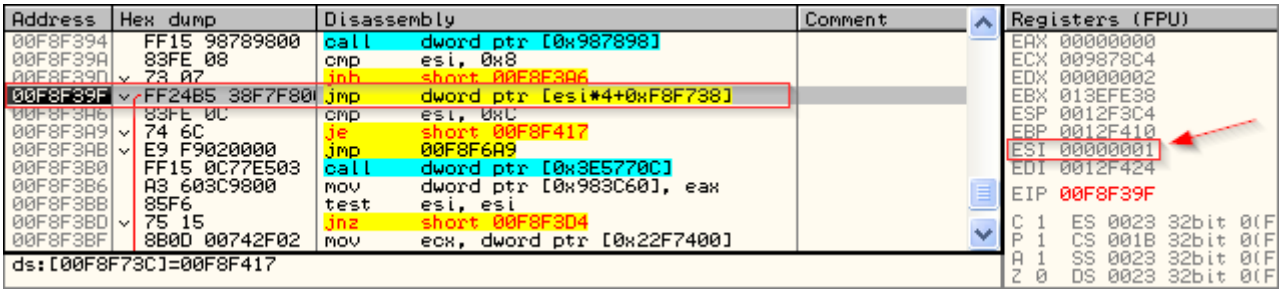



El call previo es el responsable, por tanto, ya sabemos seguro que este salto SI se debe ejecutar:




Se ejecutará SI encuentra la variable WLRegGetStatus.

Seguimos traceando (con la variable creada), llegamos a este salto indirecto:




En función del valor de ESI irá a una zona u otra, estudiando la rutina vemos que este valor no es ni más ni menos que el que tenga la variable de entorno WLRegGetStatus, lo que significa que si esta variable tiene valor NO INFERIOR a 0x08h llegamos al salto indirecto, el cual parece a una sentencia *Switch Case (var)*, al traducirse a ASM suelen quedar este tipo de saltos donde ESI sería el valor de la variable var.

Una cosa que yo hago muchas veces es modificar el valor del registro (ESI en este caso) para ver los posibles casos para diferentes valores.

Los posibles casos son éstos:

```
Address   Hex dump     Disassembly                          Comment
00F8F394  FF15 98789800 call    dword ptr [0x987898]
00F8F39A  83FE 08      cmp     esi, 0x8                     // ** WLRegGetStatus
00F8F39D  73 07        jnb     short 00F8F3A6
00F8F39F  FF24B5 38F7F80 jmp   dword ptr [esi*4+0xF8F738]
00F8F3A6  83FE 0C      cmp     esi, 0xC                     // ** ESI > 0x08h
00F8F3A9  74 6C        je      short 00F8F417
00F8F3AB  E9 F9020000  jmp     00F8F6A9
00F8F3B0  FF15 0C77E503 call   dword ptr [0x3E5770C]        // ** ESI==0x00h, 0x02h
00F8F3B6  A3 603C9800  mov     dword ptr [0x983C60], eax
00F8F3BB  85F6         test    esi, esi
00F8F3BD  75 15        jnz     short 00F8F3D4
00F8F3BF  8B0D 00742F02 mov    ecx, dword ptr [0x22F7400]
```

Para ESI==0x00h y ESI==0x02h llegará a 0xF8F3B0h.

Veamos el siguiente:

```
Address   Hex dump     Disassembly                          Comment
00F8F417  8D45 EC      lea     eax, dword ptr [ebp-0x14]    // ** ESI==0x01h, 0x0Ch
00F8F41A  50           push    eax
00F8F41B  8D45 E0      lea     eax, dword ptr [ebp-0x20]
00F8F41E  50           push    eax
00F8F41F  8D45 E8      lea     eax, dword ptr [ebp-0x18]
00F8F422  50           push    eax
00F8F423  8D45 E4      lea     eax, dword ptr [ebp-0x1C]
00F8F426  50           push    eax
00F8F427  8D45 D8      lea     eax, dword ptr [ebp-0x28]
00F8F42A  50           push    eax
00F8F42B  8D45 D0      lea     eax, dword ptr [ebp-0x30]
00F8F42E  50           push    eax
```

Aquí se llega si ESI==0x01h o ESI==0x0Ch.

Hay otros como este:

```
Address   Hex dump     Disassembly                          Comment
00F8F6A9  837D D4 00   cmp     dword ptr [ebp-0x2C], 0x0    // ** ESI==0x03h, 0x04h, 0x06h
00F8F6AD  74 51        je      short 00F8F700               // ** ESI > 0x08h
00F8F6AF  85F6         test    esi, esi
00F8F6B1  75 15        jnz     short 00F8F6C8
00F8F6B3  8B0D 00742F02 mov    ecx, dword ptr [0x22F7400]
00F8F6B9  8B15 44A62F02 mov    edx, dword ptr [0x22FA644]
00F8F6BF  3909         cmp     dword ptr [ecx], ecx
00F8F6C1  E8 E22AFFFF  call    00F821A8
00F8F6C6  EB 15        jmp     short 00F8F6DD
00F8F6C8  56           push    esi
00F8F6C9  8B0D 00742F02 mov    ecx, dword ptr [0x22F7400]
00F8F6CF  8B15 48A62F02 mov    edx, dword ptr [0x22FA648]
```

Donde se llegará para ESI==0x03h, ESI==0x04h, ESI==0x06h y cuando ESI>0x08h (cuatro casos).

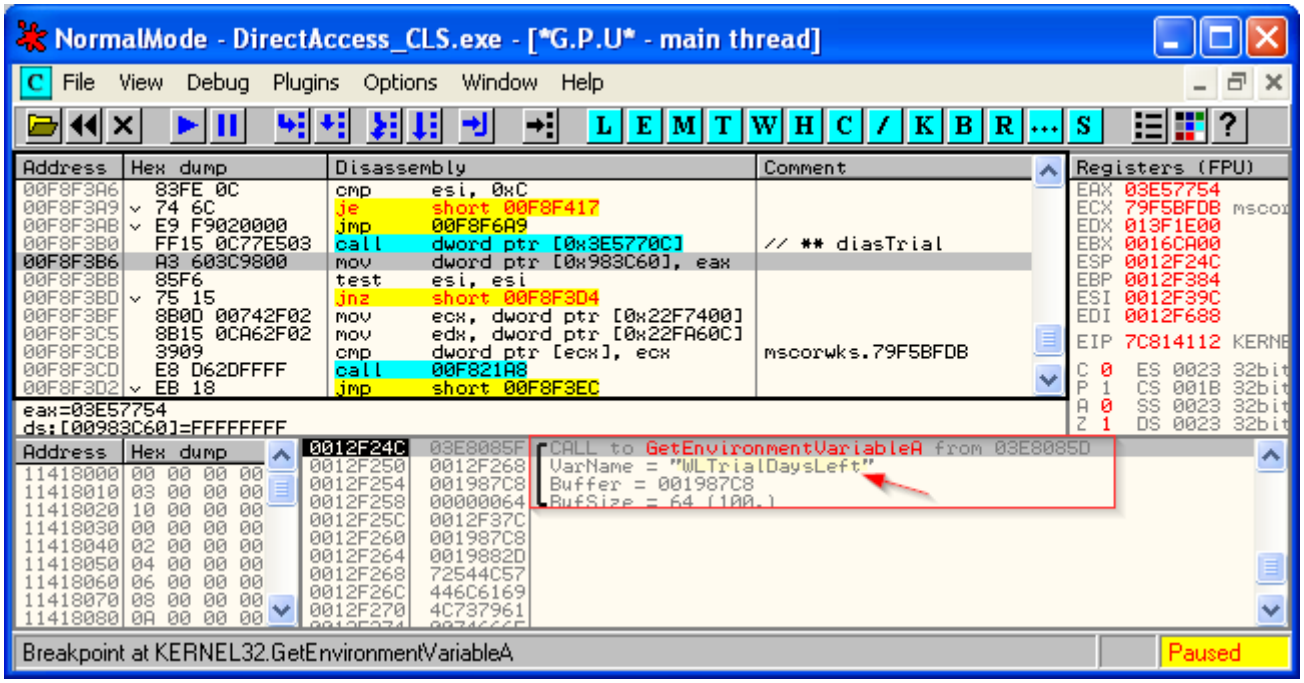
Por último estos otros:

```
Address   Hex dump     Disassembly                          Comment
00F8F681  8B0D 00742F02 mov    ecx, dword ptr [0x22F7400]   // ** ESI==0x05h
00F8F687  8B15 3CA62F02 mov    edx, dword ptr [0x22FA63C]
00F8F68D  3909         cmp     dword ptr [ecx], ecx
00F8F68F  E8 142BFFFF  call    00F821A8
00F8F694  EB 13        jmp     short 00F8F6A9
00F8F696  8B0D 00742F02 mov    ecx, dword ptr [0x22F7400]   // ** ESI==0x07h
00F8F69C  8B15 40A62F02 mov    edx, dword ptr [0x22FA640]
00F8F6A2  3909         cmp     dword ptr [ecx], ecx
00F8F6A4  E8 FF2AFFFF  call    00F821A8
00F8F6A9  837D D4 00   cmp     dword ptr [ebp-0x2C], 0x0    // ** ESI==0x03h, 0x04h, 0x06h
00F8F6AD  74 51        je      short 00F8F700               // ** ESI > 0x08h
00F8F6AF  85F6         test    esi, esi
```

En el primer punto 0xF8F681h se llegará cuando ESI==0x05h, y al segundo 0xF8F696h cuando ESI==0x07h.

Lo que haremos será explorar TODOS los posibles casos en busca de información. De esta manera podremos después desde nuestro Loader (si se da el caso) establecer el que más nos convenga.

Cambiamos el valor a ESI==0x00h y pasamos a ejecutar su código (*case*) asociado:



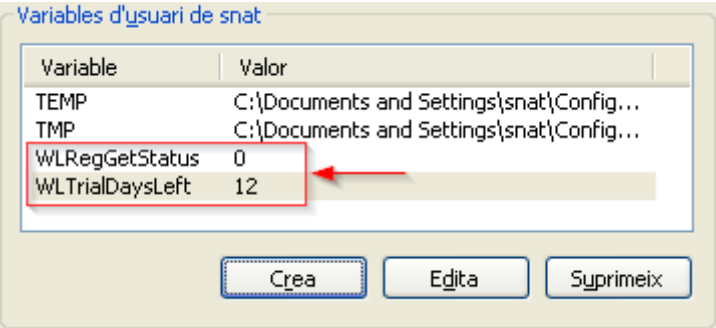

Se va a leer la variable de entorno WLTrialDaysLeft, esto significa que si dejamos estas variables:

- ✓ WLRegGetStatus: valor 0x00h
- ✓ WLTrialDaysLeft: valor > 0x00h

La cosa mejorará (o al menos debería).

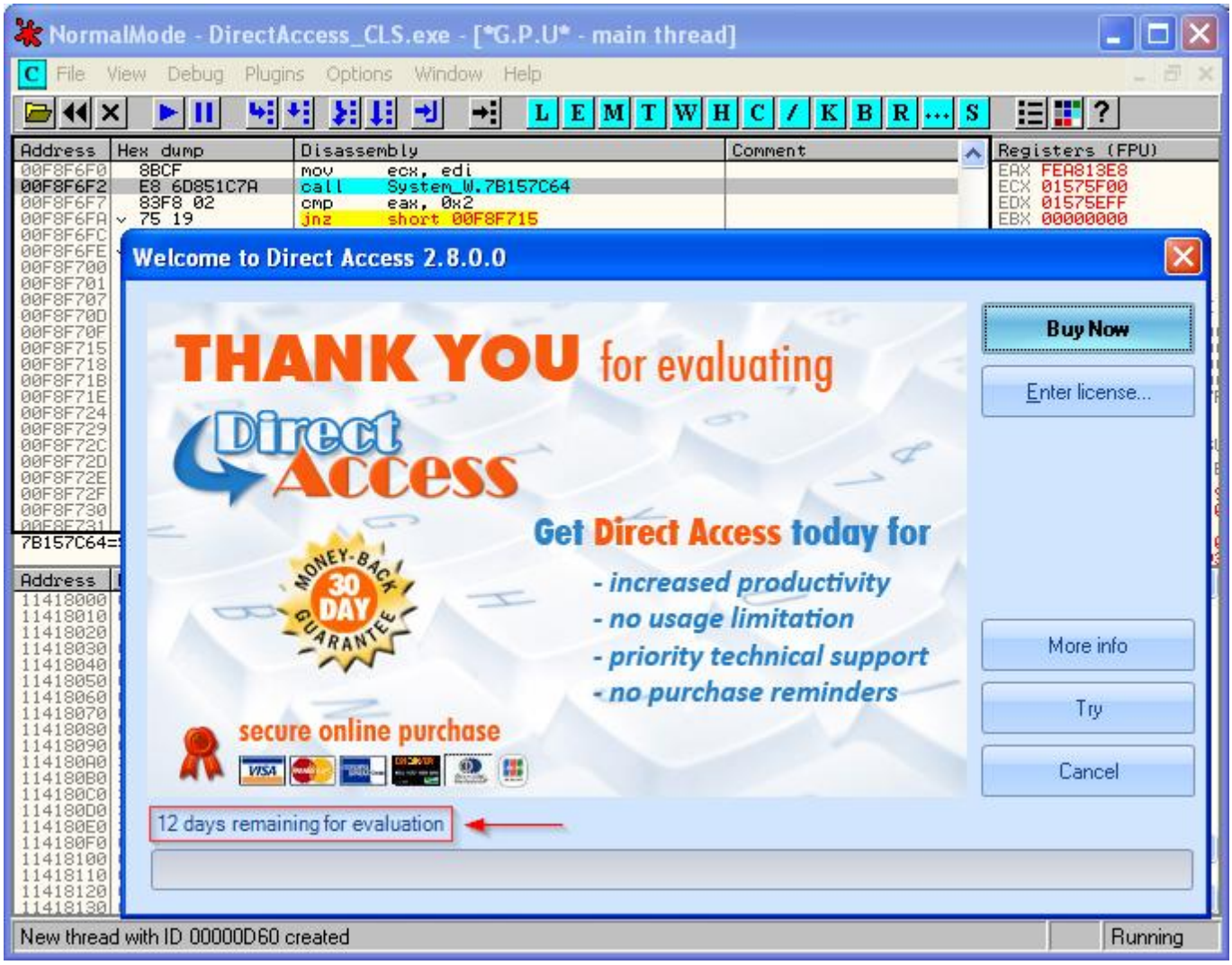
Para salir de dudas creamos la segunda variable WLTrialDaysLeft y le asignamos un valor mayor que cero, por ejemplo 12d. La primera variable WLRegGetStatus ya existe (con valor 0x01h) la modificamos para que tenga valor 0x00h o 0x02h (ambos salen por la misma cláusula case del switch), en el código es la dirección 0xF8F3B0h que hemos visto hace un momento.

Dejamos ambas variables así:

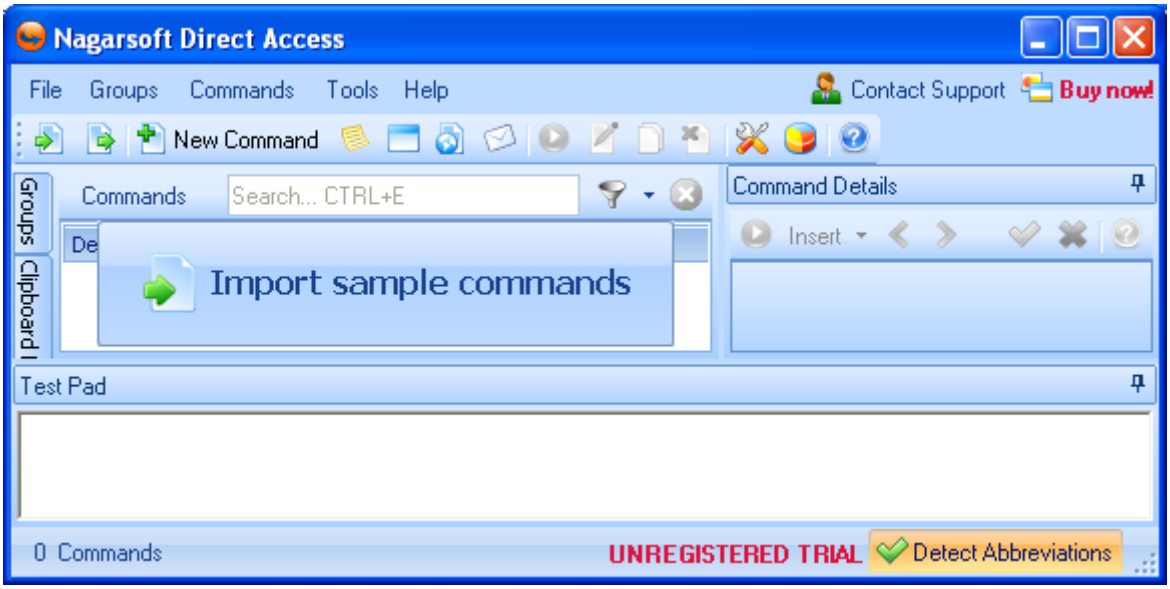



Cerramos Olly (importante cerrarlo para que tenga efecto el cambio), y lo volvemos a abrir de nuevo, lo dejamos correr libremente con <F9>, irán parando los HBPs que tenemos en la rutina que estamos estudiando, seguimos con <F9> y a los pocos instantes aparece la nag inicial del programa y que antes estaba EXPIRED.

Este es su nuevo aspecto:



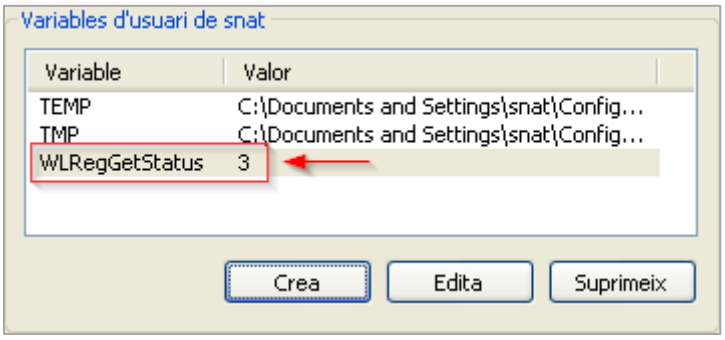

Si pulsamos el botón Try el programa arranca :)



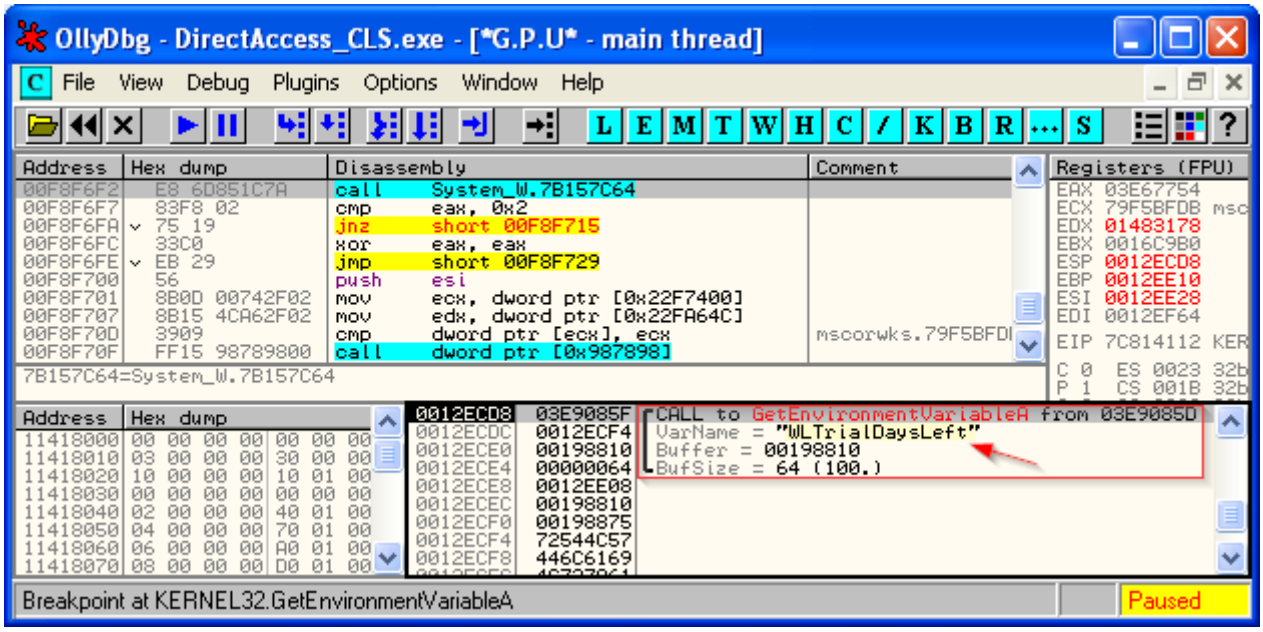

Vamos mejorando y mucho.

Podríamos dejarlo así, modificar nuestro proyecto de RadASM para que cree la segunda variable y listos, pero ya no me fío, esta es la segunda vez que doy por finalizado el trabajo y espero que la última.

Así que profundizaremos todavía más en la rutina. Modifiquemos las variables de entorno:




A ver qué se cuece para este valor (hemos eliminado WLTrialDaysLeft), Cerramos Olly y lo volvemos a abrir para trabajar con la nueva variable, al llegar al salto indirecto ahora toma otro camino (0xF8F6A9h), ponemos BPs comunes en GetEnvironmentVariableA/W a ver si busca algo:

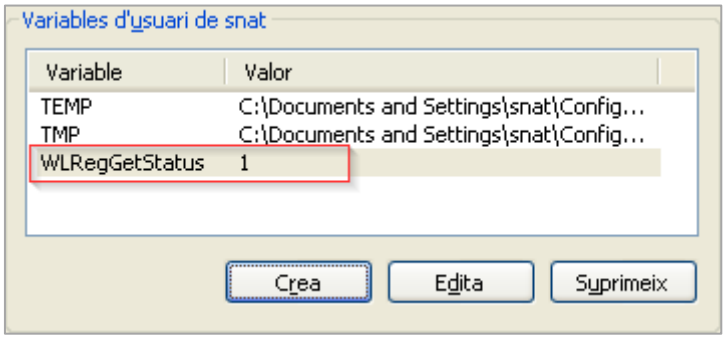



Parece la misma captura de antes pero es otra. Así que más de lo mismo, modifiquemos las variables de entorno otra vez, ahora para ESI==0x05h llegamos al mismo punto, para ESI==0x07h la licencia es inválida:

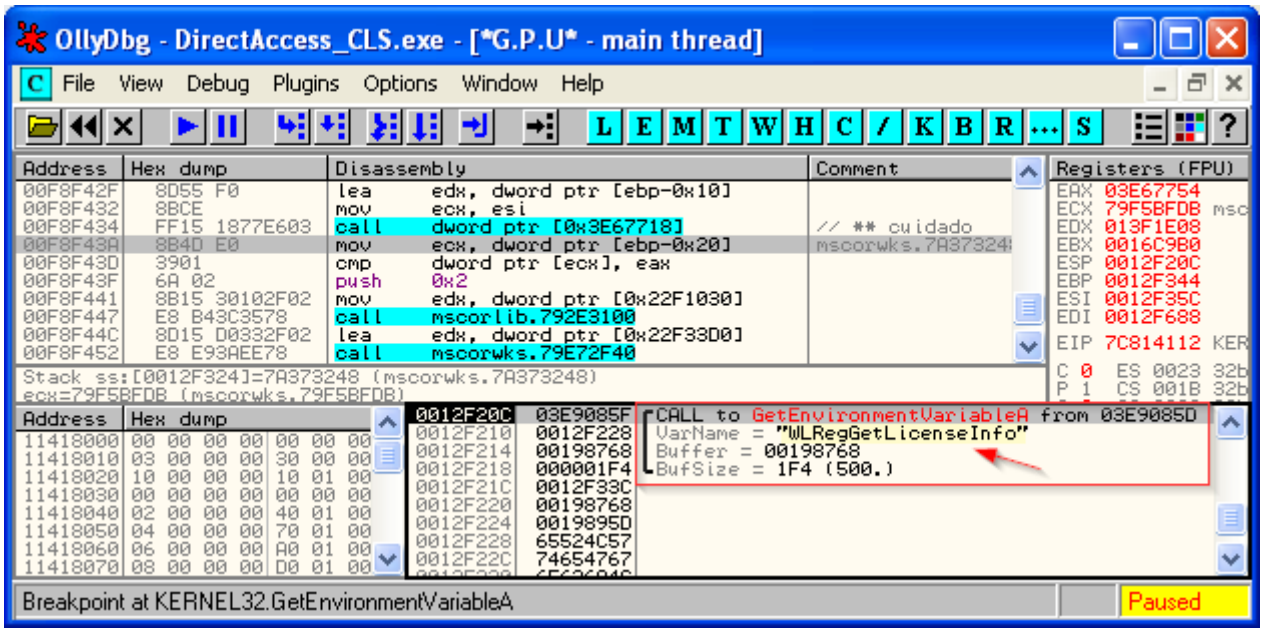
Esta captura es de la nag inicial (la parte baja), el botón Try por su puesto está deshabilitado, y al cerrar la ventana Olly queda Terminated!. El proceso terminó.

Nos quedan dos opciones todavía (que llevan a la misma zona), para ESI==0x01h (que ya teníamos al principio) y cuando ESI > 0x08h (que sale por la cláusula default del switch-case) puesto que para cualquier valor superior a ocho llega al mismo punto.

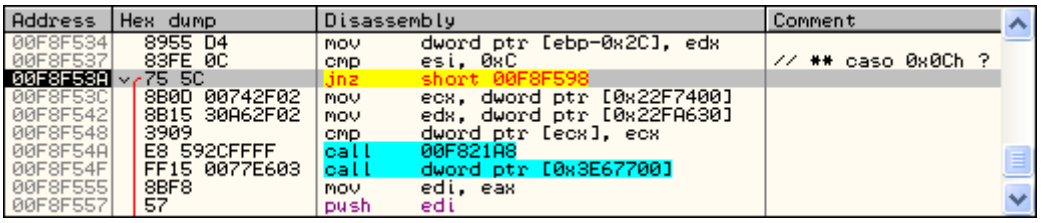
Nos quedamos como estábamos:



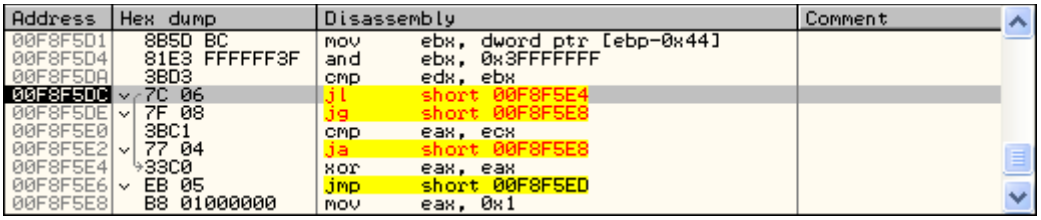

Esto nos lleva a la zona de código que comienza en 0xF8F417, seguimos traceando desde allí con los BPs puestos en GetEnvironmentVariableA/W a ver si busca algo:




Vaya, busca la variable WLRegGetLicenseInfo, de momento seguimos traceando y llegamos aquí:




Está mirando si ESI==0x0Ch (lo que significa WLRegGetStatus==0x0Ch), y esto nos lleva al call de la dirección 0xF8F675h donde nos saca la nag EXPIRED, pero si forzamos este salto:



Basta con invertir el flag de desbordamiento (OV, *overflow*) como en la imagen:

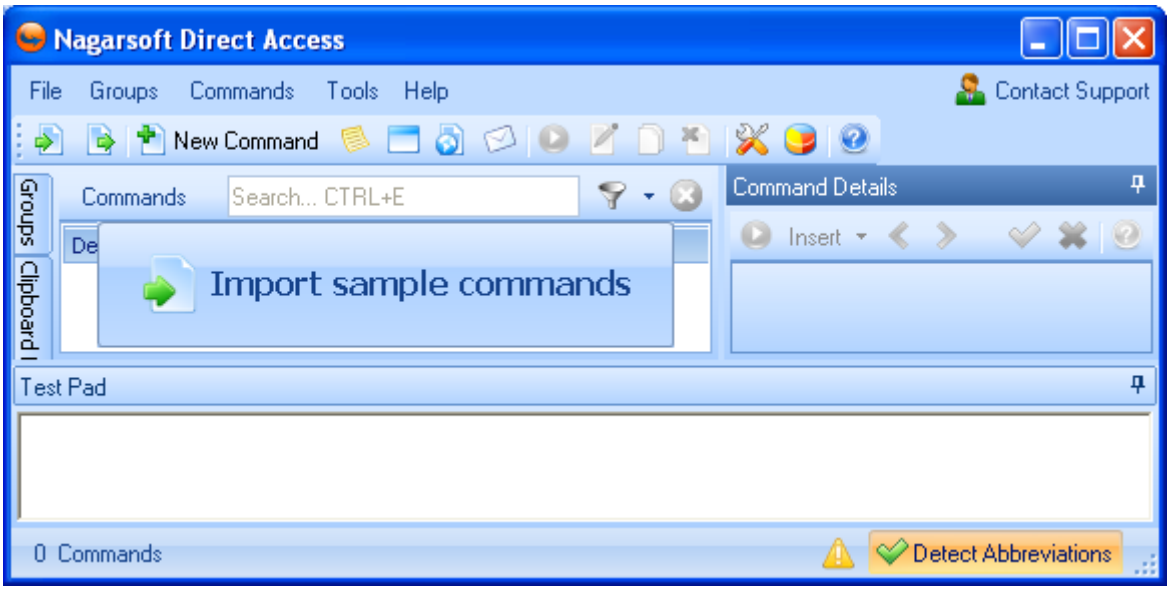
```
Address   Hex dump    Disassembly                          Comment
00F8F5D1  8B5D BC     mov     ebx, dword ptr [ebp-0x44]
00F8F5D4  81E3 FFFFFF3F and   ebx, 0x3FFFFFFF
00F8F5DA  3BD3        cmp     edx, ebx
00F8F5DC  7C 06       jl      short 00F8F5E4
00F8F5DE  7F 08       jg      short 00F8F5E8
00F8F5E0  3BC1        cmp     eax, ecx
00F8F5E2  77 04       ja      short 00F8F5E8
00F8F5E4  33C0        xor     eax, eax
00F8F5E6  EB 05       jmp     short 00F8F5ED
00F8F5E8  B8 01000000 mov     eax, 0x1
Jump is taken

Registers (FPU)
ESI 0000000C
EDI 00000000
EIP 00F8F5DC
C 0  ES 0023 32b
P 1  CS 001B 32b
A 1  SS 0023 32b
Z 0  DS 0023 32b
S 0  FS 003B 32b
T 0  GS 0000 NUL
D 0
O 1  LastErr ERR
```

El salto SE ejecuta y pasamos por encima de todos los subcall que muestran nags, comprueban variables de entorno y demás, llegando al final del mismo y activando el flag de éxito:

```
Address   Hex dump    Disassembly                          Comment
00F8F718  0F94C1      sete    cl
00F8F71B  0FB6C9      movzx   ecx, cl
00F8F71E  FF15 A83E9800 call  dword ptr [0x983EA8]
00F8F724  B8 01000000 mov     eax, 0x1                     // ** Ok!
00F8F729  8D65 F4     lea     esp, dword ptr [ebp-0xC]
00F8F72C  5B          pop     ebx
00F8F72D  5E          pop     esi
00F8F72E  5F          pop     edi
00F8F72F  5D          pop     ebp
00F8F730  C3          ret
```

Si lo dejamos correr con <F9> el programa arranca:




Conclusión:

1. Necesita la variable WLRegGetStatus con valor 0x01h o valor 0x0Ch.
2. Necesita la variable WLRegGetLicenseInfo con un valor que todavía no sabemos.

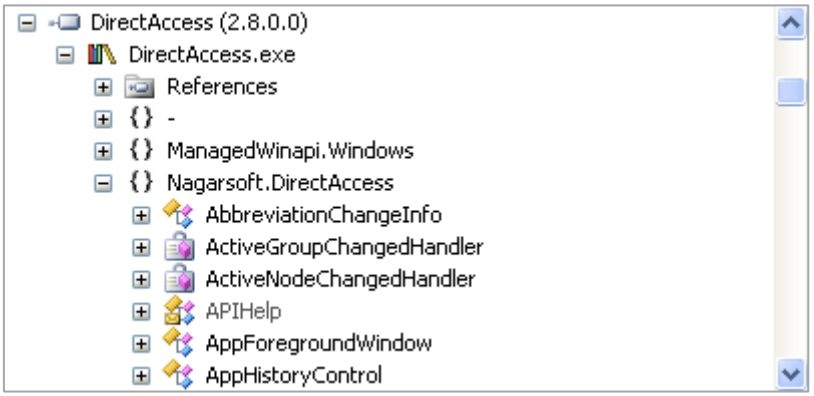
Si lo creamos con uno cualquiera continúa apareciendo la nag de EXPIRED.

No nos queda otra que sumergirnos en el Assembly.

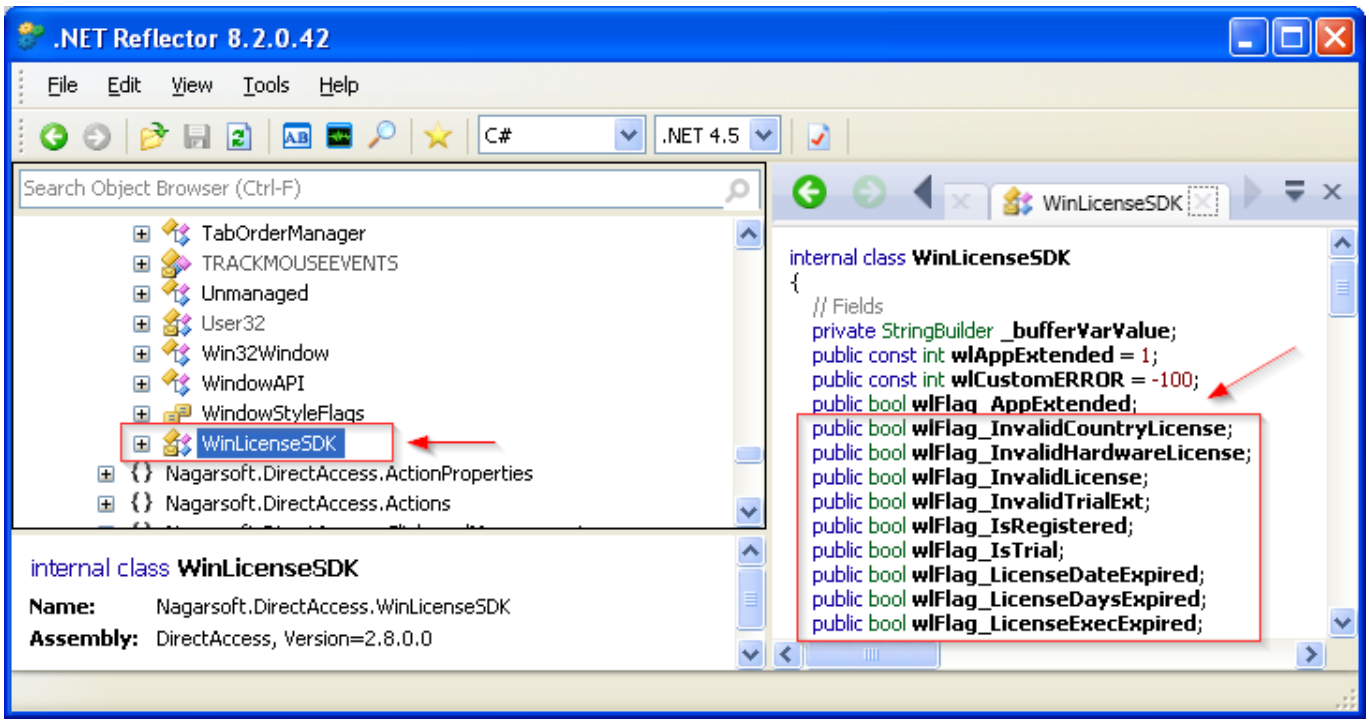
# 14. Último asalto

Pues sí, esto esperamos. Vamos a utilizar .NET Reflector para la tarea, no tenía pensado utilizarlo así que lo agregaré en la sección 3. Material necesario.

Lo abrimos y cargamos el dumpeado DirectAccess_CLS.exe, hecho esto vamos a la rama con nombre Nagarsoft.DirectAccess y la expandimos:

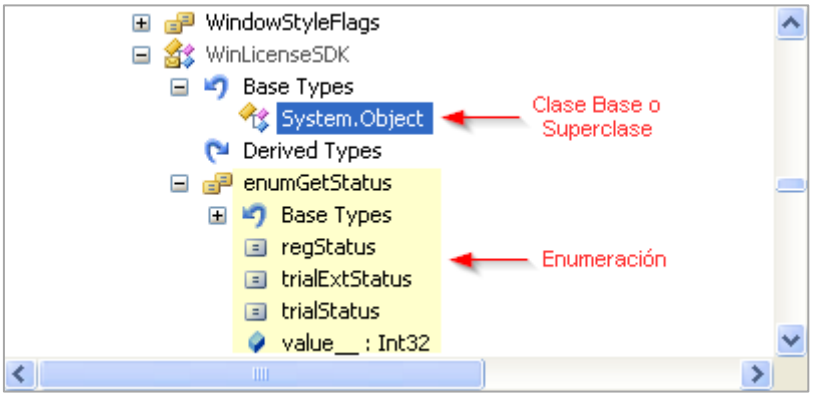



Vamos bajando hasta encontrar al WinLicense:



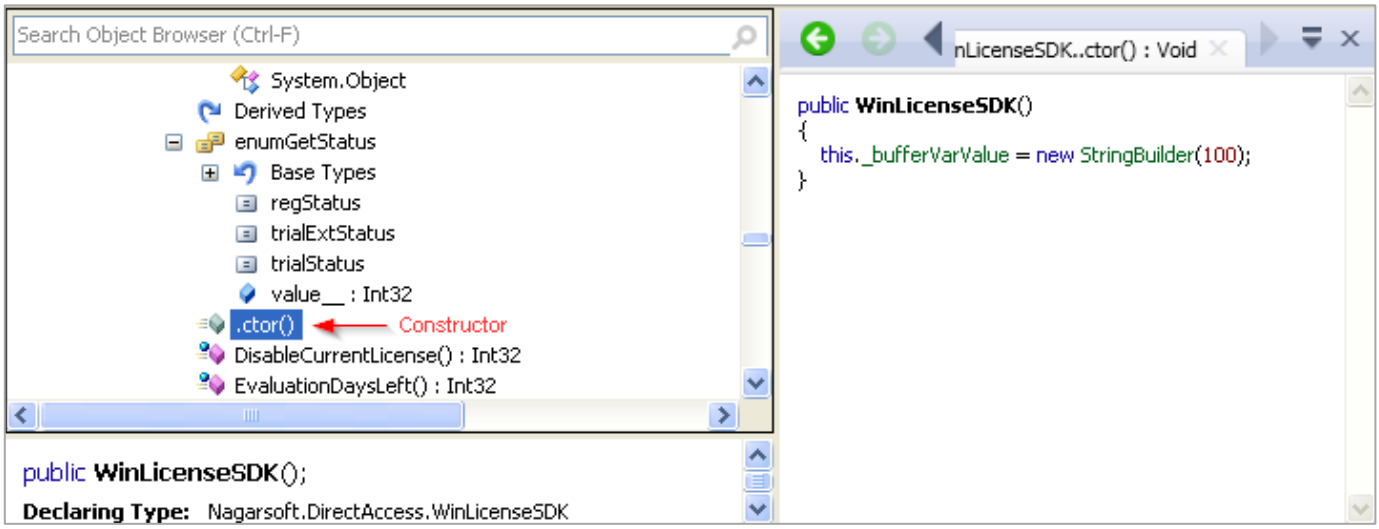

Allí está, bajo el nombre WinLicenseSDK, en la parte derecha encontramos muchísima información acerca de Licencias, incluso se ven variables booleanas con nombres como wlFlag_IsRegistered, wlFlag_IsTrial, etc., parece que vamos bien :)

Expandimos el subárbol WinLicenseSDK para ver qué información más podemos encontrar:




Se ve una referencia a su superclase o clase base System.Object, que a su vez, es la clase Base de TODAS las clases, es decir, si una clase no deriva explícitamente de ninguna otra, implícitamente derivará de System.Object (igual que en java para java.lang.Object).

Después vemos una enumeración con tres Objetos (las enumeraciones también lo son, al menos en java) con valores tipo entero. Justo después, aunque en la imagen no se aprecia aparece un constructor de Clase con identificador .ctor(), éstos se utilizan para inicializar los campos del Objeto creado mediante el operador new, newobj en función del lenguaje:




Su única misión es inicializar el campo _bufferVarValue mediante la creación de un objeto tipo StringBuilder de tamaño 100 chars (la variable toma cuerpo a partir de aquí, ya no es NULL).

Si echamos un vistazo hacia abajo comienzan a aparecer métodos como este:

```
public static int DisableCurrentLicense()
{
   StringBuilder lpBuffer = new StringBuilder(100);
   GetEnvironmentVariable("WLRegDisableKeyCurrentInstance", lpBuffer, 100);
   int num = Convert.ToInt32(lpBuffer.ToString());
   Logger.Main.LogMessage("WLRegDisableKeyCurrentInstance: " + lpBuffer);
   if (num == 1)
   {
      Logger.Main.LogMessage("Current Key Disabled for the instance");
   }
   return num;
}
```

El código fuente del mismo, es un método de clase (lleva el modificador *static* delante), retorna un valor tipo entero (de ahí el *int*), lo que hace es lo siguiente: crea un objeto tipo StringBuilder y lo asigna a la variable lpBuffer de este mismo tipo, hace una llamada a GetEnvironmentVariable a la que le pasa como parámetro el nombre de la variable a buscar WLRegDisableKeyCurrentInstance, si la encuentra escribirá su contenido en la variable lpBuffer con un máximo de 100 caracteres.

Pero me llama muchísimo la atención este otro método GetRegistrationInfo:

```
public static void GetRegistrationInfo(int status, out string name, out string company, out string serialnumber,
{
    name = string.Empty;
    company = string.Empty;
    email = string.Empty;
    numlic = string.Empty;
    serialnumber = string.Empty;
    subscribeExpireDate = DateTime.Today;
    powerLicensePack = false;
    string str = string.Empty;
    if ((status == 1) || (status == 12))
    {
        StringBuilder lpBuffer = new StringBuilder(500);
        GetEnvironmentVariable("WLRegGetLicenseInfo", lpBuffer, 500);
        Logger.Main.LogString("WLRegGetLicenseInfo", lpBuffer.ToString());
        string[] strArray = lpBuffer.ToString().Split(new char[] { ',' });
        string s = string.Empty;
        if (strArray.Length >= 3)
        {
            name = strArray[0];
            company = strArray[1];
            string[] strArray2 = strArray[2].Split(new char[] { ';' });
            if (strArray2.Length == 4)
            {
                serialnumber = strArray2[0];
                email = strArray2[1];
                numlic = strArray2[2];
                s = strArray2[3];
            }
            else if (strArray2.Length == 5)
            {
                serialnumber = strArray2[0];
                email = strArray2[1];
                numlic = strArray2[2];
                s = strArray2[3];
                str = strArray2[4];
                if (str == "PowerLicense")
                {
                    powerLicensePack = true;
                }
            }
            CultureInfo invariantCulture = CultureInfo.InvariantCulture;
            try
            {
                subscribeExpireDate = DateTime.ParseExact(s, "yyyyMMdd", invariantCulture);
            }
            catch (FormatException exception)
            {
                Logger.Main.LogException("Error parsing datetime", exception);
                subscribeExpireDate = new DateTime();
            }
        }
    }
}
```

Además hace referencia a la variable de entorno que nos trajo hasta aquí WLRegGetLicenseInfo, parece más que interesante ciertamente.

Veámosla de cerca desglosando lo que hace:

```
if ((status == 1) || (status == 12))
{
    // ** hacemos algo
}
```

He eliminado el código por claridad. Aquí comienza valorando el valor que tiene la variable status, no casualmente hace referencia al valor 0x01h y 0x0Ch (12d), recordemos esta imagen:

| Address | Hex dump | Disassembly | | Comment |
|---|---|---|---|---|
| 00F8F417 | 8D45 EC | lea | eax, dword ptr [ebp-0x14] | // ** ESI==0x01h, 0x0Ch |
| 00F8F41A | 50 | push | eax | |
| 00F8F41B | 8D45 E0 | lea | eax, dword ptr [ebp-0x20] | |
| 00F8F41E | 50 | push | eax | |
| 00F8F41F | 8D45 E8 | lea | eax, dword ptr [ebp-0x18] | |
| 00F8F422 | 50 | push | eax | |
| 00F8F423 | 8D45 E4 | lea | eax, dword ptr [ebp-0x1C] | |
| 00F8F426 | 50 | push | eax | |
| 00F8F427 | 8D45 D8 | lea | eax, dword ptr [ebp-0x28] | |
| 00F8F42A | 50 | push | eax | |
| 00F8F42B | 8D45 D0 | lea | eax, dword ptr [ebp-0x30] | |
| 00F8F42E | 50 | push | eax | |

Es la rutina que acabamos de ver en la sección 13. WinLicense al contraataque, justo para los casos de ESI==0x01h y ESI==0x0Ch. Representa el valor de la variable status.

Si profundizamos un poco más vemos esto dentro del if:

```
    StringBuilder lpBuffer = new StringBuilder(500);
1   GetEnvironmentVariable("WLRegGetLicenseInfo", lpBuffer, 500);
    Logger.Main.LogString("WLRegGetLicenseInfo", lpBuffer.ToString());
2   string[] strArray = lpBuffer.ToString().Split(new char[] { ',' });
    string s = string.Empty;
3   if (strArray.Length >= 3)
    {
        name = strArray[0];
        company = strArray[1];
4       string[] strArray2 = strArray[2].Split(new char[] { ';' });
5       if (strArray2.Length == 4)
        {
            serialnumber = strArray2[0];
            email = strArray2[1];
            numlic = strArray2[2];
            s = strArray2[3];
        }
6       else if (strArray2.Length == 5)
        {
            serialnumber = strArray2[0];
            email = strArray2[1];
            numlic = strArray2[2];
            s = strArray2[3];
            str = strArray2[4];
7           if (str == "PowerLicense")
            {
                powerLicensePack = true;
            }
        }
    . . .
    }
```

En el punto 1 Obtiene la variable y la pasa a lpBuffer (tamaño máximo 500 chars). En el punto dos divide el contenido de la variable en función de las comas que encuentre.

Por ejemplo, partiendo de esta cadena:

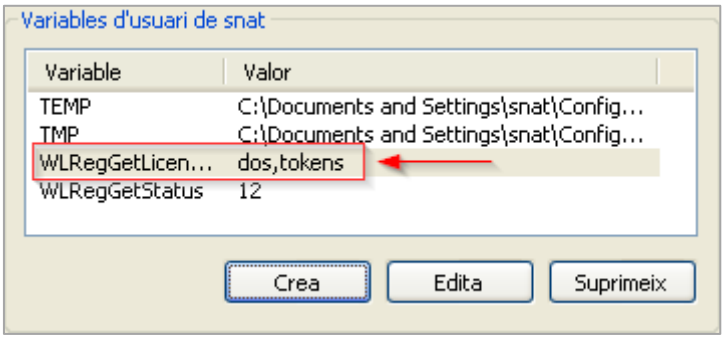
- ✓ "hola,amigo,que,tal,estas"

Obtendrá tantas subcadenas como comas haya en la misma (más 1), en este caso concreto quedaría:

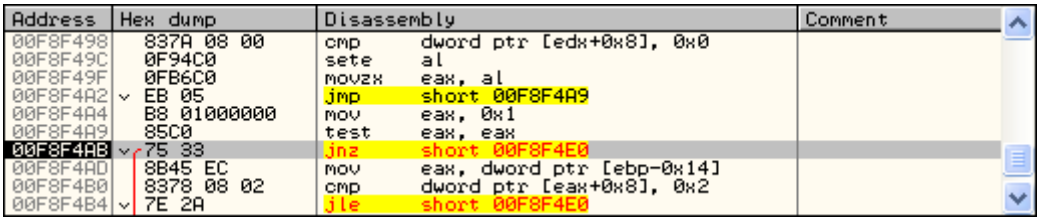
1. hola
2. amigo
3. que
4. tal
5. estas

En total 5 subcadenas de la cadena principal. Similar a la clase StringTokenizer de java (divide la string en substrings o *tokens*).

El punto 3 es muy llamativo porque sólo se accede si la cadena tiene al menos 3 subcadenas (2 comas). Hagamos una prueba y creamos la variable de entorno de esta manera:

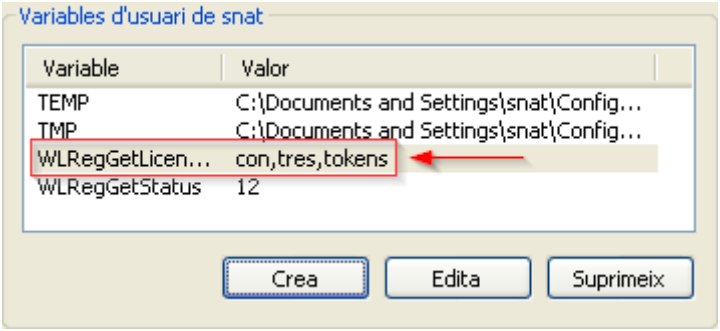



Cerramos y abrimos Olly y cargamos el dumpeado, traceamos la rutina desde 0xF8F417h (casos ESI=0x1h o ESI==0x0Ch), llegamos a este salto que SI se produce:




En este caso EAX==0x01h y el JNZ es tomado.

Cambiemos las variables de entorno, aumentemos el número de subcadenas en uno:



Cerramos Olly y lo volvemos a abrir, llegamos al salto anterior:

```
Address   Hex dump     Disassembly                          Comment
00F8F498  837A 08 00   cmp     dword ptr [edx+0x8], 0x0
00F8F49C  0F94C0       sete    al
00F8F49F  0FB6C0       movzx   eax, al
00F8F4A2  EB 05        jmp     short 00F8F4A9
00F8F4A4  B8 01000000  mov     eax, 0x1
00F8F4A9  85C0         test    eax, eax
00F8F4AB  75 33        jnz     short 00F8F4E0
00F8F4AD  8B45 EC      mov     eax, dword ptr [ebp-0x14]
00F8F4B0  8378 08 02   cmp     dword ptr [eax+0x8], 0x2
00F8F4B4  7E 2A        jle     short 00F8F4E0
```

Ahora NO salta, el JLE siguiente tampoco lo hace y sigue haciendo comprobaciones. Todas ellas las tenemos en el método GetRegistrationInfo, sigamos estudiándolo:

```
4 string[] strArray2 = strArray[2].Split(new char[] { ';' });
5 if (strArray2.Length == 4)
  {
   // ****
  }
```

- ✓ El punto 4 vuelve a dividir el tercer token (strArray[2]) en substrings, esta vez separadas por ';'.
- ✓ El punto 5 mira si la longitud del tercer token es igual a 4, en cuyo caso asigna estos subtokens:
  - token 1: SerialNumer
  - token 2: email
  - token 3: número de licencias
  - token 4: fecha en formato anglosajón (yyyyMMdd)

Después viene este código:

```
6 else if (strArray2.Length == 5)
  {
      serialnumber = strArray2[0];
      email = strArray2[1];
      numlic = strArray2[2];
      s = strArray2[3];
      str = strArray2[4];
  7   if (str == "PowerLicense")
      {
          powerLicensePack = true;
      }
  }
```

- El punto 6 revisa si la longitud del tercer token es igual a 5. En caso afirmativo asigna las mismas variables que en el punto 5 pero además lee un último token:
  - token 5: cadena PowerLicense (verificado en el punto 7).

Si encuentra este valor asigna TRUE a la variable booleana powerLicensePack.

Por último encontramos un try-catch:

```
try {
   subscribeExpireDate = DateTime.ParseExact(s, "yyyyMMdd", invariantCulture);
}
catch (FormatException exception)
{
    // ****
}
```

Esta estructura es equivalente a instalar un manejador de excepción, se ejecutará el bloque k (el código susceptible de generar una excepción) que será tratada en el bloque *catch* (en caso de suceder). Pero lo importante aquí es darse cuenta que está procesando el sexto token (token6) de la variable estudiada WLRegGetLicenseInfo

Entonces esta variable (WLRegGetLicenseInfo) deberá tener esta estructura:

- token1,token2,token3;token4;token5;token6;[token7(opcional)]

Que vendría a ser:

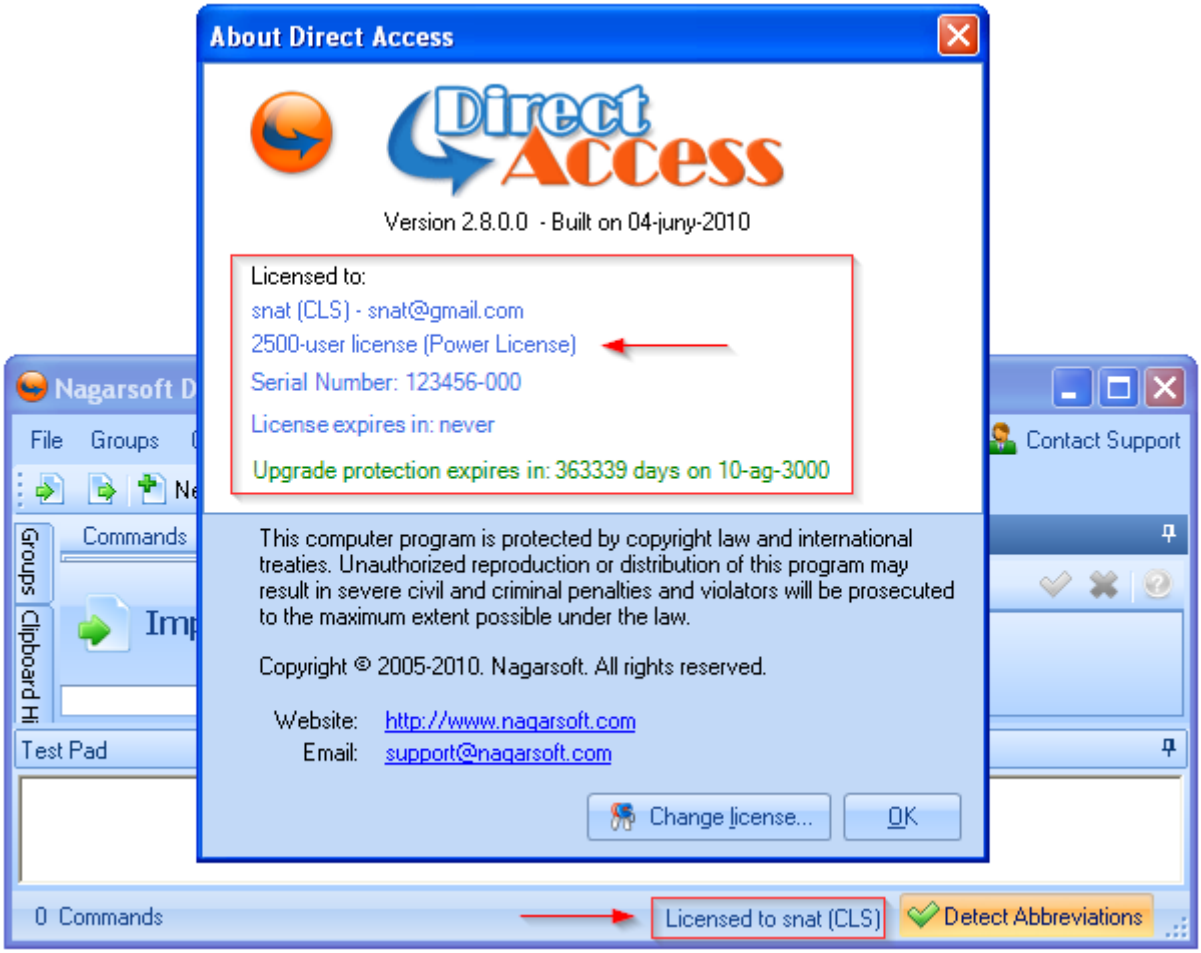
- Name,Company,SerialNumber;email;numeroLicencias;Fecha(yyyyMMdd);[opcional PowerLicense]

Siendo el token7 opcional.

Hagamos una prueba y coloquemos esta:

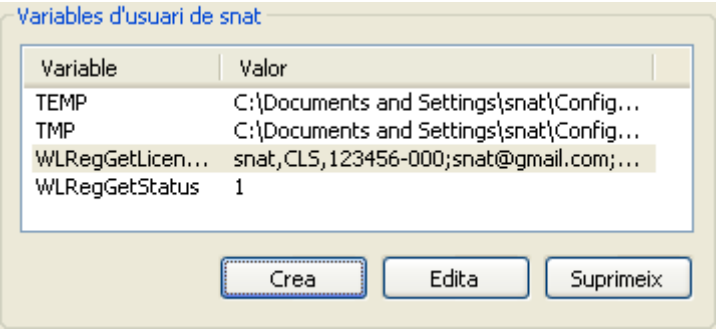
✓ snat,CLS,123456-000;snat@gmail.com;2500;30000810;PowerLicense

Creamos la variable y ejecutamos el programa:




Con esto NO debería dar más problemas.

Las variables de entorno quedan así:




1. Adelantamos la fecha del reloj hasta Enero de 2098 y lo probamos: Funciona
2. Atrasamos la Fecha a Octubre de 1980, lo ejecutamos y también funciona.
3. Movemos el dumpeado a Windows 7 x32 bit y también funciona.

Nunca hay nada seguro, pero esta vez parece que estamos a salvo.

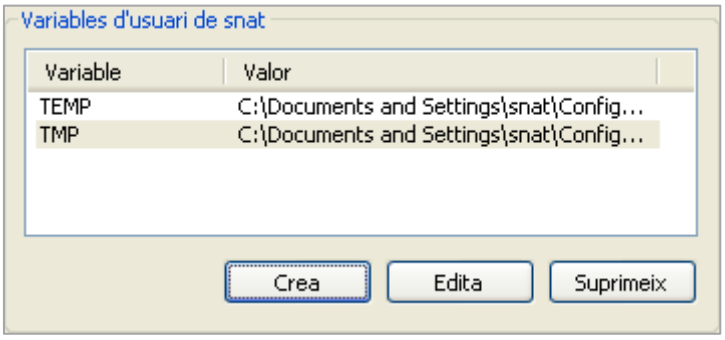
Nos queda una última modificación en el proyecto de RadASM para que cree la segunda variable de entorno, agregamos estas líneas al archivo DirectLoader.Inc:

```
30 nombre_variable2    db 'WLRegGetLicenseInfo',0
31 valor_variable2     db 'snat,CLS,123456-000;snat@gmail.com;2500;30000810;PowerLicense',0
```

Y en DirectLoader.Asm esta otra:

```
19  invoke SetEnvironmentVariable,addr nombre_variable2,addr valor_variable2
```

Recompilamos el proyecto, desestablecemos las variables de entorno:




Lo probamos y funciona perfecto. Hemos terminado :)

NOTA: Las Notas Finales ya están escritas, puesto que las partes 13 y 14 se han escrito después (tras encontrar el segundo error), por lo que aprovecho para decir lo mucho que he disfrutado al escribir este texto y que espero que todos vosotros lo hagáis también al leerlo, no puedo agregar MASM32 ni el proyecto completo por motivos de peso de archivos. Pero agrego MegaDumper y Universal Fixer con permiso de SnD (si me lo concede) para que podáis sacar un dumpeado para agregarlo a la carpeta del proyecto de RadASM y recompilar sin problemas ya que agrego el Instalador al texto.

# 15. Notas Finales

Este es un principio, por mi parte todavía no había trabajado nada en la plataforma .NET, tal como se ha podido comprobar todavía nos queda mucho camino por recorrer en este sentido, es como todo, hay que adaptarse como dijo Darwin en sus teorías, supongo que con el transcurso del tiempo y con algo más de práctica podremos desvelar auténticos misterios como por qué se necesita el JMP del OEP si no se utiliza? Por qué cambia el valor del AddressOfEntryPoint del header? Quién lo modifica? Recordad que este valor era diferente INCLUSO en el System BreakPoint cuando ni si quiera la mscoree.dll estaba cargada. Son muchos los aspectos a investigar.

A todo esto, hemos podido ver cómo un súper-packer como Themida se queda casi sin armas a la hora de proteger al .NET, no puede instalar su MV como sí hacía en C/C++ donde hacía verdaderas peripecias, aquí NO puede hacer NADA de esto, lo único que puede hacer es insertar algunos métodos en el Assembly para las comprobaciones adicionales, todas las que realizaban a través de sus FV# como vimos en el anterior texto, estas comprobaciones, a su vez las hace en zonas fuera del ejecutable (pues no ejecuta código, se interpreta), por lo que se decantó por las variables de entorno. Realmente a través de un método podría comprobar otras cosas, si no lo ha hecho por algo será.

Por otro lado, se ha intentado mostrar técnicas que solemos utilizar habitualmente, lo fácil era estudiar un poco más el Assembly, encontrar la zona donde decide si el proceso debe ejecutarse o no, buscar los bytes del salto con cualquier editor hexadecimal y parchear el binario. Todo esto está muy bien, pero ya que se trata de un tutorial (y no un trabajo con un fin únicamente personal) siempre es de agradecer cualquier método alternativo que se pueda ofrecer, cuestión de gustos claro.

Por mi parte quedo realmente contento de haber podido terminar este pequeño proyecto, he de decir que en un primer momento no pensé en escribir el tutorial, más que nada porque no encontraba nada nuevo que ofrecer, TODAS las técnicas aquí mostradas ya han sido ampliamente utilizadas y no aportan novedad alguna, es una metodología que todos nosotros conocemos bien. Fue a partir de los correos enviados a la lista por parte de miembros de nuestro grupo (Apokliptico y sequeyo) por los que realmente me animé a escribir. He de agradecérselo a ambos. Además tengo un agradecimiento especial para sequeyo quién me ayudó en este mismo hilo, un saludo amigo :)

También decidí escribir este tutorial porque se trata de un .NET, era realmente un desafío para mí, además de complementar a los otros dos tutoriales de Themida en cuanto a compilador, tenemos el Visual Basic, el Visual C++ y ahora el .NET, por esta razón pensé que podría ser una buena idea.

Y no me quiero despedir sin AGRADECER a TODAS aquéllas personas que escriben tutoriales para que los demás podamos seguir progresando, así como a los autores de las herramientas aquí utilizadas: °£·çôÂó°, Hellsp@wn & Archer, CodeCracker / SND, Ketil0, Daniel Pistelli, Epsylon, Gigapede y RDGMax.

Y por supuesto un gran saludo para TODOS los miembros del grupo CrackSLatinoS con los que me encuentro cada día mejor. Gracias por todo y hasta la próxima :)

CrackSLatinoS
Snat.~